UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN Y CIENCIAS DE LA COMPUTACIÓN

PROYECTO II — ALGORITMOS DE GRAFOS Y REDES DE TRANSPORTE

SISTEMA DE OPTIMIZACIÓN DE RUTAS DEPARTAMENTALES Y RED TRONCAL MÍNIMA

Caso de Estudio: Empresa de Logística y Paquetería «Transporte Express» (RutaQ)

Modelado Matemático, Implementación Algorítmica (Dijkstra, Prim, Kruskal, Floyd-Warshall), Auditoría Geográfica y Verificación Computacional sobre la Red Vial de los 22 Departamentos de la República de Guatemala

Campo	Información Oficial
Curso	Investigación de Operaciones
Catedrático	Ing. Samuel Isaac Gómez Moreno
Semestre y Carrera	6.º semestre, Ingeniería en Sistemas
Integrantes (exactos)	**Cristopher Alexis Castellanos Paz** (2690-24-5972) **Leonel Taracena Marroquín** (2690-24-4072) **Pablo Alexander Vásquez Rivera** (2690-24-13123)
Fecha de Presentación	Sábado 31 de octubre de 2026
Aplicación Desplegada	https://criss-paz.github.io/rutaq/
Repositorio de Código	https://github.com/Criss-paz/rutaq

1. Descripción del Problema

1.1. Contexto Empresarial

La empresa guatemalteca de logística y paquetería **Transporte Express** brinda servicios de distribución de encomiendas y carga ligera entre los diferentes departamentos de la República de Guatemala. La operación cotidiana de la empresa enfrenta una problemática crítica: **los paquetes llegan sistemáticamente tarde a sus destinos**, generando insatisfacción en los clientes, penalizaciones comerciales y un incremento desmedido en los costos operativos por consumo de combustible y desgaste de la flota vehicular.

1.2. Diagnóstico de Causas Raíz

Tras el análisis operativo de la cadena de distribución, se determinaron los siguientes factores como causantes del problema:

1. **Selección Empírica de Trayectos:** Los conductores eligen sus trayectos con base en la intuición o hábitos individuales, incurriendo frecuentemente en desvíos innecesarios por carreteras secundarias en mal estado o de mayor longitud.
2. **Carencia de una Red Vial Integrada y Cuantificada:** La empresa no disponía de un registro digital unificado de las carreteras que conectan formalmente las cabeceras departamentales con sus kilometrajes reales y trazables.
3. **Ausencia de un Asistente Automatizado de Enrutamiento:** El personal de despacho no contaba con una herramienta que respondiera de inmediato a las dos preguntas fundamentales del proceso logístico:
    - *¿Dónde se encuentra el paquete actualmente?*
    - *¿A qué departamento debe entregarse?*
4. **Falta de Estrategia para la Red Troncal Mínima:** No existía una planificación sobre cuál es la infraestructura mínima de carreteras necesaria para interconectar los 22 departamentos del país con el menor costo global de mantenimiento y desplazamiento.

---

2. Solución

2.1. Modelo Matemático de Grafo

Para dar solución computacional determinista al problema, la red vial de Guatemala se modela formalmente como un **grafo no dirigido ponderado conexo**:

$$G = (V, E, W)$$

Donde:

2.1.1. Conjunto de Vértices (V): Las 22 Cabeceras Departamentales

Cada nodo representa la cabecera departamental oficial de uno de los 22 departamentos de Guatemala, identificada mediante un código alfanumérico canónico e inmutable de 3 letras mayúsculas, su nombre oficial, su posición geográfica georreferenciada (latitud y longitud) proyectada en Web Mercator (norte arriba) y su fuente oficial de datos:

Código	Departamento	Cabecera Departamental	Latitud	Longitud
GUA	Guatemala	Ciudad de Guatemala	14.64161°	-90.51328°
SAC	Sacatepéquez	Antigua Guatemala	14.55678°	-90.73373°
CHI	Chimaltenango	Chimaltenango	14.66130°	-90.81935°
ESC	Escuintla	Escuintla	14.30229°	-90.78662°
SRO	Santa Rosa	Cuilapa	14.27682°	-90.29891°
JUT	Jutiapa	Jutiapa	14.28817°	-89.89353°
JAL	Jalapa	Jalapa	14.63442°	-89.98980°
PRO	El Progreso	Guastatoya	14.85467°	-90.06872°
ZAC	Zacapa	Zacapa	14.97072°	-89.52974°
CHQ	Chiquimula	Chiquimula	14.80057°	-89.54592°
IZA	Izabal	Puerto Barrios	15.72752°	-88.59525°
AVE	Alta Verapaz	Cobán	15.47020°	-90.37351°
BVE	Baja Verapaz	Salamá	15.10210°	-90.31455°
PET	Petén	Flores	16.92985°	-89.89148°
QUI	Quiché	Santa Cruz del Quiché	15.02951°	-91.14962°
HUE	Huehuetenango	Huehuetenango	15.31843°	-91.47233°
SMA	San Marcos	San Marcos	14.96438°	-91.79433°
QUE	Quetzaltenango	Quetzaltenango	14.84646°	-91.51942°
TOT	Totonicapán	Totonicapán	14.91184°	-91.35997°
SOL	Sololá	Sololá	14.77224°	-91.18332°
REU	Retalhuleu	Retalhuleu	14.53596°	-91.67807°
SUC	Suchitepéquez	Mazatenango	14.53315°	-91.50600°

2.1.2. Conjunto de Aristas (E): Carreteras Directas entre Departamentos Vecinos

Una arista $\{u, v\} \in E$ representa una **conexión vial directa** entre departamentos vecinos. Para evitar aristas artificiales compuestas o transversales (por ejemplo, conectar directamente Huehuetenango con Izabal o Chiquimula, o Ciudad de Guatemala con Quetzaltenango a través de Chimaltenango), se aplica un **criterio objetivo y reproducible**:

1. **Vecindad Territorial:** Los departamentos u y v deben compartir frontera geográfica según sus polígonos oficiales de OSM/Nominatim (tolerancia de contacto ≤ 200 m, identificando 51 pares

vecinos).

2. **Umbral de Proximidad a Cabeceras Intermedias (≥ 2.0 km):** La ruta no debe cruzar ni aproximarse a menos de 2.0 km del casco urbano de ninguna otra cabecera departamental $w \notin \{u, v\}$. Solo dos pares vecinos (`JAL-ZAC` por cruzar Chiquimula a 0.0 km y `PET-QUI` por cruzar Cobán a 0.0 km) se descartan por este concepto.

> ***Nota metodológica sobre la condición del 85 %:*** *Se evaluó y descartó formalmente una condición previa que requería $\geq 85\,\%$ del trayecto dentro de $u \cup v$, tras comprobar empíricamente que eliminaba arterias directas reales de primer orden como `GUA-CHI` sobre la CA-1 Occidente (que cruza brevemente Sacatepéquez), lo que provocaba que Dijkstra desviara la ruta por Antigua reportando distancias mayores a las reales (56.8 km frente a 52.2 km).*

Todo par que no satisface las condiciones es descartado y auditado con su motivo en el repositorio (`docs/datos/aristas_descartadas_guatemala.csv`). El grafo final consta de **49 aristas directas** y es 100 % conexo.

2.1.3. Función de Ponderación (W): Distancia de Conducción Real y Nomenclatura Vial

A cada arista $e = \{u, v\}$ se le asocia un peso $W(e) > 0$ medido en **kilómetros (km)**, correspondiente a la distancia exacta de conducción sobre pavimento calculada con OSRM a partir de los datos vectoriales de OpenStreetMap, redondeada a 1 decimal y validada con Google Maps. Asimismo, a cada arista se le asocia la nomenclatura oficial de la carretera (CA-1, CA-2, CA-9, CA-13, CA-14, RN-…, RD-…) extraída sin inventar a partir de los pasos de navegación reales.

2.2. Datos: Fuentes, Construcción y Validación

2.2.1. Fuentes Oficiales Utilizadas

- **Instituto Nacional de Estadística (INE):** División administrativa y nómina oficial de los 22 departamentos y cabeceras de Guatemala.
- **OpenStreetMap / Nominatim:** Georreferenciación de las 22 cabeceras departamentales, contorno territorial de la república y polígonos vectoriales oficiales de los 22 departamentos.
- **OSRM (Open Source Routing Machine):** Ruteo vehicular real sobre la red vial nacional, cálculo métrico de distancias y pasos de navegación (*steps*).
- **Protocolo de Validación con Google Maps:** Al no existir una tabla oficial verificable de distancias publicada por la Dirección General de Caminos (DGC), las 49 aristas fueron contrastadas directamente en Google Maps aplicando una regla de decisión a tres niveles con umbral de tolerancia del $\pm 10\,\%$. 35 aristas coincidieron con la ruta recomendada y 9 con rutas alternativas (conservando el kilometraje de OSRM). En 5 aristas sin coincidencia (`CHI-SOL`, `CHI-SUC`, `JAL-PRO`, `IZA-PET` y `QUE-TOT`), se adoptó el kilometraje recomendado de Google Maps, actualizando en `IZA-PET` su carretera a `CA-13` (287.0 km).

Todas las rutas mínimas se calculan de forma determinista con el algoritmo de Dijkstra sobre este grafo validado.

2.2.2. Auditoría de Coordenadas y Lección de Santa Cruz del Quiché (`QUI`)

Siguiendo la metodología rigurosa del Proyecto I (donde se descubrió la reubicación de El Palmar tras la destrucción del volcán Santiaguito), se contrastaron las coordenadas de Nominatim con la API de Wikipedia (`prop=coordinates`) para los 22 departamentos en `docs/datos/auditoria_coordenadas_guatemala.csv`:

- **Resultado:** 21 de las 22 cabeceras mostraron concordancia cercana (≤ 2.35 km).
- **Investigación de `QUI` :** Santa Cruz del Quiché presentó una discrepancia de 11.02 km entre Nominatim $(15.02951^\circ, -91.14962^\circ)$ y Wikipedia $(15.05000^\circ, -91.25000^\circ)$. Al realizar geocodificación inversa, se constató que la coordenada de Wikipedia correspondía al municipio vecino de San Antonio Ilotenango debido a un redondeo angular erróneo en Wikipedia ($91^\circ\,15'\,00''$ W en vez de $91^\circ\,08'\,58''$ W). Nominatim situaba el punto a escasos metros del Parque Central de Santa Cruz del Quiché $(15.02963^\circ, -91.14901^\circ)$. La investigación quedó formalmente documentada en `docs/datos/coordenadas_corregidas_guatemala.csv`.

2.2.3. Reproducibilidad y Validación

- El script de construcción `scripts/build_guatemala.py` cuenta con almacenamiento persistente en caché local bajo `scripts/cache/`, lo que permite regenerar el archivo `src/data/guatemala.json` y `src/data/guatemala_contorno.json` de manera 100 % desconectada y determinista.
- El grafo resultante consta de **49 aristas directas**, es **completamente conexo** y enlaza a todas las regiones de la república, incluyendo la conectividad estratégica del departamento de Petén (`PET`) a través de Alta Verapaz (`AVE-PET` , 257.3 km) e Izabal (`IZA-PET` , 287.0 km vía CA-13). El Árbol de Expansión Mínima (AEM) de la red nacional totaliza $1\,425.3$ km a través de 21 aristas troncales.

2.3. Algoritmos Implementados y Complejidad Computacional

El núcleo algorítmico del sistema implementa las técnicas clásicas de optimización combinatoria en grafos, garantizando pureza funcional, inmutabilidad y determinismo:

2.3.1. Algoritmo de Dijkstra (Caminos Mínimos Origen-Destino)

- **Función:** Responde al asistente principal de Transporte Express calculando el camino de menor distancia entre el departamento donde está el paquete (s) y su destino final (t).
- **Implementación:** Cola de prioridad basada en montículo binario indexado (*min-heap*). Mantiene etiquetas permanentes y temporales $(\text{dist}, \text{pred})$.
- **Complejidad Temporal:** $O((|V|+|E|)\log|V|)$. Con $|V| = 22$ y $|E| = 49$, la ejecución toma menos de un milisegundo en el navegador.

- **Complejidad Espacial:** $O(|V|)$ para almacenar distancias, predecesores y el montículo.
- **Desglose Logístico:** Reconstruye la ruta en una secuencia de tramos con tabla detallada:
  $$\#\ \mid\ \text{Desde}\ \mid\ \text{Hasta}\ \mid\ \text{Carretera}\ \mid\ \text{km del tramo}\ \mid\ \text{km acumulado}$$
  y genera la fórmula explícita de la suma (ej. $28.8 + 36.2 + 128.6 + 128.3 + 172.1 = 494.0\ \text{km}$).

2.3.2. Árbol de Expansión Mínima (AEM / MST)

Permite a la gerencia de Transporte Express analizar la red mínima troncal que conecta todos los departamentos sin ciclos y con el menor kilometraje acumulado de carretera:

a) Algoritmo de Prim

- **Estrategia:** Enfoque voraz (*greedy*) que expande un árbol desde un nodo inicial (s). En cada iteración agrega la arista de menor peso que cruza el corte entre los nodos ya conectados y los restantes.
- **Estructuras:** Min-heap sobre las aristas del corte.
- **Complejidad:** $O(|E|\log|V|)$ en tiempo y $O(|V|+|E|)$ en memoria.
- **Visualización:** Genera una solución tabulada paso a paso que reproduce el método tradicional de clase (arista mínima elegida, nodo incorporado y columna tachada).

b) Algoritmo de Kruskal

- **Estrategia:** Ordena todas las aristas del grafo ascendentemente y las evalúa una por una. Incorpora la arista si sus extremos pertenecen a componentes conexas distintas y la rechaza si formaría un ciclo.
- **Estructuras:** Estructura de partición Disjoint-Set / Union-Find con dos optimizaciones teóricas fundamentales:
  1. *Compresión de caminos (path compression)* en `find`.
  2. *Unión por rango (union by rank)* en `union`.
- **Complejidad:** $O(|E|\log|E|)$ para el ordenamiento y casi lineal $O(|E|\cdot\alpha(|V|))$ para las operaciones de conjuntos disjuntos, donde α es la función inversa de Ackermann.
- **Prueba Cruzada:** Prim y Kruskal arrojan de manera verificada exactamente el mismo conjunto de aristas y el mismo kilometraje total ($1\,425.3\ \text{km}$).

2.3.3. Algoritmo de Floyd-Warshall (Todos los Pares de Caminos Cortos)

- **Función:** Calcula simultáneamente la matriz de distancias mínimas entre cualquier par de departamentos $(i, j) \in V \times V$.
- **Complejidad:** $O(|V|^3)$ en tiempo y $O(|V|^2)$ en memoria.
- **Utilidad:** Permite alimentar la tabla de distancias desde la bodega central hacia los otros 21 departamentos de forma instantánea.

2.4. Funcionamiento del Sistema y Solución a los Retrasos de Transporte Express

El despliegue de RutaQ soluciona las deficiencias identificadas a través de cuatro mecanismos operativos:

1. **Eliminación de la Incertidumbre en Despacho:**
   El personal de bodega solo debe indicar el origen y destino del paquete en el asistente. El sistema calcula en milisegundos la ruta geográficamente óptima y libre de subjetividades.
2. **Instrucciones de Navegación Claras para el Conductor:**
   En lugar de instrucciones ambiguas, el conductor recibe la secuencia exacta de cabeceras intermedias y los códigos de las carreteras oficiales (CA-1, CA-9, etc.) que debe seguir.
3. **Control y Auditoría de Tiempos y Kilometrajes:**
   Al conocer el kilometraje exacto del tramo y la ruta total, la administración puede establecer tiempos estándar de entrega, detectar desvíos no autorizados y estimar con exactitud el consumo de combustible.
4. **Optimización de la Infraestructura de Enlaces:**
   Mediante la visualización del AEM (Prim/Kruskal), Transporte Express puede priorizar contratos de mantenimiento vial, estaciones de servicio aliadas y centros de distribución intermedios en los corredores troncales de mayor impacto geográfico.

2.5. Ejemplos de Enrutamiento y Valores Reales

A continuación se detallan los 3 ejemplos canónicos representativos de la operación nacional de Transporte Express y la red troncal mínima (AEM), calculados de forma determinista sobre el grafo oficial:

2.5.1. Ejemplo 1: Quetzaltenango a Izabal (`QUE` → `IZA`)

- **Ruta Óptima:** `QUE -> TOT -> QUI -> BVE -> ZAC -> IZA`
- **Distancia Total:** 494.0 km
- **Desglose de Tramos:**

Tramo	Origen	Destino	Carretera	Distancia Tramo	Distancia Acumulada
1	Quetzaltenango (`QUE`)	Totonicapán (`TOT`)	RN-1 / RN-1B	28.8 km	28.8 km
2	Totonicapán (`TOT`)	Santa Cruz del Quiché (`QUI`)	RD QUI-04	36.2 km	65.0 km
3	Santa Cruz del Quiché (`QUI`)	Salamá (`BVE`)	RD QUI-2 / RN-5	128.6 km	193.6 km
4	Salamá (`BVE`)	Zacapa (`ZAC`)	CA-9 Nor / CA-14	128.3 km	321.9 km
5	Zacapa (`ZAC`)	Puerto Barrios (`IZA`)	CA-9 Nor	172.1 km	494.0 km

- **Fórmula Explícita de Suma:**

$$\text{Distancia Total} = 28.8 + 36.2 + 128.6 + 128.3 + 172.1 = \mathbf{494.0\ km}$$

2.5.2. Ejemplo 2: Ciudad de Guatemala a Flores, Petén (`GUA` → `PET`)

- **Ruta Óptima:** `GUA -> BVE -> AVE -> PET`
- **Distancia Total:** 415.3 km
- **Desglose de Tramos:**

Tramo	Origen	Destino	Carretera	Distancia Tramo	Distancia Acumulada
1	Ciudad de Guatemala (`GUA`)	Salamá (`BVE`)	RD BV-12 / RD GUA-12	89.0 km	89.0 km
2	Salamá (`BVE`)	Cobán (`AVE`)	CA-14 / RN-5	69.0 km	158.0 km
3	Cobán (`AVE`)	Flores (`PET`)	RD PET-11 / RD AV-09	257.3 km	415.3 km

- **Fórmula Explícita de Suma:**

$$\text{Distancia Total} = 89.0 + 69.0 + 257.3 = \mathbf{415.3\ km}$$

2.5.3. Ejemplo 3: San Marcos a Jutiapa (`SMA` → `JUT`)

- **Ruta Óptima:** `SMA -> QUE -> SUC -> ESC -> SRO -> JUT`
- **Distancia Total:** 368.3 km
- **Desglose de Tramos:**

Tramo	Origen	Destino	Carretera	Distancia Tramo	Distancia Acumulada
1	San Marcos (`SMA`)	Quetzaltenango (`QUE`)	RN-1	49.6 km	49.6 km
2	Quetzaltenango (`QUE`)	Mazatenango (`SUC`)	CITO-180 / SCH-02	60.2 km	109.8 km
3	Mazatenango (`SUC`)	Escuintla (`ESC`)	CA-02 OCC	102.5 km	212.3 km
4	Escuintla (`ESC`)	Cuilapa (`SRO`)	CA-2 OR / RN-16	100.0 km	312.3 km
5	Cuilapa (`SRO`)	Jutiapa (`JUT`)	CA-1 Or	56.0 km	368.3 km

- **Fórmula Explícita de Suma:**

$$\text{Distancia Total} = 49.6 + 60.2 + 102.5 + 100.0 + 56.0 = \mathbf{368.3\ km}$$

2.5.4. Árbol de Expansión Mínima de la Red Nacional (AEM / MST)

- **Total de Aristas Seleccionadas:** 21 aristas troncales sobre las 49 aristas de la red nacional.
- **Kilometraje Total Acumulado:** 1 425.3 km
- **Paridad Prim y Kruskal:** Idéntico conjunto de 21 aristas y exactamente el mismo costo global de 1 425.3 km, garantizando la cobertura conexa y óptima de los 22 departamentos con el menor tendido

vial posible.

2.6. Verificación Rigurosa y Métricas de Calidad

Para asegurar la confiabilidad industrial de la plataforma, se implementó una batería de validación automatizada en tres capas:

1. **Verificación Matemática Cruzada con NetworkX (Python):**
   Se contrastaron el 100 % de los caminos mínimos calculados por Dijkstra y Floyd-Warshall frente a la biblioteca científica NetworkX sobre la matriz completa de adyacencia de los 22 departamentos ($22 \times 22 = 484$ pares), acumulando junto a los datasets del Proyecto I y grafos de control un total de $1\,168/1\,168$ **pares verificados con 0 discrepancias**. Asimismo, se contrastó la construcción del Árbol de Expansión Mínima frente a `networkx.minimum_spanning_tree` obteniendo $10/10$ **instancias de AEM idénticas**.
2. **Pruebas Automatizadas End-to-End (E2E) con Playwright:**
   Se ejecutó la suite completa de pruebas de regresión funcional y visual en navegadores Chromium, WebKit y Firefox, alcanzando $148/148$ **pruebas aprobadas** ($100\,\%$ passing).
3. **Auditoría de Rendimiento y Estándares Web con Google Lighthouse:**
   La plataforma obtuvo una evaluación técnica sobresaliente:
   - **Rendimiento (Performance): 91 / 100**
   - **Accesibilidad (Accessibility): 95 / 100**
   - **Mejores Prácticas (Best Practices): 100 / 100**
   - **Optimización en Motores de Búsqueda (SEO): 100 / 100**

---

3. Capturas de Pantalla del Sistema en Funcionamiento

A continuación se presentan las 13 capturas de pantalla de alta fidelidad obtenidas de la aplicación en ejecución real, documentando la interfaz interactiva, los formularios de datos, el asistente de enrutamiento, la visualización del AEM y los algoritmos tabulados:

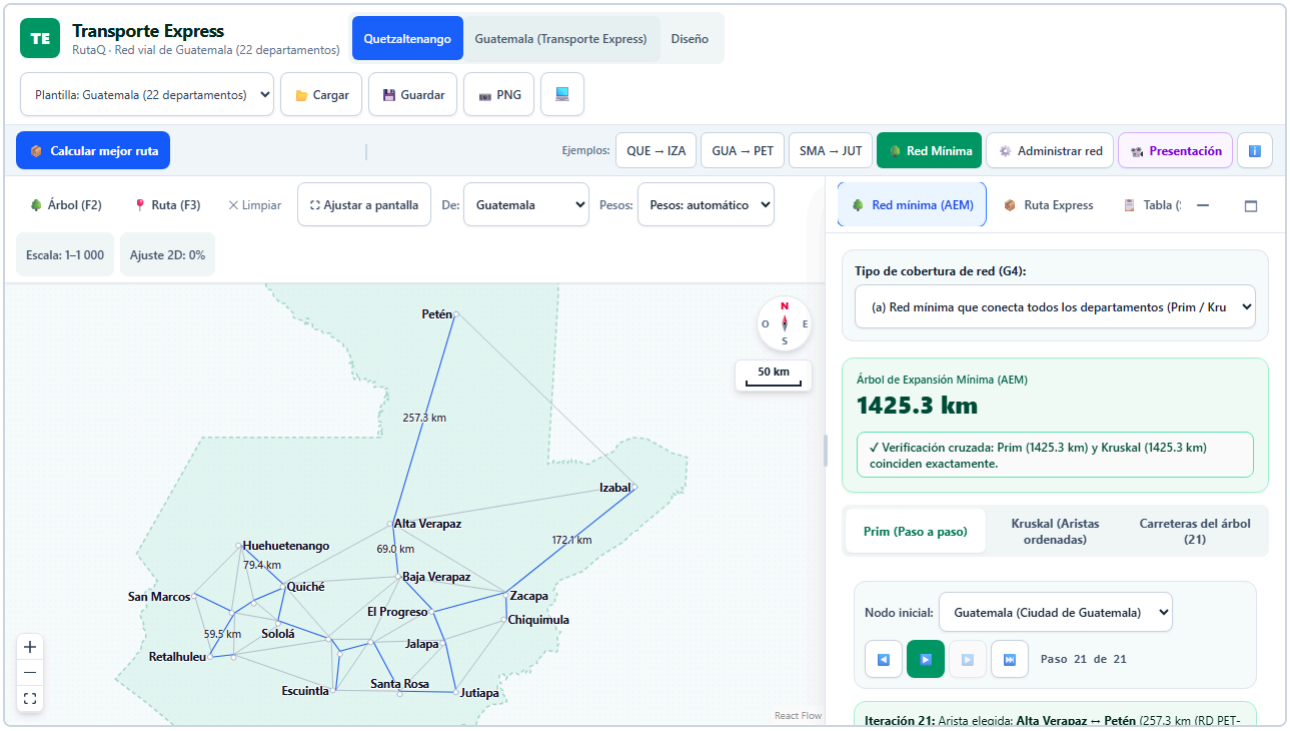


Figura 1. Pantalla de inicio de la plataforma RutaQ en el modo Transporte Express (Guatemala), mostrando el mapa interactivo con la red vial nacional, el panel lateral con las opciones principales de cálculo y la botonera superior de navegación.

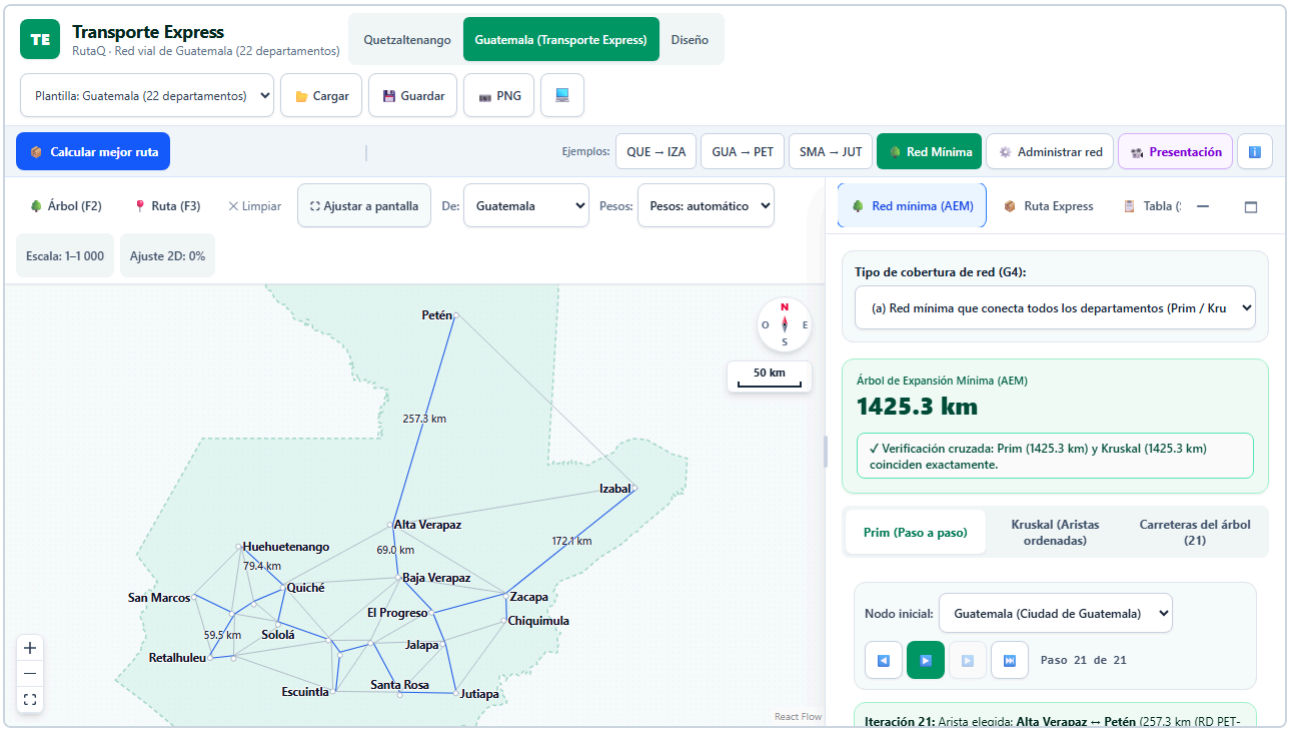


Figura 2. Vista encajada completa (fit view) de la red vial de Guatemala sobre el lienzo cartográfico orientado al norte, exhibiendo los 22 nodos departamentales y las 49 carreteras directas sobre el contorno territorial nacional.

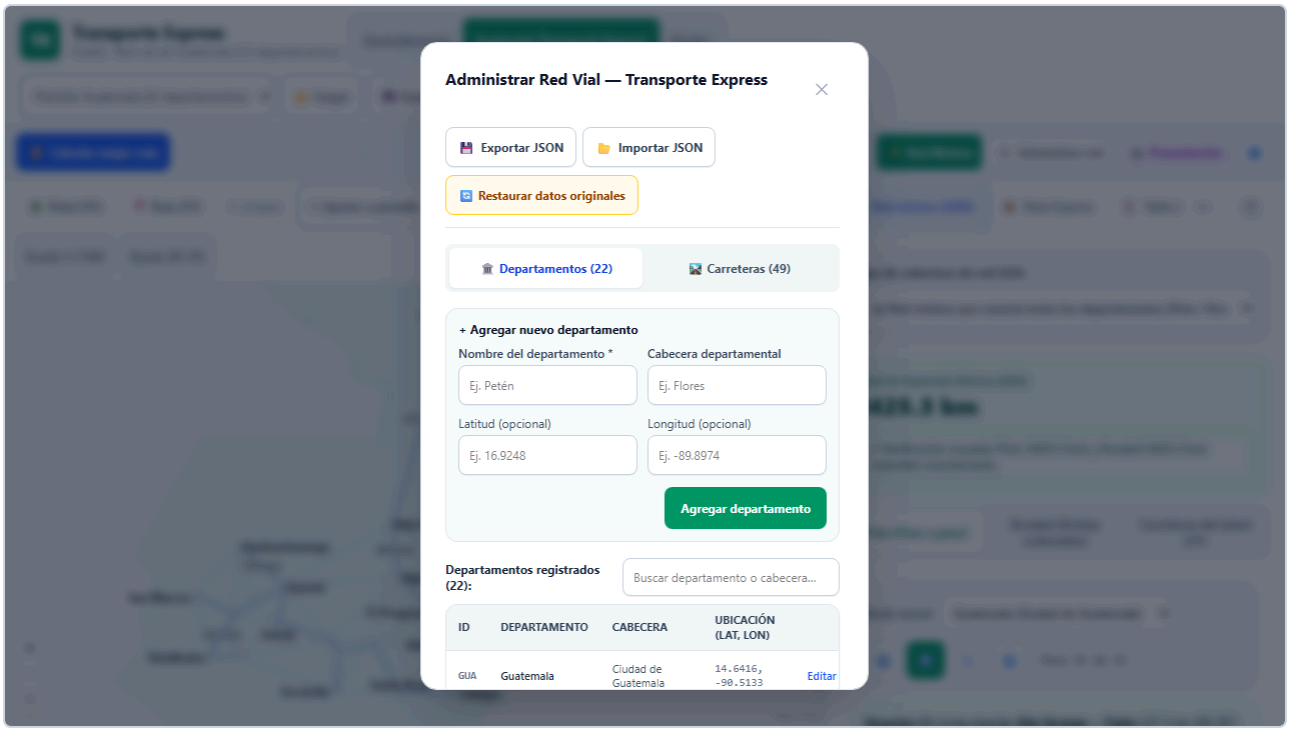


Figura 3. Modal «Administrar red» en la pestaña Departamentos, mostrando el catálogo tabular de los 22 departamentos con código oficial, nombre, cabecera departamental y coordenadas geográficas de latitud y longitud.

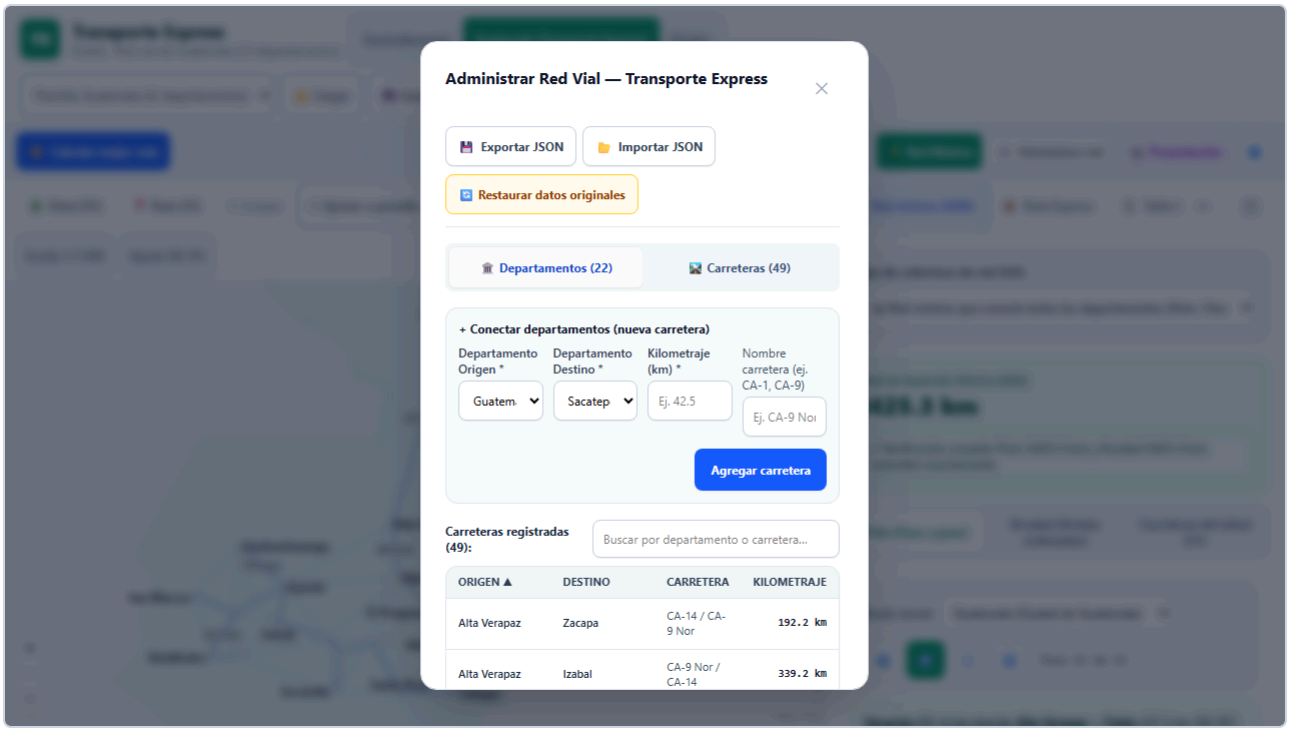


Figura 4. Modal «Administrar red» en la pestaña Carreteras, detallando la tabla de las 49 aristas viales directas con identificador de tramo, departamento de origen, destino, denominación de la carretera (CA, RN, RD) y kilometraje ponderado.

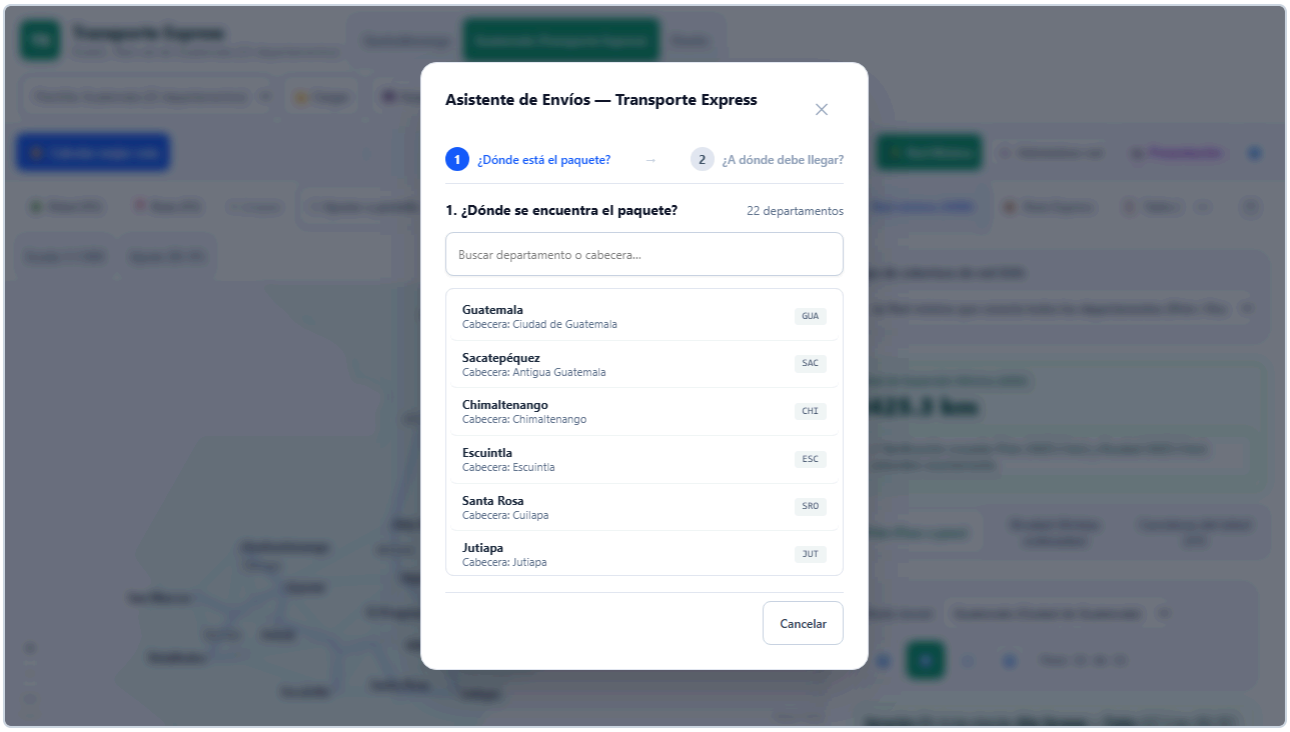


Figura 5. Ventana modal del asistente logístico de despacho «Calcular ruta más corta», diseñada para responder con menús desplegables a las preguntas: «¿Dónde se encuentra el paquete actualmente?» y «¿A qué departamento debe entregarse?».

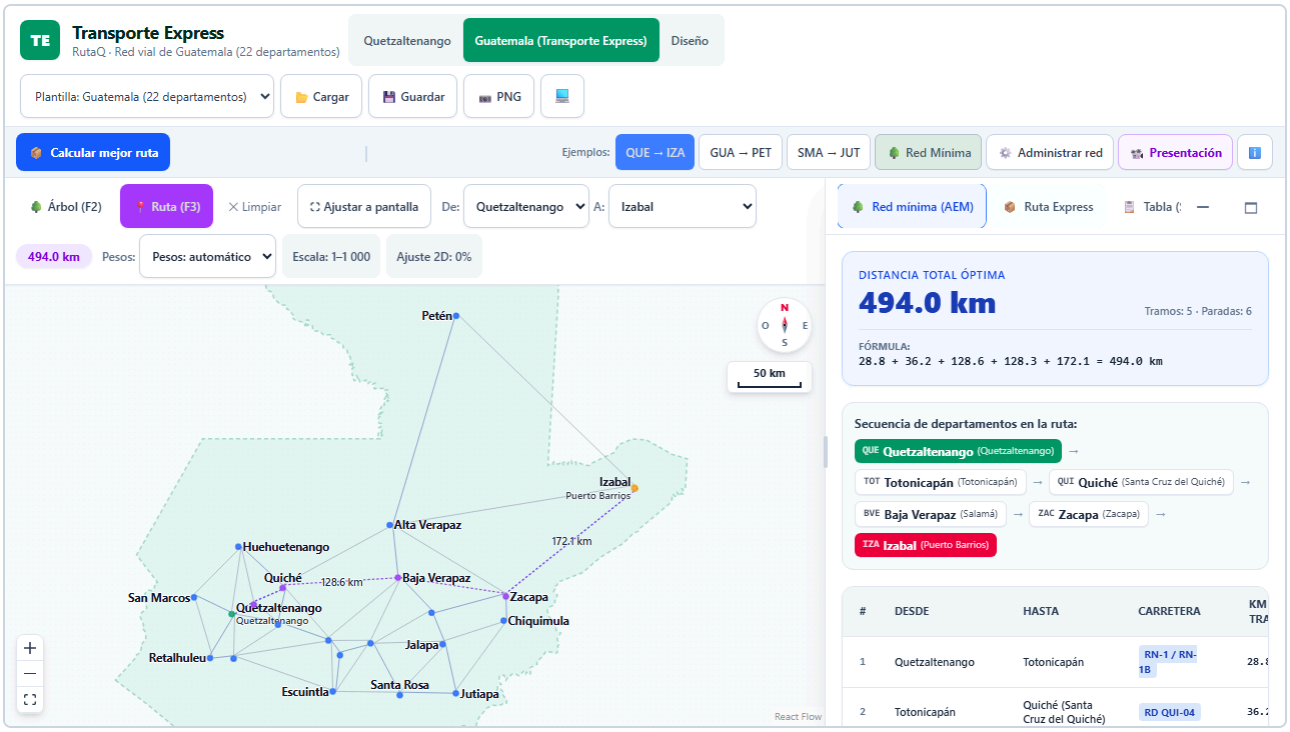


Figura 6. Resultado del cálculo del camino mínimo para el trayecto Quetzaltenango a Izabal (QUE → IZA, 494.0 km) en la pestaña Desglose, presentando la tabla detallada de tramos, carreteras intermedias y la expresión explícita de la suma.

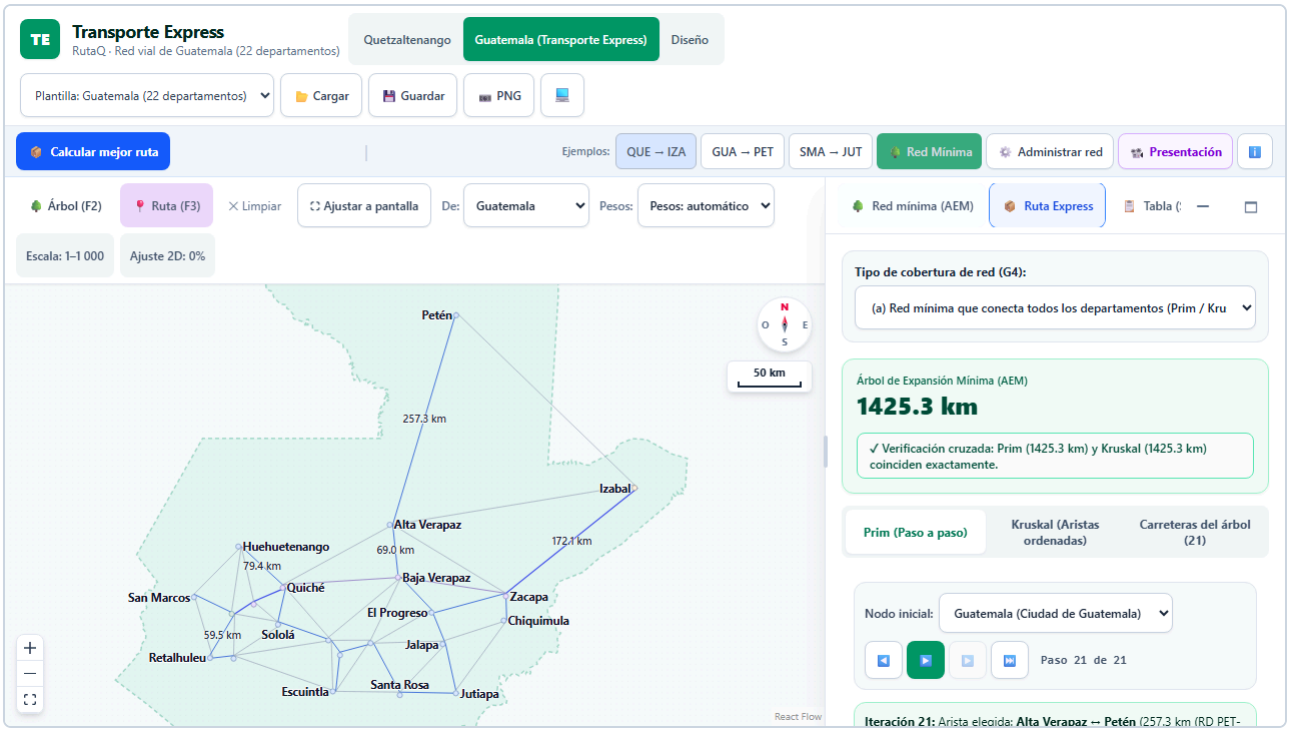
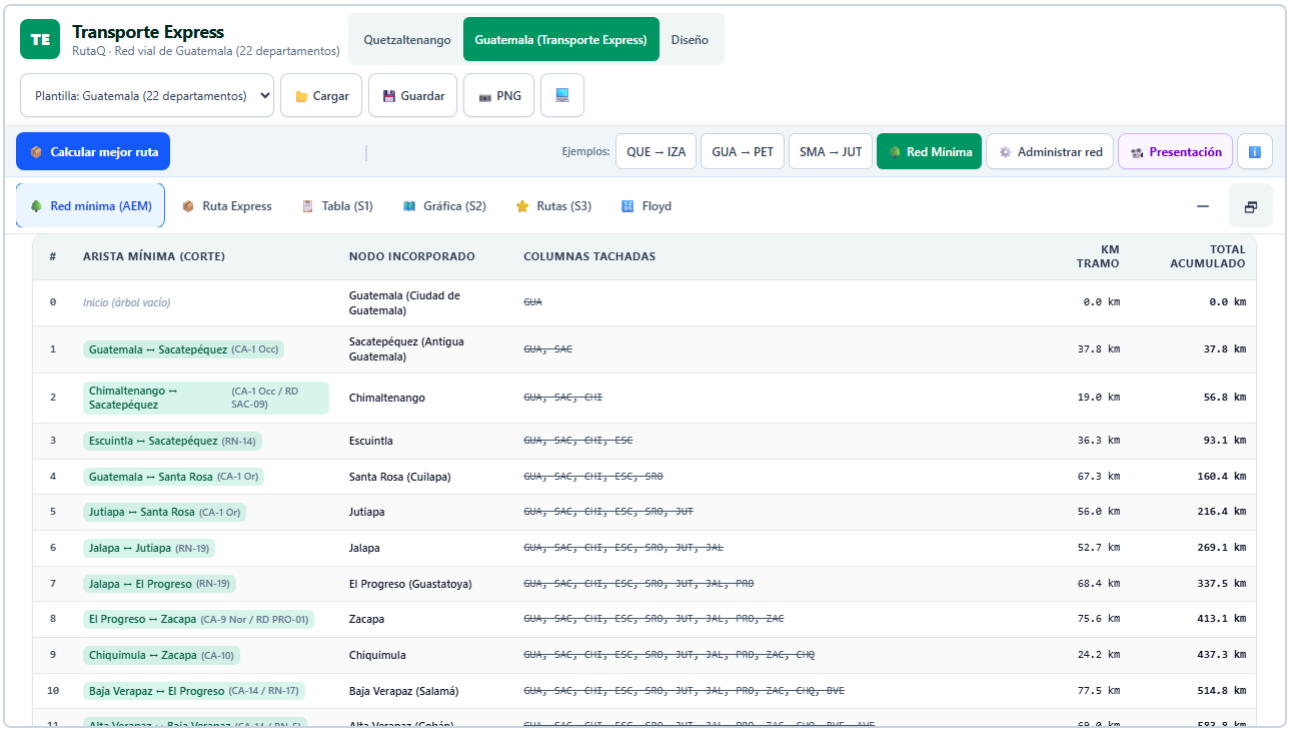
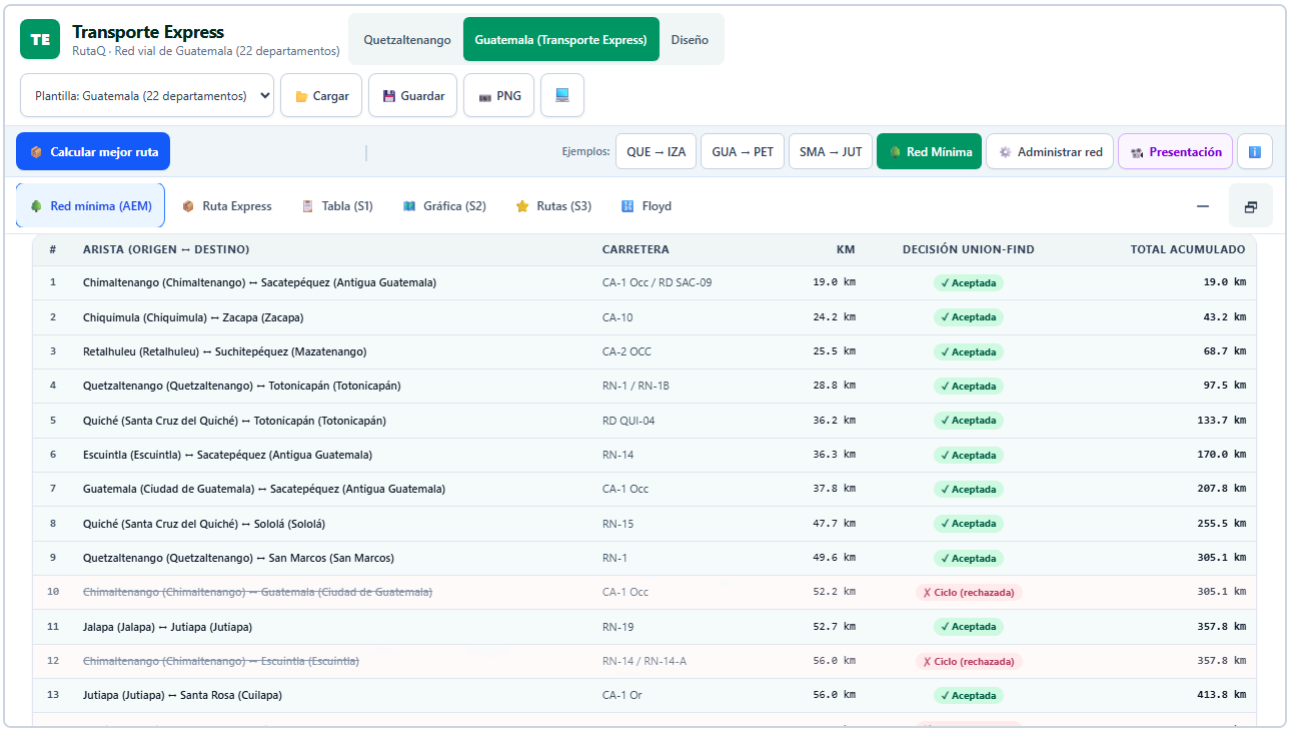


Figura 7. Visualización del Árbol de Expansión Mínima (AEM / MST) en la pestaña correspondiente, resaltando sobre el mapa las 21 aristas troncales de la red nacional con un costo global de 1425.3 km y el aviso de paridad total entre Prim y Kruskal.



#	ARISTA MÍNIMA (CORTE)	NODO INCORPORADO	COLUMNAS TACHADAS	KM TRAMO	TOTAL ACUMULADO
0	*Inicio (árbol vacío)*	Guatemala (Ciudad de Guatemala)	~~GUA~~	0.0 km	0.0 km
1	Guatemala ↔ Sacatepéquez (CA-1 Occ)	Sacatepéquez (Antigua Guatemala)	~~GUA, SAC~~	37.8 km	37.8 km
2	Chimaltenango ↔ Sacatepéquez (CA-1 Occ / RD SAC-09)	Chimaltenango	~~GUA, SAC, CHI~~	19.0 km	56.8 km
3	Escuintla ↔ Sacatepéquez (RN-14)	Escuintla	~~GUA, SAC, CHI, ESC~~	36.3 km	93.1 km
4	Guatemala ↔ Santa Rosa (CA-1 Or)	Santa Rosa (Cuilapa)	~~GUA, SAC, CHI, ESC, SRO~~	67.3 km	160.4 km
5	Jutiapa ↔ Santa Rosa (CA-1 Or)	Jutiapa	~~GUA, SAC, CHI, ESC, SRO, JUT~~	56.0 km	216.4 km
6	Jalapa ↔ Jutiapa (RN-19)	Jalapa	~~GUA, SAC, CHI, ESC, SRO, JUT, JAL~~	52.7 km	269.1 km
7	Jalapa ↔ El Progreso (RN-19)	El Progreso (Guastatoya)	~~GUA, SAC, CHI, ESC, SRO, JUT, JAL, PRO~~	68.4 km	337.5 km
8	El Progreso ↔ Zacapa (CA-9 Nor / RD PRO-01)	Zacapa	~~GUA, SAC, CHI, ESC, SRO, JUT, JAL, PRO, ZAC~~	75.6 km	413.1 km
9	Chiquimula ↔ Zacapa (CA-10)	Chiquimula	~~GUA, SAC, CHI, ESC, SRO, JUT, JAL, PRO, ZAC, CHQ~~	24.2 km	437.3 km
10	Baja Verapaz ↔ El Progreso (CA-14 / RN-17)	Baja Verapaz (Salamá)	~~GUA, SAC, CHI, ESC, SRO, JUT, JAL, PRO, ZAC, CHQ, BVE~~	77.5 km	514.8 km
11	Alta Verapaz ↔ Baja Verapaz (CA-14 / RN-5)	Alta Verapaz (Cobán)	~~GUA, SAC, CHI, ESC, SRO, JUT, JAL, PRO, ZAC, CHQ, BVE, AVE~~	68.0 km	582.8 km

Figura 8. Panel de resultados maximizado exhibiendo la resolución tabulada paso a paso del Algoritmo de Prim, con columnas de iteración, vértices en el árbol, arista mínima de corte elegida, peso individual y columna tachada.



#	ARISTA (ORIGEN ↔ DESTINO)	CARRETERA	KM	DECISIÓN UNION-FIND	TOTAL ACUMULADO
1	Chimaltenango (Chimaltenango) ↔ Sacatepéquez (Antigua Guatemala)	CA-1 Occ / RD SAC-09	19.0 km	✓ Aceptada	19.0 km
2	Chiquimula (Chiquimula) ↔ Zacapa (Zacapa)	CA-10	24.2 km	✓ Aceptada	43.2 km
3	Retalhuleu (Retalhuleu) ↔ Suchitepéquez (Mazatenango)	CA-2 OCC	25.5 km	✓ Aceptada	68.7 km
4	Quetzaltenango (Quetzaltenango) ↔ Totonicapán (Totonicapán)	RN-1 / RN-1B	28.8 km	✓ Aceptada	97.5 km
5	Quiché (Santa Cruz del Quiché) ↔ Totonicapán (Totonicapán)	RD QUI-04	36.2 km	✓ Aceptada	133.7 km
6	Escuintla (Escuintla) ↔ Sacatepéquez (Antigua Guatemala)	RN-14	36.3 km	✓ Aceptada	170.0 km
7	Guatemala (Ciudad de Guatemala) ↔ Sacatepéquez (Antigua Guatemala)	CA-1 Occ	37.8 km	✓ Aceptada	207.8 km
8	Quiché (Santa Cruz del Quiché) ↔ Sololá (Sololá)	RN-15	47.7 km	✓ Aceptada	255.5 km
9	Quetzaltenango (Quetzaltenango) ↔ San Marcos (San Marcos)	RN-1	49.6 km	✓ Aceptada	305.1 km
10	~~Chimaltenango (Chimaltenango) ↔ Guatemala (Ciudad de Guatemala)~~	CA-1 Occ	52.2 km	✗ Ciclo (rechazada)	305.1 km
11	Jalapa (Jalapa) ↔ Jutiapa (Jutiapa)	RN-19	52.7 km	✓ Aceptada	357.8 km
12	~~Chimaltenango (Chimaltenango) ↔ Escuintla (Escuintla)~~	RN-14 / RN-14-A	56.0 km	✗ Ciclo (rechazada)	357.8 km
13	Jutiapa (Jutiapa) ↔ Santa Rosa (Cuilapa)	CA-1 Or	56.0 km	✓ Aceptada	413.8 km

Figura 9. Panel de resultados maximizado mostrando la tabla de ejecución secuencial del Algoritmo de Kruskal, con ordenamiento creciente de aristas, componentes evaluadas, decisión de aceptación o rechazo por ciclo y distancia acumulada.

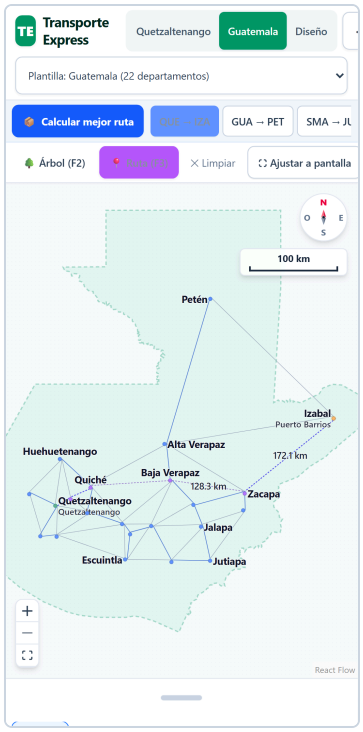


Figura 10. Vista responsiva y adaptativa del sistema en un dispositivo móvil inteligente (Google Pixel 7), con la interfaz optimizada para operadores en ruta y panel inferior deslizable con el resultado del enrutamiento.

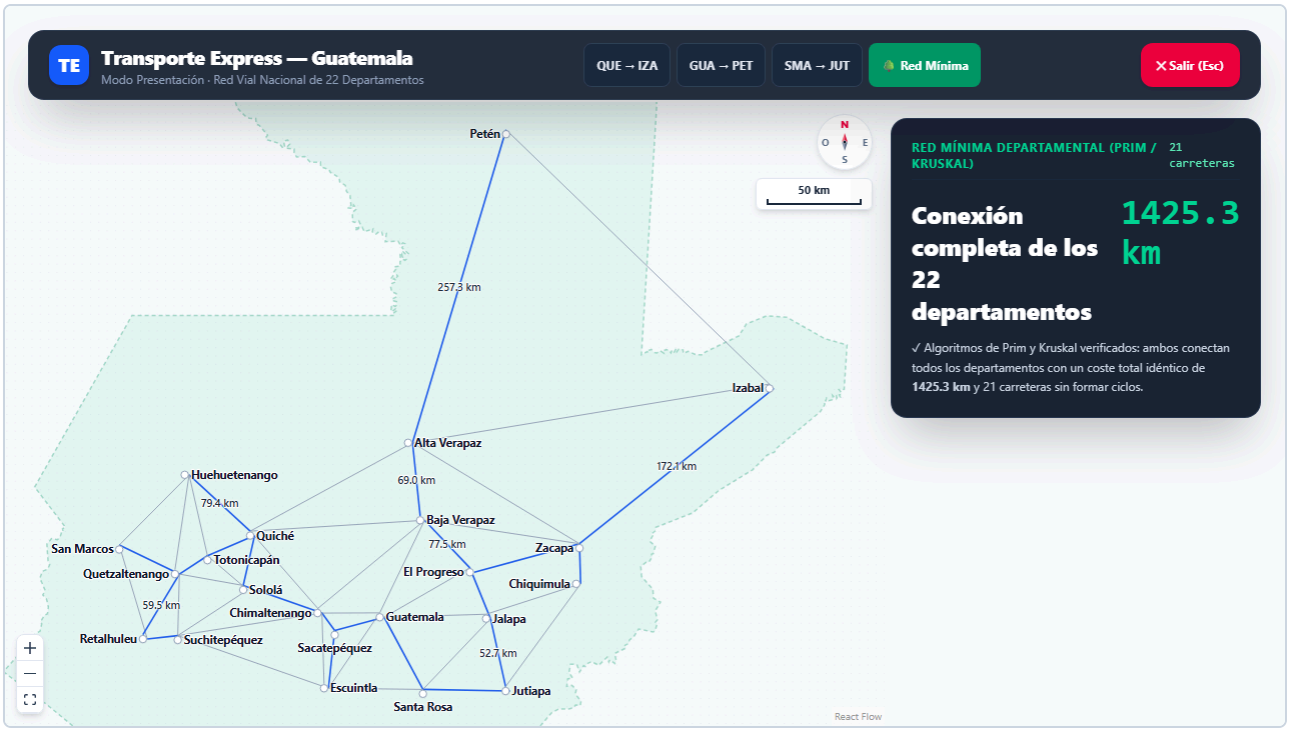


Figura 11. Modalidad de presentación a pantalla completa configurada a resolución HD (1280×720 píxeles), optimizada para proyectores de aula universitaria con controles simplificados y mapa de alta visibilidad.

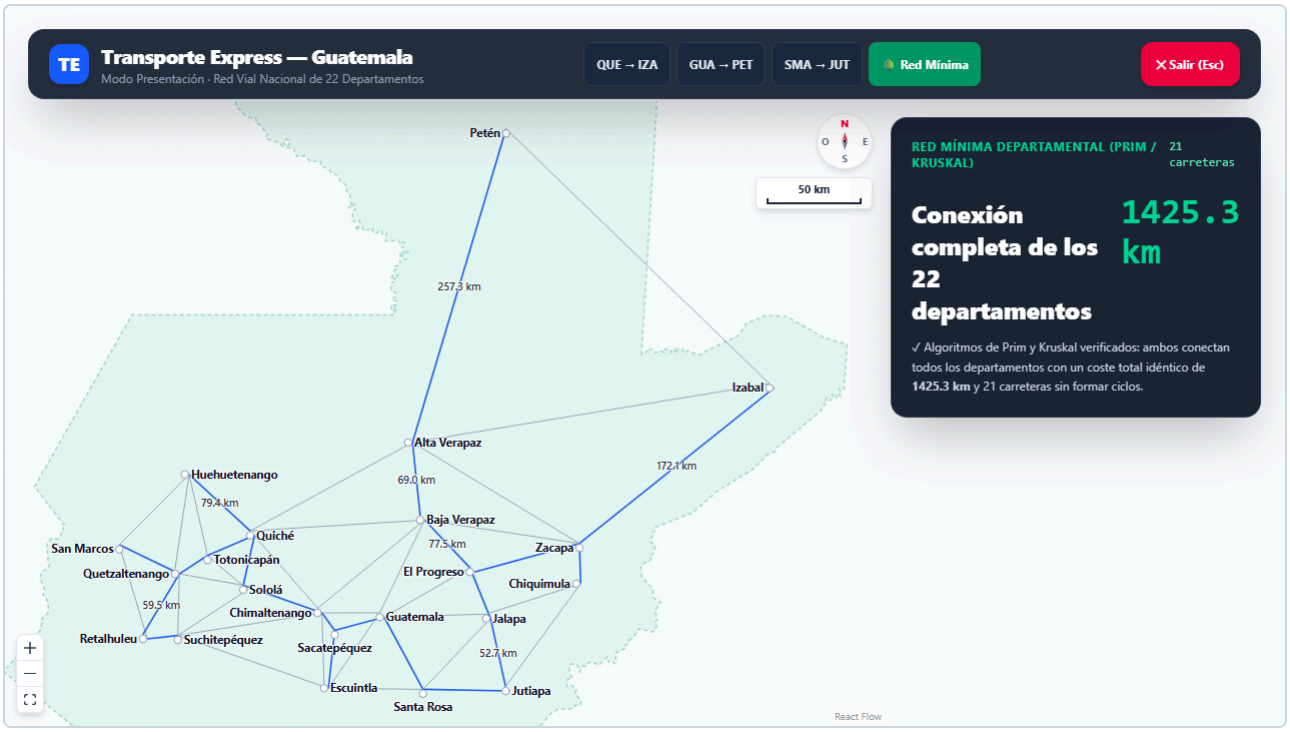


Figura 12. Vista panorámica del modo presentación interactivo con acceso directo a los ejemplos de prueba, métricas operativas en vivo y mapa departamental sin elementos distractores de la interfaz.

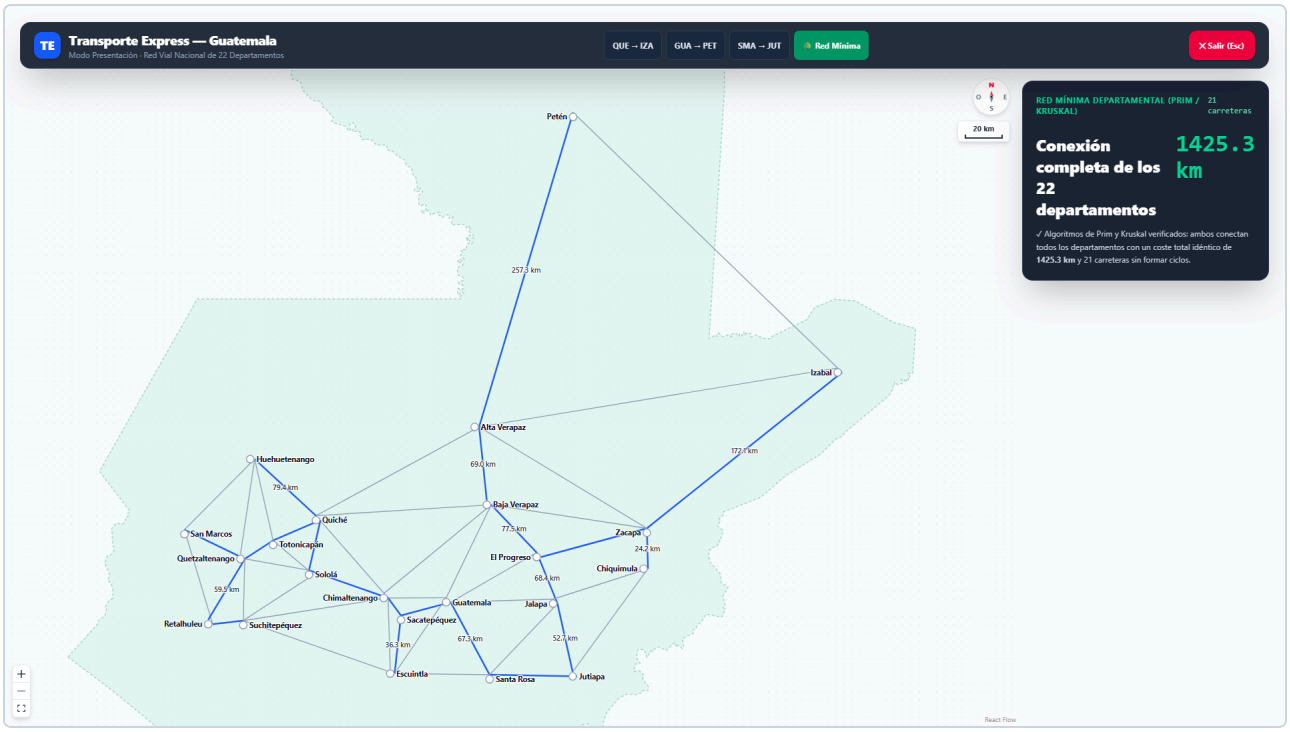


Figura 13. Vista en alta definición Full HD (1920×1080 píxeles) del modo presentación, mostrando el lienzo geográfico completo a escala óptima con la red vial nacional interconectada y panel de métricas lateral.

4. Código Fuente

Proyecto II «Transporte Express» · RutaQ · https://github.com/Criss-paz/rutaq

Generado automáticamente con `npm run anexo-codigo` el 2026-10-26.

Núcleo de algoritmos

Algoritmos puros en TypeScript, sin dependencias de la interfaz.

`src/core/dijkstra.ts`

154 líneas.

```ts
import type { CoreApi, DijkstraStep, Graph, NodeId, PathResult } from './types';
import { adjacencyList } from './graph';



/** Heap binario mínimo ordenado por distancia y NodeId. */
class Heap {
  a: { id: string; d: number }[] = [];
  push(x: { id: string; d: number }) {
    this.a.push(x);
    let i = this.a.length - 1;
    while (i) {
      const p = (i - 1) >> 1;
      if (this.c(this.a[p]!, x) <= 0) break;
      this.a[i] = this.a[p]!;
```

```
        i = p;
      }
      this.a[i] = x;
    }
    pop() {
      if (!this.a.length) return undefined;
      const root = this.a[0]!,
        last = this.a.pop()!;
      if (this.a.length) {
        let i = 0;
        while (i * 2 + 1 < this.a.length) {
          let c = i * 2 + 1;
          if (c + 1 < this.a.length && this.c(this.a[c + 1]!, this.a[c]!) < 0) c++;
          if (this.c(last, this.a[c]!) <= 0) break;
          this.a[i] = this.a[c]!;
          i = c;
        }
        this.a[i] = last;
      }
      return root;
```

```
    }
    c(a: { id: string; d: number }, b: { id: string; d: number }) {
      return a.d - b.d || (a.id < b.id ? -1 : a.id > b.id ? 1 : 0);
    }
  }
  const assert = (g: Graph, s: NodeId) => {
    if (!g.nodes.some((n) => n.id === s)) throw new Error(`No existe el nodo «${s}».`);
  };
  /**
   * Calcula caminos mínimos con Dijkstra y conserva una instantánea por iteración para la tabla.
   * Complejidad: O((V + E) log V) con heap binario; memoria O(V + E).
   * Referencia: Cormen et al., Introduction to Algorithms, capítulo 24.
   */
  export const dijkstra: CoreApi['dijkstra'] = (g, s) => {
    assert(g, s);
    const adj = adjacencyList(g),
      labels = Object.fromEntries(g.nodes.map((n) => [n.id, { dist: Infinity, pred: null }])) as Record<
        string,
        { dist: number; pred: string | null }
```

```
    >;
  labels[s] = { dist: 0, pred: null };
  const permanent: string[] = [],
    crossed: string[] = [],
    steps: DijkstraStep[] = [];
  steps.push({
    iter: 0,
    current: null,
    permanent: [],
    crossedOut: [],
    labels: structuredClone(labels),
    updated: [s],
    evaluatedEdges: [],
  });
  const heap = new Heap();
  heap.push({ id: s, d: 0 });
  const done = new Set<string>();
  let iter = 0;
  while (true) {
```

```
    // Se omiten entradas antiguas del heap cuando la distancia del nodo mejoró posteriormente.
    let item = heap.pop();
    while (item && (done.has(item.id) || item.d !== labels[item.id]!.dist)) item = heap.pop();
    if (!item) break;
    const u = item.id;
    done.add(u);
    permanent.push(u);
    crossed.push(u);
    const updated: string[] = [],
      evaluatedEdges: string[] = [];
    for (const v of adj.get(u) ?? []) {
      if (done.has(v.node)) continue;
      evaluatedEdges.push(v.edgeId);
      const nd = item.d + v.weight;
      // Solo una mejora estricta cambia la etiqueta; los empates conservan el predecesor previo.
      if (nd < labels[v.node]!.dist) {
        labels[v.node] = { dist: nd, pred: u };
        updated.push(v.node);
        heap.push({ id: v.node, d: nd });
```

```
        }
      }
      steps.push({
        iter: ++iter,
        current: u,
        permanent: [...permanent],
        crossedOut: [...crossed],
        labels: structuredClone(labels),
        updated,
        evaluatedEdges,
      });
    }
    return {
      source: s,
      labels,
      steps,
      treeEdges: g.edges
        .filter((e) => labels[e.source]?.pred === e.target || labels[e.target]?.pred ===
e.source)
        .map((e) => e.id),
```

```
110     };
111   };
112
113   export const shortestPaths: CoreApi['shortestPaths'] = (g, s, t, maxTies = 10) => {
114     assert(g, s);
115     assert(g, t);
116     if (s === t) return [{ nodes: [s], edges: [], distance: 0 }];
117     const adj = adjacencyList(g),
118       dist = dijkstra(g, s).labels[t]!.dist,
119       toTarget = dijkstra(g, t).labels;
120     if (!Number.isFinite(dist)) return [];
121     const out: PathResult[] = [],
122       nodes = [s],
123       edges: string[] = [],
124       seen = new Set([s]),
125       limit = Math.max(1, maxTies);
126     const equal = (a: number, b: number) => Math.abs(a - b) <= 1e-9 * Math.max(1,
    Math.abs(a), Math.abs(b));
127     const walk = (u: string, d: number) => {
128       if (out.length >= Math.max(1, maxTies)) return;
```

```
    if (u === t) {
      if (equal(d, dist)) out.push({ nodes: [...nodes], edges: [...edges], distance: d });
      return;
    }
    for (const v of adj.get(u) ?? []) {
      if (seen.has(v.node)) continue;
      const remaining = toTarget[v.node]!.dist,
        nextDistance = d + v.weight;
      if (Number.isFinite(remaining) && nextDistance + remaining <= dist * (1 + 1e-9)) {
        seen.add(v.node);
        nodes.push(v.node);
        edges.push(v.edgeId);
        walk(v.node, nextDistance);
        edges.pop();
        nodes.pop();
        seen.delete(v.node);
      }
    }
  };
  walk(s, 0);
```

```
149     const compare = (a: string[], b: string[]) => {
150       for (let i = 0; i < Math.min(a.length, b.length); i++) if (a[i] !== b[i]) return a[i]!
    < b[i]! ? -1 : 1;
151       return a.length - b.length;
152     };
153     return out.sort((a, b) => compare(a.nodes, b.nodes)).slice(0, limit);
154   };
```

`src/core/mst.ts`

170 líneas.

```
1   import type { CoreApiP2, Graph, MSTStep, NodeId } from './types';
2
3   /**
4    * Calcula árboles y bosques de expansión mínima con Prim y Kruskal.
5    * Complejidad: el heap de Prim cuesta O(E log E); registrar todos los cortes cuesta O(VE).
6    * Kruskal cuesta O(E log E + VE), incluido el historial visual de nodos por paso.
7    * Referencia: Cormen et al., Introduction to Algorithms, capítulos 21 y 23.
8    */
9
10  type Edge = Graph['edges'][number];
11  const edgeOrder = (a: Edge, b: Edge) => a.weight - b.weight || (a.id < b.id ? -1 : a.id >
    b.id ? 1 : 0);
```

```
12  const assertStart = (g: Graph, start: NodeId) => {
13    if (!g.nodes.some((node) => node.id === start)) throw new Error(`No existe el nodo «${start}».`);
14  };
15
16  /** Min-heap de aristas: permite extraer cada arista candidata en O(log E). */
17  class EdgeHeap {
18    private values: Edge[] = [];
19    push(value: Edge) {
20      let i = this.values.push(value) - 1;
21      while (i > 0) {
22        const parent = (i - 1) >> 1;
23        if (edgeOrder(this.values[parent]!, value) <= 0) break;
24        this.values[i] = this.values[parent]!;
25        i = parent;
26      }
27      this.values[i] = value;
28    }
29    pop(): Edge | undefined {
30      if (!this.values.length) return undefined;
```

```
    const first = this.values[0]!,
      last = this.values.pop()!;
    if (this.values.length) {
      let i = 0;
      while (2 * i + 1 < this.values.length) {
        let child = 2 * i + 1;
        if (child + 1 < this.values.length && edgeOrder(this.values[child + 1]!, this.values[child]!) < 0) child++;
        if (edgeOrder(last, this.values[child]!) <= 0) break;
        this.values[i] = this.values[child]!;
        i = child;
      }
      this.values[i] = last;
    }
    return first;
  }
}

/** Prim crece desde `start`, incorporando en cada fila la arista más ligera del corte. */
export const prim: CoreApiP2['prim'] = (g, requestedStart) => {
```

```
  const start = requestedStart ?? g.nodes[0]?.id;
  if (start === undefined) throw new Error('El grafo no contiene nodos.');
  assertStart(g, start);
  const adjacency = new Map<NodeId, Edge[]>();
  for (const node of g.nodes) adjacency.set(node.id, []);
  for (const edge of g.edges) {
    adjacency.get(edge.source)?.push(edge);
    adjacency.get(edge.target)?.push(edge);
  }
  const inside = new Set<NodeId>([start]),
    inTree = [start],
    edges: string[] = [],
    steps: MSTStep[] = [];
  let total = 0;
  steps.push({
    iter: 0,
    chosenEdge: null,
    inTree: [...inTree],
    crossedOut: [...inTree],
    candidates: [],
```

```
      totalSoFar: 0,
    });
    const heap = new EdgeHeap(),
      queued = new Set<string>();
    const offer = (node: NodeId) => {
      for (const edge of adjacency.get(node) ?? []) {
        const other = edge.source === node ? edge.target : edge.source;
        if (!inside.has(other) && !queued.has(edge.id)) {
          queued.add(edge.id);
          heap.push(edge);
        }
      }
    };
    offer(start);
    let iter = 0;
    while (true) {
      // Descarta del heap aristas que ya no cruzan el corte tras incorporar su extremo.
      let edge = heap.pop();
      while (edge && inside.has(edge.source) === inside.has(edge.target)) edge = heap.pop();
```

```
    if (!edge) break;
    const candidates = g.edges
      .filter((candidate) => inside.has(candidate.source) !== inside.has(candidate.target))
      .sort(edgeOrder)
      .map((candidate) => candidate.id);
    const node = inside.has(edge.source) ? edge.target : edge.source;
    inside.add(node);
    inTree.push(node);
    edges.push(edge.id);
    total += edge.weight;
    offer(node);
    // La instantánea de candidatos es el corte evaluado antes de incorporar el nuevo extremo.
    steps.push({
      iter: ++iter,
      chosenEdge: edge.id,
      inTree: [...inTree],
      crossedOut: [...inTree],
      candidates,
      totalSoFar: total,
```

```
    });
  }
  return { algorithm: 'prim', edges, total, steps };
};

/** Union-Find con compresión de caminos y unión por rango para evitar ciclos. */
class UnionFind {
  private parent = new Map<NodeId, NodeId>();
  private rank = new Map<NodeId, number>();
  constructor(nodes: NodeId[]) {
    for (const node of nodes) {
      this.parent.set(node, node);
      this.rank.set(node, 0);
    }
  }
  find(node: NodeId): NodeId {
    const parent = this.parent.get(node);
    if (parent === undefined) throw new Error(`No existe el nodo «${node}».`);
    if (parent !== node) this.parent.set(node, this.find(parent));
    return this.parent.get(node)!;
```

```
    }
    union(a: NodeId, b: NodeId): boolean {
      let rootA = this.find(a),
        rootB = this.find(b);
      if (rootA === rootB) return false;
      const rankA = this.rank.get(rootA)!,
        rankB = this.rank.get(rootB)!;
      if (rankA < rankB) [rootA, rootB] = [rootB, rootA];
      this.parent.set(rootB, rootA);
      if (rankA === rankB) this.rank.set(rootA, rankA + 1);
      return true;
    }
  }

  /** Kruskal examina aristas por peso e id; acepta las que conectan componentes distintas.
  */
  export const kruskal: CoreApiP2['kruskal'] = (g) => {
    const ordered = [...g.edges].sort(edgeOrder),
      uf = new UnionFind(g.nodes.map((node) => node.id));
    const edges: string[] = [],
```

```
147       touched = new Set<NodeId>(),
148       steps: MSTStep[] = [];
149     let total = 0;
150     steps.push({ iter: 0, chosenEdge: null, inTree: [], crossedOut: [], candidates: [],
    totalSoFar: 0 });
151     ordered.forEach((edge, index) => {
152       const accepted = uf.union(edge.source, edge.target);
153       if (accepted) {
154         edges.push(edge.id);
155         total += edge.weight;
156         touched.add(edge.source);
157         touched.add(edge.target);
158       }
159       steps.push({
160         iter: index + 1,
161         chosenEdge: accepted ? edge.id : null,
162         ...(accepted ? {} : { rejectedEdge: edge.id }),
163         inTree: g.nodes.map((node) => node.id).filter((id) => touched.has(id)),
164         crossedOut: [],
165         candidates: [edge.id],
```

```
        totalSoFar: total,
      });
    });
    return { algorithm: 'kruskal', edges, total, steps };
  };
```

`src/core/route.ts`

37 líneas.

```
import type { CoreApiP2, PathResult } from './types';

/**
 * Desglosa un camino en sus tramos y acumula sus distancias.
 * Complejidad: O(V + E + L), donde L es el número de tramos.
 * Referencia: especificación del Proyecto II, apartado G3 (tabla de tramos).
 */
export const routeBreakdown: CoreApiP2['routeBreakdown'] = (g, path: PathResult) => {
  if (!path.nodes.length || path.edges.length !== path.nodes.length - 1)
    throw new Error('La ruta debe tener un nodo más que aristas.');
  const nodeIds = new Set(g.nodes.map((node) => node.id));
  const edgeById = new Map(g.edges.map((edge) => [edge.id, edge]));
  if (path.nodes.some((id) => !nodeIds.has(id))) throw new Error('La ruta contiene un nodo
que no existe.');
```

```
14    let cumulative = 0;
15    const legs = path.edges.map((edgeId, index) => {
16      const from = path.nodes[index]!,
17        to = path.nodes[index + 1]!;
18      const edge = edgeById.get(edgeId);
19      if (!edge || !((edge.source === from && edge.target === to) || (edge.source === to &&
    edge.target === from)))
20        throw new Error(`La arista «${edgeId}» no conecta ${from} con ${to}.`);
21      cumulative += edge.weight;
22      const road = edge.meta?.carretera;
23      return {
24        from,
25        to,
26        edgeId,
27        ...(typeof road === 'string' && road.trim() ? { road } : {}),
28        km: edge.weight,
29        cumulative,
30      };
31    });
32    const total = cumulative;
```

```
    const formula = legs.length
      ? `${legs.map((leg) => leg.km.toFixed(1)).join(' + ')} = ${total.toFixed(1)} ${g.unit}`
      : `0 ${g.unit}`;
    return { nodes: [...path.nodes], legs, total, formula };
  };
```

`src/core/tour.ts`

50 líneas.

```
import { floydWarshall } from './floyd';
import type { CoreApiP2, NodeId } from './types';

/**
 * Construye un recorrido cerrado aproximado: vecino más cercano y mejoras 2-opt.
 * Complejidad: Floyd-Warshall O(V³), vecino más cercano O(V²) y 2-opt O(V²) por pasada.
 * Referencia: Rosenkrantz, Stearns y Lewis, An Analysis of Several Heuristics for the
Traveling Salesman Problem (1977).
 */
export const deliveryTour: CoreApiP2['deliveryTour'] = (g, start: NodeId) => {
  const startIndex = g.nodes.findIndex((node) => node.id === start);
  if (startIndex < 0) throw new Error(`No existe el nodo «${start}».`);
  const f = floydWarshall(g),
    ids = f.order;
```

```
    const reachable = ids.filter((_, i) => Number.isFinite(f.dist[startIndex]![i]!));
    const unvisited = new Set(reachable.filter((id) => id !== start));
    const order = [start];
    // Elige repetidamente el destino aún no visitado más cercano; NodeId resuelve los
empates.
    while (unvisited.size) {
      const current = ids.indexOf(order[order.length - 1]!);
      const next = [...unvisited].sort((a, b) => {
        const da = f.dist[current]![ids.indexOf(a)]!,
          db = f.dist[current]![ids.indexOf(b)]!;
        return da - db || (a < b ? -1 : a > b ? 1 : 0);
      })[0]!;
      order.push(next);
      unvisited.delete(next);
    }
    const distance = (a: NodeId, b: NodeId) => f.dist[ids.indexOf(a)]![ids.indexOf(b)]!;
    // Invierte segmentos cuando el cambio de dos enlaces reduce la longitud del ciclo.
    if (order.length > 3) {
      let improved = true;
      while (improved) {
```

```
      improved = false;
      for (let i = 1; i < order.length - 1 && !improved; i++) {
        for (let k = i + 1; k < order.length; k++) {
          const before = distance(order[i - 1]!, order[i]!) + distance(order[k]!, order[(k + 1) % order.length]!);
          const after = distance(order[i - 1]!, order[k]!) + distance(order[i]!, order[(k + 1) % order.length]!);
          if (after < before - 1e-9 * Math.max(1, Math.abs(before), Math.abs(after))) {
            order.splice(i, k - i + 1, ...order.slice(i, k + 1).reverse());
            improved = true;
            break;
          }
        }
      }
    }
  }
  let total = 0;
  for (let i = 0; i < order.length; i++) total += distance(order[i]!, order[(i + 1) % order.length]!);
  return { order, total, method: 'nn+2opt' };
};
```

`src/core/graph.ts`

69 líneas.

```ts
import type { CoreApi, Graph, NodeId } from './types';



export interface Neighbor {
  node: NodeId;
  weight: number;
  edgeId: string;
}
export const adjacencyList = (g: Graph): Map<NodeId, Neighbor[]> => {
  const map = new Map(g.nodes.map((n) => [n.id, [] as Neighbor[]]));
  for (const e of g.edges) {
    map.get(e.source)?.push({ node: e.target, weight: e.weight, edgeId: e.id });
    map.get(e.target)?.push({ node: e.source, weight: e.weight, edgeId: e.id });
  }
  for (const list of map.values())
    list.sort((a, b) => (a.node < b.node ? -1 : a.node > b.node ? 1 :
a.edgeId.localeCompare(b.edgeId)));
  return map;
};

```

```
export const validateGraph: CoreApi['validateGraph'] = (g) => {
  const issues: ReturnType<CoreApi['validateGraph']>['issues'] = [];
  const issue = (code: (typeof issues)[number]['code'], message: string, ids?: string[],
edge = false) =>
    issues.push({
      code,
      severity: code === 'DISCONNECTED' ? 'warning' : 'error',
      message,
      ...(ids ? (edge ? { edgeIds: ids } : { nodeIds: ids }) : {}),
    });
  if (!g.nodes.length) issue('EMPTY_GRAPH', 'El grafo no contiene nodos.');
  const nodes = new Set<string>();
  for (const n of g.nodes) {
    if (nodes.has(n.id)) issue('DUPLICATE_NODE_ID', `El identificador de nodo «${n.id}»
está repetido.`, [n.id]);
    nodes.add(n.id);
  }
  const edges = new Set<string>(),
    pairs = new Set<string>();
  for (const e of g.edges) {
    if (edges.has(e.id))
```

```
      issue('DUPLICATE_EDGE_ID', `El identificador de arista «${e.id}» está repetido.`, [e.id], true);
    edges.add(e.id);
    if (!nodes.has(e.source) || !nodes.has(e.target))
      issue('MISSING_NODE', `La arista «${e.id}» referencia un nodo inexistente.`, [e.id], true);
    if (e.source === e.target) issue('SELF_LOOP', `La arista «${e.id}» conecta un nodo consigo mismo.`, [e.id], true);
    const pair = [e.source, e.target].sort().join('\0');
    if (pairs.has(pair))
      issue('DUPLICATE_EDGE', `Hay más de una arista entre «${e.source}» y «${e.target}».`, [e.id], true);
    pairs.add(pair);
    if (!Number.isFinite(e.weight))
      issue('INVALID_WEIGHT', `El peso de «${e.id}» debe ser un número finito.`, [e.id], true);
    else if (e.weight <= 0) issue('NON_POSITIVE_WEIGHT', `El peso de «${e.id}» debe ser mayor que cero.`, [e.id], true);
  }
  if (g.nodes.length > 0 && nodes.size === g.nodes.length) {
    const adj = adjacencyList(g),
      seen = new Set<NodeId>(),
      stack = [g.nodes[0]!.id];
```

```
55      while (stack.length) {
56        const id = stack.pop()!;
57        if (seen.has(id)) continue;
58        seen.add(id);
59        for (const n of adj.get(id) ?? []) if (!seen.has(n.node)) stack.push(n.node);
60      }
61      if (seen.size !== g.nodes.length)
62        issue(
63          'DISCONNECTED',
64          'El grafo está desconectado; algunas rutas no existen.',
65          g.nodes.filter((n) => !seen.has(n.id)).map((n) => n.id),
66        );
67    }
68    return { ok: !issues.some((i) => i.severity === 'error'), issues };
69  };
```

Contrato de tipos

Tipos compartidos entre el núcleo, el estado y la interfaz.

`src/core/types.ts`

418 líneas.

```ts
/**
 * CONTRATO DE RutaQ — v1 (congelado en la Fase F0).
 *
 * Dueño: Claude Code (orquestador). Ningún agente modifica este archivo sin aprobación;
 * las dudas o propuestas de cambio se anotan en docs/agentes/<agente>.md.
 *
 * Convenciones globales:
 * - Distancia inalcanzable = `Infinity` (nunca `null` ni -1). Se transfiere bien a Workers
 *   (structured clone); al serializar a JSON se debe convertir explícitamente.
 * - Desempate determinista: ante distancias iguales gana el `NodeId` menor por comparación
 *   de cadenas (`a < b`, sin locale). Aplica al nodo que se vuelve permanente y al orden de rutas.
 * - Predecesor: una etiqueta solo cambia ante una mejora ESTRICTA (como en el método tabular a mano);
 *   ante un empate se conserva el predecesor que ya tenía.
 * - Comparaciones de distancias acumuladas con tolerancia relativa 1e-9 (sumas de decimales).
 * - Todas las funciones del núcleo son puras: no mutan el `Graph` recibido.
 * - Los grafos son no dirigidos: una arista {source, target} se recorre en ambos sentidos.
 */

export type NodeId = string;
```

```
export type EdgeShape = 'straight' | 'bezier' | 'arc' | 'step' | 'smoothstep';

/** Nodo del lienzo. (x, y) es la esquina superior izquierda en px; (w, h) su tamaño en px.
*/
export interface GNode {
  id: NodeId;
  label: string;
  x: number;
  y: number;
  w: number;
  h: number;
  lat?: number;
  lon?: number;
}

/**
 * Arista no dirigida. `weight` > 0 en la unidad del grafo.
 * `shapeAuto = true` → `autoEdgeShapes` puede cambiar su forma; `false` → la fijó el
usuario.
 * `meta` guarda datos del dataset (carretera, fuente, km_google, nota, geometry…).
 */
```

```
export interface GEdge {
  id: string;
  source: NodeId;
  target: NodeId;
  weight: number;
  shape: EdgeShape;
  shapeAuto: boolean;
  meta?: Record<string, unknown>;
}

export interface Graph {
  nodes: GNode[];
  edges: GEdge[];
  directed: false;
  unit: string;
}

// ──────────────────────────── Dijkstra ────────────────────────────
```

```ts
/** Etiqueta del método tabular: distancia acumulada y nodo predecesor. */
export interface Label {
  dist: number;
  pred: NodeId | null;
}

/**
 * Un paso del método tabular.
 * - `iter = 0` es la inicialización: `current = null`, origen con `{dist: 0, pred: null}`,
 *   el resto `{dist: Infinity, pred: null}`, `permanent = []`, `updated = [origen]`.
 * - `iter = k ≥ 1`: `current` es el nodo con la etiqueta temporal mínima, que se vuelve permanente.
 *   `permanent` y `crossedOut` incluyen a `current` (crossedOut = columnas tachadas; hoy coincide
 *   con permanent pero se mantiene separado para variantes del método).
 *   `labels` es la instantánea DESPUÉS de relajar los vecinos de `current`.
 *   `updated` = nodos cuya etiqueta cambió en este paso (mejora estricta).
 *   `evaluatedEdges` = ids de aristas de `current` hacia vecinos no permanentes examinadas.
 * - El algoritmo termina cuando no quedan temporales finitas; los nodos inalcanzables
 *   nunca se vuelven permanentes y conservan `dist = Infinity`.
 */
```

```ts
export interface DijkstraStep {
  iter: number;
  current: NodeId | null;
  permanent: NodeId[];
  crossedOut: NodeId[];
  labels: Record<NodeId, Label>;
  updated: NodeId[];
  evaluatedEdges: string[];
}

/** `treeEdges` = ids de las aristas (pred(v), v) del árbol de caminos más cortos. */
export interface DijkstraResult {
  source: NodeId;
  labels: Record<NodeId, Label>;
  steps: DijkstraStep[];
  treeEdges: string[];
}

/** Camino simple: `nodes` incluye origen y destino; `edges.length = nodes.length - 1`. */
```

```ts
export interface PathResult {
  nodes: NodeId[];
  edges: string[];
  distance: number;
}

// ─────────────────────────── Método tabular (S1) ───────────────────────────

/**
 * Estado visual de una casilla de la tabla de iteraciones.
 * - `min`: la etiqueta temporal mínima elegida en esta fila (se vuelve permanente).
 * - `updated`: la etiqueta mejoró en esta fila.
 * - `crossed`: la columna ya estaba tachada (nodo permanente en una fila anterior).
 * - `unreached`: sin etiqueta finita todavía.
 * - `normal`: etiqueta temporal sin cambios.
 */
export type TabularCellState = 'normal' | 'min' | 'updated' | 'crossed' | 'unreached';

export interface TabularCell {
  label: Label;
```

```
  state: TabularCellState;
}

/** Fila `iter` de la tabla; corresponde 1:1 con `DijkstraResult.steps[iter]`. */
export interface TabularRow {
  iter: number;
  current: NodeId | null;
  cells: Record<NodeId, TabularCell>;
}

/**
 * Modelo completo de la tabla S1. `order` = orden de columnas (orden de `graph.nodes`).
 * `adjacency[i][j]` = peso de la arista entre order[i] y order[j], o `null` si no conectan
 * (la diagonal también es `null`). El formateo de texto lo hace la UI con `TabularConfig`.
 */
export interface TabularModel {
  source: NodeId;
  order: NodeId[];
  adjacency: (number | null)[][];
```

```
  rows: TabularRow[];
}

/** Opciones de presentación de S1 (ajustables cuando Cris confirme el formato del catedrático). */
export interface TabularConfig {
  showAdjacency: boolean;
  /** `paren` → "12 (B)" · `bracket` → "[12, B]" · `slash` → "12/B" */
  labelFormat: 'paren' | 'bracket' | 'slash';
  crossOut: 'column' | 'row';
  /** Repetir en filas posteriores la etiqueta de columnas ya tachadas. */
  repeatPermanent: boolean;
}

// ─────────────────────────────── Floyd-Warshall ────────────────────────────────

/**
 * `order` = orden de índices (orden de `graph.nodes`).
 * `dist` = matriz final; `next[i][j]` = índice del siguiente nodo desde i hacia j (null si no hay camino o i = j).
 * `matrices` = [D⁽⁰⁾, D⁽¹⁾, …, D⁽ⁿ⁾]; D⁽⁰⁾ es la matriz de pesos (0 en diagonal, Infinity sin arista),
```

```
154   * D⁽ᵏ⁾ usa como intermedios los primeros k nodos de `order`. Longitud = n + 1.
155   */
156  export interface FloydResult {
157    order: NodeId[];
158    dist: number[][];
159    next: (number | null)[][];
160    matrices: number[][][];
161  }
162
163  // ──────────────────────────── Todas las rutas (S3) ────────────────────────────
164
165  /**
166   * `limit`: máximo de rutas a listar (por defecto 5 000). Si el DFS encuentra más de
     `limit`
167   * rutas o supera `timeoutMs`, se abandona y se devuelven las `limit` más cortas con Yen.
168   */
169  export interface AllPathsOptions {
170    limit: number;
171    timeoutMs: number;
172  }
```

```
173
174  /**
175   * `paths` ordenadas por distancia ascendente (desempate: secuencia de ids lexicográfica).
176   * `complete = true` ⇔ `paths` contiene TODOS los caminos simples (method = 'dfs').
177   * `totalFound`: con complete = true, el total exacto; si no, los encontrados por el DFS
    antes
178   * de abandonar (cota inferior del total, sirve para el aviso "más de N rutas").
179   */
180  export interface AllPathsResult {
181    paths: PathResult[];
182    complete: boolean;
183    method: 'dfs' | 'yen';
184    totalFound: number;
185  }
186
187  // ──────────────────────────── Escala y distribución (F6) ────────────────────────────
188
189  /**
190   * `magnitude = 10^ceil(log10(peso_máx))` (mínimo 1) · `range` = texto para la UI, p. ej.
    "1–1 000".
191   * `pxPerUnit` = 160 / mediana de los pesos (la arista mediana mide ~160 px).
```

```
 */
export interface ScaleInfo {
  magnitude: number;
  range: string;
  pxPerUnit: number;
}

/**
 * Posiciones sugeridas (centro del nodo, en px) por stress majorization con distancias objetivo
 * = caminos más cortos × pxPerUnit. `stressError` en % (0–100):
 * 100 · sqrt( Σ (‖p_i − p_j‖/pxPerUnit − d_ij)² / Σ d_ij² ) sobre pares conectados i < j.
 */
export interface LayoutSuggestion {
  positions: Record<NodeId, { x: number; y: number }>;
  stressError: number;
  scale: ScaleInfo;
}

// ──────────────────────────── Validación ────────────────────────────
```

```
211
212 export type ValidationCode =
213   | 'EMPTY_GRAPH'
214   | 'DUPLICATE_NODE_ID'
215   | 'DUPLICATE_EDGE_ID'
216   | 'MISSING_NODE'
217   | 'SELF_LOOP'
218   | 'DUPLICATE_EDGE'
219   | 'NON_POSITIVE_WEIGHT'
220   | 'INVALID_WEIGHT'
221   | 'DISCONNECTED';
222
223 /** `DISCONNECTED` es `warning` (el grafo se puede resolver; F3 informa "No existe ruta");
    el resto `error`. */
224 export interface ValidationIssue {
225   code: ValidationCode;
226   severity: 'error' | 'warning';
227   /** Mensaje en español, listo para mostrar. */
228   message: string;
229   nodeIds?: NodeId[];
```

```
230     edgeIds?: string[];
231   }
232
233   /** `ok` ⇔ no hay issues con severity 'error'. */
234   export interface ValidationResult {
235     ok: boolean;
236     issues: ValidationIssue[];
237   }
238
239   // ─────────────────────────── API del núcleo ────────────────────────────
240
241   /**
242    * Firmas públicas de src/core. Cada módulo exporta su función tipada contra esta interfaz,
243    * p. ej. `export const dijkstra: CoreApi['dijkstra'] = (g, s) => …`.
244    * Precondición común: el grafo pasó `validateGraph` sin errores y los NodeId existen;
245    * si no, la función lanza `Error` con mensaje en español.
246    */
247   export interface CoreApi {
248     /** graph.ts */
249     validateGraph(g: Graph): ValidationResult;
```

```
  /** dijkstra.ts — min-heap, O((V+E) log V), con todos los pasos. */
  dijkstra(g: Graph, s: NodeId): DijkstraResult;
  /**
   * dijkstra.ts — TODAS las rutas mínimas empatadas de s a t (máx. `maxTies`, por defecto 10),
   * en orden lexicográfico de secuencia. `[]` si no hay ruta. s = t → una ruta de distancia 0.
   */
  shortestPaths(g: Graph, s: NodeId, t: NodeId, maxTies?: number): PathResult[];
  /** tabular.ts */
  buildTabular(g: Graph, result: DijkstraResult): TabularModel;
  /** floyd.ts */
  floydWarshall(g: Graph): FloydResult;
  /** floyd.ts — reconstruye el camino i→j desde `next`; `null` si no hay. */
  floydPath(g: Graph, f: FloydResult, s: NodeId, t: NodeId): PathResult | null;
  /** allPaths.ts — DFS con backtracking; cae a Yen según `AllPathsOptions`.
`onProgress(encontradas)`. */
  allSimplePaths(
    g: Graph,
    s: NodeId,
    t: NodeId,
```

```
    opts: AllPathsOptions,
    onProgress?: (found: number) => void,
  ): AllPathsResult;
  /** yen.ts — los k caminos simples más cortos, en orden exacto. */
  yenKShortest(g: Graph, s: NodeId, t: NodeId, k: number): PathResult[];
  /** scale.ts */
  detectScale(weights: number[]): ScaleInfo;
  /** layout.ts — stress majorization. */
  suggestLayout(g: Graph): LayoutSuggestion;
  /** edgeShape.ts — forma sugerida SOLO para aristas con shapeAuto = true (id → forma). */
  autoEdgeShapes(g: Graph): Record<string, EdgeShape>;
}

/**
 * API expuesta por src/core/worker.ts vía Comlink. Cancelar = `worker.terminate()` y crear
 * uno nuevo (lo gestiona el store), por eso no hay método `cancel`.
 */
export interface CoreWorkerApi {
  allSimplePaths(
    g: Graph,
```

```
    s: NodeId,
    t: NodeId,
    opts: AllPathsOptions,
    onProgress?: (found: number) => void,
  ): Promise<AllPathsResult>;
  floydWarshall(g: Graph): Promise<FloydResult>;
  suggestLayout(g: Graph): Promise<LayoutSuggestion>;
}

// ─────────────────────────── Archivos de datos ────────────────────────────

/** Archivo de grafo guardado/cargado por el usuario y formato de src/data/ejemplo6.json.
*/
export interface GraphFile {
  format: 'rutaq-graph';
  version: 1;
  name: string;
  graph: Graph;
}
```

```ts
/** Nodo de src/data/quetzaltenango.json (sección 3.3 del plan). */
export interface DatasetNode {
  id: NodeId;
  label: string;
  lat: number;
  lon: number;
  fuente: string;
}

/** Arista de src/data/quetzaltenango.json. `geometry` = polyline codificada (precisión 5).
*/
export interface DatasetEdge {
  id: string;
  source: NodeId;
  target: NodeId;
  km: number;
  carretera: string;
  fuente: string;
  verificado_google: boolean;
  km_google: number | null;
```

```
    nota: string;
    geometry: string;
  }

  export interface Dataset {
    meta: { nombre: string; unidad: 'km'; fuentes: string[]; fecha: string };
    nodes: DatasetNode[];
    edges: DatasetEdge[];
  }

  // ═══════════════════════════ PROYECTO II — Transporte Express (solo adiciones)
  ═══════════════════════════

  // Nada de lo anterior cambia. Ver docs/PLAN_PROYECTO2.md.

  /**
   * Un paso de la solución tabulada del árbol de expansión mínima (AEM).
   * - `iter = 0`: inicialización (Prim: `inTree = [start]`; Kruskal: `inTree = []`),
  `chosenEdge = null`, `totalSoFar = 0`.
   * - Prim, `iter = k ≥ 1`: `chosenEdge` es la arista de menor peso que cruza el corte
  (árbol → fuera); su extremo nuevo
   *   se agrega a `inTree` y a `crossedOut` (columna tachada). `candidates` = aristas del
  corte evaluadas en el paso
```

```
344    *   (incluida la elegida), en orden de peso ascendente.
345    * - Kruskal, `iter = k ≥ 1`: se examina la k-ésima arista en orden ascendente de peso. Si une dos componentes distintas,
346    *   `chosenEdge = id`; si formaría ciclo, `chosenEdge = null` y `rejectedEdge = id`. `inTree` = nodos tocados por las
347    *   aristas aceptadas hasta ahora; `crossedOut = []`; `candidates = [id]`.
348    * - Desempate determinista: a igual peso gana el `id` de arista menor (comparación `<`, sin locale).
349    * - `totalSoFar` = suma de pesos aceptados hasta este paso (tolerancia relativa 1e-9 en comparaciones).
350    */
351   export interface MSTStep {
352     iter: number;
353     chosenEdge: string | null;
354     rejectedEdge?: string;
355     inTree: NodeId[];
356     crossedOut: NodeId[];
357     candidates: string[];
358     totalSoFar: number;
359   }
360
361   /**
```

```
362   * `edges` en orden de selección. Grafo conexo ⇒ `edges.length = V − 1`. Grafo desconexo ⇒
    bosque de expansión mínima
363   * (Kruskal) o árbol de la componente de `start` (Prim); la UI lo informa.
364   */
365  export interface MSTResult {
366    algorithm: 'prim' | 'kruskal';
367    edges: string[];
368    total: number;
369    steps: MSTStep[];
370  }
371
372  /**
373   * Desglose de una ruta para el asistente de Transporte Express.
374   * `legs[i]` = tramo nodes[i] → nodes[i+1]; `road` = `meta.carretera` de la arista si
    existe y no está vacía;
375   * `cumulative` = suma de km hasta ese tramo inclusive. `total` = suma de los tramos (=
    distancia de Dijkstra).
376   * `formula` = "12.5 + 30.2 + … = 245.7 km" (pesos tal como están en el grafo, total
    redondeado a 1 decimal, unidad
377   * del grafo). Ruta de un solo nodo ⇒ `legs = []`, `total = 0`, `formula = "0 km"`.
378   */
379  export interface RouteBreakdown {
```

```
  nodes: NodeId[];
  legs: { from: NodeId; to: NodeId; edgeId: string; road?: string; km: number; cumulative: number }[];
  total: number;
  formula: string;
}

/** Recorrido de reparto aproximado (vecino más cercano + 2-opt) sobre distancias de camino más corto. Cerrado: vuelve a `order[0]`. */
export interface TourResult {
  order: NodeId[];
  total: number;
  method: 'nn+2opt';
}

/** API añadida del núcleo (Proyecto II). Mismas convenciones que `CoreApi`. */
export interface CoreApiP2 {
  /** mst.ts — Prim con min-heap desde `start` (por defecto, el primer nodo de `graph.nodes`). */
  prim(g: Graph, start?: NodeId): MSTResult;
  /** mst.ts — Kruskal con Union-Find (compresión de caminos y unión por rango). */
  kruskal(g: Graph): MSTResult;
```

```
  /** route.ts — desglose de una ruta (`path` de `shortestPaths` o `PathResult`
equivalente). */
  routeBreakdown(g: Graph, path: PathResult): RouteBreakdown;
  /** tour.ts — recorrido de reparto que visita todos los nodos alcanzables desde `start`.
*/
  deliveryTour(g: Graph, start: NodeId): TourResult;
}

/** API del Worker con las adiciones del Proyecto II (el Worker existente la implementa
completa). */
export interface CoreWorkerApiP2 extends CoreWorkerApi {
  deliveryTour(g: Graph, start: NodeId): Promise<TourResult>;
}

/** Nodo de src/data/guatemala.json: `label` = nombre del departamento; `cabecera` =
cabecera departamental. */
export interface GuatemalaDatasetNode extends DatasetNode {
  cabecera: string;
}

/** src/data/guatemala.json: mismo formato que `Dataset`, con la cabecera en cada nodo. */
export interface GuatemalaDataset extends Omit<Dataset, 'nodes'> {
```

```
  nodes: GuatemalaDatasetNode[];
}
```

Estado de la aplicación

Store global: datos de la red, formularios, consultas y resultados.

`src/store/dataset.ts`

100 líneas.

```ts
import type { Dataset, GNode, Graph, GuatemalaDataset } from '../core/types';



export const DEFAULT_NODE_SIZE = { w: 132, h: 44 } as const;

/**
 * En el mapa real cada municipio es un punto (la UI dibuja el nombre al lado): así se conserva la posición
 * geográfica exacta sin apartar nodos que, a escala, quedan a 1.5–3 km entre sí.
 */
export const DATASET_NODE_SIZE = { w: 18, h: 18 } as const;



/** Ancho del mapa en px. Se expone para que la UI dibuje la barra de escala en km. */
export const DATASET_WIDTH_PX = 1400;



/**
```

```
 * Proyecciones con el norte arriba:
 * - `equirect`: equirectangular centrada en la latitud media (Quetzaltenango, ~50 km: error de escala < 2 %).
 * - `mercator`: Web Mercator, como Google Maps (Guatemala completa, ~450 km de norte a sur).
 */
export type ProjectionKind = 'equirect' | 'mercator';

/** Proyección lat/lon ↔ lienzo. `project` devuelve el CENTRO del punto en px del lienzo. */
export interface GeoProjection {
  kind: ProjectionKind;
  project(lat: number, lon: number): { x: number; y: number };
  unproject(x: number, y: number): { lat: number; lon: number };
}

const RAD = Math.PI / 180;

/** Construye la proyección que ajusta `points` a `widthPx` de ancho (el oeste en x = 0, el punto más al norte en y = 0). */
export function makeProjection(
  points: { lat: number; lon: number }[],
```

```
  kind: ProjectionKind,
  widthPx = DATASET_WIDTH_PX,
): GeoProjection {
  const lat0 = points.length ? points.reduce((a, p) => a + p.lat, 0) / points.length : 0;
  const cos0 = Math.cos(lat0 * RAD);
  const fwd =
    kind === 'mercator'
      ? (lat: number, lon: number) => ({ px: lon * RAD, py: -Math.log(Math.tan(Math.PI / 4
+ (lat * RAD) / 2)) })
      : (lat: number, lon: number) => ({ px: lon * cos0, py: -lat });
  const inv =
    kind === 'mercator'
      ? (px: number, py: number) => ({ lat: (2 * Math.atan(Math.exp(-py)) - Math.PI / 2) /
RAD, lon: px / RAD })
      : (px: number, py: number) => ({ lat: -py, lon: px / cos0 });

  const raw = points.map((p) => fwd(p.lat, p.lon));
  const minX = raw.length ? Math.min(...raw.map((r) => r.px)) : 0;
  const maxX = raw.length ? Math.max(...raw.map((r) => r.px)) : 0;
  const minY = raw.length ? Math.min(...raw.map((r) => r.py)) : 0;
  const k = maxX > minX ? widthPx / (maxX - minX) : 1;
```

```
51
52    return {
53      kind,
54      project(lat, lon) {
55        const { px, py } = fwd(lat, lon);
56        return { x: (px - minX) * k, y: (py - minY) * k };
57      },
58      unproject(x, y) {
59        return inv(x / k + minX, y / k + minY);
60      },
61    };
62  }
63
64  /**
65   * Convierte el dataset geográfico en un Graph del lienzo con la proyección indicada (equirectangular por defecto,
66   * que es la del Proyecto I). (x, y) del nodo es la esquina superior izquierda del punto.
67   */
68  export function datasetToGraph(
69    ds: Dataset | GuatemalaDataset,
```

```
70    widthPx = DATASET_WIDTH_PX,
71    kind: ProjectionKind = 'equirect',
72  ): Graph {
73    if (ds.nodes.length === 0) return { nodes: [], edges: [], directed: false, unit:
    ds.meta.unidad };
74
75    const proj = makeProjection(ds.nodes, kind, widthPx);
76    const nodes: GNode[] = ds.nodes.map((n) => {
77      const c = proj.project(n.lat, n.lon);
78      return {
79        id: n.id,
80        label: n.label,
81        x: Math.round(c.x - DATASET_NODE_SIZE.w / 2),
82        y: Math.round(c.y - DATASET_NODE_SIZE.h / 2),
83        ...DATASET_NODE_SIZE,
84        lat: n.lat,
85        lon: n.lon,
86      };
87    });
88
```

```
  const edges = ds.edges.map(({ id, source, target, km, ...meta }) => ({
    id,
    source,
    target,
    weight: km,
    shape: 'straight' as const,
    shapeAuto: true,
    meta,
  }));

  return { nodes, edges, directed: false, unit: ds.meta.unidad };
}
```

`src/store/types.ts`

201 líneas.

```
/**
 * CONTRATO DEL STORE — v1 (congelado en F0). Dueño: Claude Code.
 * La UI (Antigravity) lee estado y llama acciones SOLO a través de esta interfaz; puede
 * construir un mock que la implemente mientras el store real se integra.
 */
import type {
```

```
  AllPathsResult,
  DijkstraResult,
  EdgeShape,
  FloydResult,
  Graph,
  GraphFile,
  GuatemalaDataset,
  LayoutSuggestion,
  MSTResult,
  NodeId,
  PathResult,
  RouteBreakdown,
  ScaleInfo,
  TabularConfig,
  TabularModel,
  TourResult,
  ValidationResult,
} from '../core/types';

```

```
/** `guatemala` = módulo Transporte Express (Proyecto II). */
export type AppMode = 'quetzaltenango' | 'guatemala' | 'design';
export type ResultTab = 'tabla' | 'grafica' | 'rutas' | 'floyd';
export type Theme = 'light' | 'dark' | 'system';

/** F2 = árbol desde un nodo; F3 = ruta entre dos nodos (target null mientras se elige). */
export type Query = { kind: 'tree'; source: NodeId } | { kind: 'path'; source: NodeId; target: NodeId | null };

export type JobStatus = 'idle' | 'running' | 'done' | 'cancelled' | 'error';

export interface AllPathsState {
  status: JobStatus;
  /** Rutas encontradas hasta ahora (progreso del Worker). */
  found: number;
  result: AllPathsResult | null;
  error: string | null;
}

/** Reproducción de la tabla S1; `stepIndex` indexa `dijkstra.steps` y sincroniza lienzo ↔ tabla. */
```

```
45  export interface PlaybackState {
46    stepIndex: number;
47    playing: boolean;
48    /** Milisegundos entre pasos al reproducir. */
49    speedMs: number;
50  }
51
52  export type AddEdgeResult =
53    | { ok: true; edgeId: string }
54    | { ok: false; reason: 'self-loop' | 'invalid-weight' | 'missing-node' }
55    /** Ya existe arista entre el par: la UI ofrece editar `existingEdgeId`. */
56    | { ok: false; reason: 'duplicate'; existingEdgeId: string };
57
58  export type ImportResult = { ok: true } | { ok: false; message: string };
59
60  // ──────────────────────────── Proyecto II — Transporte Express
    ────────────────────────────
61
62  export type Template = 'quetzaltenango' | 'guatemala' | 'ejemplo6' | 'ejemplo_mst' |
    'vacio';
63
```

```ts
/** G4: qué responde «recorrer y conectar todos los nodos». */
export type CoverMode = 'mst' | 'spt' | 'tour';

export interface DepartmentInput {
  label: string;
  cabecera: string;
  /** Si hay lat y lon, el punto se proyecta en su posición real; si no, se coloca en `at` (centro, px del lienzo). */
  lat?: number;
  lon?: number;
  at?: { x: number; y: number };
}

export type AddDepartmentResult =
  { ok: true; id: NodeId } | { ok: false; reason: 'empty-name' | 'duplicate-name' | 'invalid-coords' };

export interface TourState {
  status: JobStatus;
  start: NodeId | null;
  result: TourResult | null;
```

```
  error: string | null;
}

export interface AppState {
  // ── Datos ──
  mode: AppMode;
  graph: Graph;
  graphName: string;
  validation: ValidationResult;
  scale: ScaleInfo;

  // ── Selección y consulta ──
  selectedNodeIds: NodeId[];
  selectedEdgeIds: string[];
  query: Query | null;

  // ── Resultados (se recalculan al cambiar graph o query) ──
  dijkstra: DijkstraResult | null;
  tabular: TabularModel | null;
```

```
  /** Rutas mínimas empatadas de F3 ([] = no existe ruta; null = sin consulta F3 completa). */
  shortest: PathResult[] | null;
  floyd: FloydResult | null;
  allPaths: AllPathsState;
  /** Índice en `allPaths.result.paths` (o en `shortest` si viene de empates) resaltado en el lienzo. */
  highlightedPath: { from: 'shortest' | 'all'; index: number } | null;
  playback: PlaybackState;
  layoutSuggestion: LayoutSuggestion | null;

  // ── Preferencias de UI ──
  activeTab: ResultTab;
  tabularConfig: TabularConfig;
  allPathsLimit: number;
  theme: Theme;
  keepProportion: boolean;

  // ── Acciones: modo y archivos ──
  setMode(mode: AppMode): void;
  loadTemplate(template: Template): void;
```

```
    exportGraphFile(): GraphFile;
    importGraphFile(json: string): ImportResult;

    // ── Acciones: edición (modo diseño; todas registran deshacer/rehacer) ──
    addNode(at: { x: number; y: number }, label?: string): NodeId;
    moveNodes(moves: Record<NodeId, { x: number; y: number }>): void;
    resizeNode(id: NodeId, size: { x: number; y: number; w: number; h: number }): void;
    renameNode(id: NodeId, label: string): void;
    removeNodes(ids: NodeId[]): void;
    addEdge(source: NodeId, target: NodeId, weight: number): AddEdgeResult;
    setEdgeWeight(id: string, weight: number): boolean;
    /** `ids = 'all'` cambia todas; deja `shapeAuto = false` en las afectadas. */
    setEdgeShape(ids: string[] | 'all', shape: EdgeShape): void;
    /** Vuelve a `shapeAuto = true` y recalcula la forma automática. */
    resetEdgeShapeAuto(ids: string[] | 'all'): void;
    removeEdges(ids: string[]): void;
    applySuggestedLayout(): Promise<void>;
    undo(): void;
    redo(): void;
    canUndo(): boolean;
```

```
    canRedo(): boolean;

    // ── Acciones: selección y consulta ──
    select(nodeIds: NodeId[], edgeIds: string[]): void;
    setTreeQuery(source: NodeId): void;
    setPathQuery(source: NodeId, target: NodeId | null): void;
    clearQuery(): void;

    // ── Acciones: reproducción S1 ──
    stepTo(index: number): void;
    nextStep(): void;
    prevStep(): void;
    play(): void;
    pause(): void;
    toEnd(): void;
    setSpeed(ms: number): void;

    // ── Acciones: S3 ──
    runAllPaths(): void;
```

```
    cancelAllPaths(): void;
    setAllPathsLimit(limit: number): void;
    highlightPath(sel: AppState['highlightedPath']): void;

    // ── Proyecto II: Transporte Express (estado) ──
    /** Cabecera de cada departamento del módulo Guatemala (vacío en los otros modos). */
    departments: Record<NodeId, { cabecera: string }>;
    /** Contorno del país proyectado al lienzo (anillos de puntos en px), solo en modo `guatemala`. */
    outline: { x: number; y: number }[][] | null;
    /** Desglose de cada ruta mínima de `shortest` (mismo índice): listado, tramos, acumulados y fórmula. */
    breakdown: RouteBreakdown[] | null;
    coverMode: CoverMode;
    /** Árbol de expansión mínima con ambos algoritmos (deben dar el mismo total). */
    mst: { prim: MSTResult; kruskal: MSTResult } | null;
    /** Nodo de inicio de Prim (por defecto el primero). */
    mstStart: NodeId | null;
    tour: TourState;

    // ── Proyecto II: Transporte Express (acciones; editar registra deshacer/rehacer y autoguarda) ──
```

```
    addDepartment(input: DepartmentInput): AddDepartmentResult;
    /** Cambia nombre, cabecera o coordenadas (si cambian lat/lon, el punto se reubica). */
    updateDepartment(id: NodeId, patch: Partial<Omit<DepartmentInput, 'at'>>):
AddDepartmentResult;
    /** Carretera bidireccional con nombre opcional (CA-1, CA-9…). Mismas validaciones que
`addEdge`. */
    addRoad(source: NodeId, target: NodeId, km: number, road?: string): AddEdgeResult;
    updateRoad(id: string, patch: { km?: number; road?: string }): boolean;
    /** Descarta los cambios guardados del módulo Guatemala y vuelve al dataset original. */
    restoreOriginal(): void;
    /** Exporta el módulo Guatemala como `GuatemalaDataset` (conserva cabeceras y
coordenadas). */
    exportGuatemalaDataset(): GuatemalaDataset;
    setCoverMode(mode: CoverMode): void;
    setMstStart(id: NodeId): void;
    runTour(start: NodeId): void;
    cancelTour(): void;
    /** Proyecta lat/lon con la proyección del mapa actual (centro del punto en px); null si
el grafo no es geográfico. */
    projectLatLon(lat: number, lon: number): { x: number; y: number } | null;

    // ── Acciones: preferencias ──
```

```
    setActiveTab(tab: ResultTab): void;
    setTabularConfig(patch: Partial<TabularConfig>): void;
    setTheme(theme: Theme): void;
    setKeepProportion(on: boolean): void;
  }
```

`src/store/useAppStore.ts`

946 líneas.

```
import { create } from 'zustand';
import { temporal, type TemporalState } from 'zundo';
import * as Comlink from 'comlink';
import {
  autoEdgeShapes,
  buildTabular,
  detectScale,
  dijkstra,
  floydWarshall,
  kruskal,
  prim,
  routeBreakdown,
  shortestPaths,
```

```
14    validateGraph,
15  } from '../core';
16  import type {
17    CoreWorkerApiP2,
18    Dataset,
19    GEdge,
20    GNode,
21    Graph,
22    GraphFile,
23    GuatemalaDataset,
24    NodeId,
25    TabularConfig,
26    ValidationResult,
27  } from '../core';
28  import { DATASET_NODE_SIZE, DEFAULT_NODE_SIZE, datasetToGraph, makeProjection, type
    GeoProjection } from './dataset';
29  import type { AllPathsState, AppMode, AppState, Query, Template, Theme, TourState } from
    './types';
30
31  // ─────────────────────────────── Datos empaquetados ────────────────────────────────
32
```

```
33  // glob (y no import directo) para que el build no falle mientras Gemini entrega los JSON.
34  const dataFiles = import.meta.glob<{ default: unknown }>('../data/*.json', { eager: true
    });
35  const dataFile = (name: string): unknown => dataFiles[`../data/${name}.json`]?.default;
36
37  const EMPTY_GRAPH: Graph = { nodes: [], edges: [], directed: false, unit: 'km' };
38
39  // ──────────── Proyecto II: módulo Guatemala (Transporte Express) ────────────
40
41  const guatemalaOriginal = dataFile('guatemala') as GuatemalaDataset | undefined;
42  /**
43   * Proyección Web Mercator fija, construida con las cabeceras del dataset ORIGINAL: así un
    departamento agregado o
44   * importado cae siempre en el mismo lugar y el contorno del país coincide con los puntos.
45   */
46  const gtProjection: GeoProjection | null = guatemalaOriginal?.nodes.length
47    ? makeProjection(guatemalaOriginal.nodes, 'mercator')
48    : null;
49  const qtzDataset = dataFile('quetzaltenango') as Dataset | undefined;
50  const qtzProjection: GeoProjection | null = qtzDataset?.nodes.length
51    ? makeProjection(qtzDataset.nodes, 'equirect')
```

```
    : null;


/** Grafo del módulo Guatemala con la proyección fija; devuelve también las cabeceras. */
function guatemalaToGraph(ds: GuatemalaDataset): { graph: Graph; departments:
Record<NodeId, { cabecera: string }> } {
  const proj = gtProjection ?? makeProjection(ds.nodes, 'mercator');
  const base = datasetToGraph(ds, undefined, 'mercator');
  const nodes = base.nodes.map((n) => {
    const c = proj.project(n.lat!, n.lon!);
    return { ...n, x: Math.round(c.x - DATASET_NODE_SIZE.w / 2), y: Math.round(c.y -
DATASET_NODE_SIZE.h / 2) };
  });
  const departments = Object.fromEntries(ds.nodes.map((n) => [n.id, { cabecera: n.cabecera
|| n.label }]));
  return { graph: { ...base, nodes }, departments };
}


type GeoGeometry = { type?: string; coordinates?: unknown };


/** Anillos del contorno de Guatemala (GeoJSON en lon/lat) proyectados al lienzo. */
function guatemalaOutline(): { x: number; y: number }[][] | null {
```

```
  const geo = dataFile('guatemala_contorno') as
    (GeoGeometry & { geometry?: GeoGeometry; features?: { geometry?: GeoGeometry }[] }) |
undefined;
  const proj = gtProjection;
  if (!geo || !proj) return null;
  const geometries: GeoGeometry[] = geo.features
    ? geo.features.map((f) => f.geometry ?? {})
    : geo.geometry
      ? [geo.geometry]
      : [geo];
  const rings: number[][][] = [];
  for (const gm of geometries) {
    if (gm.type === 'Polygon') rings.push(...(gm.coordinates as number[][][]));
    if (gm.type === 'MultiPolygon') for (const poly of gm.coordinates as number[][][][])
rings.push(...poly);
  }
  if (!rings.length) return null;
  return rings.map((ring) => ring.map(([lon, lat]) => proj.project(lat!, lon!)));
}


function guatemalaTemplate() {
```

```
  if (!guatemalaOriginal) return { graph: EMPTY_GRAPH, name: 'Guatemala (sin datos)', departments: {} };
  const { graph, departments } = guatemalaToGraph(guatemalaOriginal);
  return { graph, name: guatemalaOriginal.meta.nombre, departments };
}


function templateGraph(t: Template): { graph: Graph; name: string } {
  if (t === 'guatemala') return guatemalaTemplate();
  if (t === 'ejemplo_mst') {
    const f = dataFile('ejemplo_mst') as GraphFile | undefined;
    return { graph: f?.graph ?? EMPTY_GRAPH, name: f?.name ?? 'Ejemplo de árbol de expansión mínima (sin datos)' };
  }
  if (t === 'quetzaltenango') {
    const ds = dataFile('quetzaltenango') as Dataset | undefined;
    return { graph: ds ? datasetToGraph(ds) : EMPTY_GRAPH, name: ds?.meta.nombre ?? 'Quetzaltenango (sin datos)' };
  }
  if (t === 'ejemplo6') {
    const f = dataFile('ejemplo6') as GraphFile | undefined;
    return { graph: f?.graph ?? EMPTY_GRAPH, name: f?.name ?? 'Ejemplo de 6 nodos (sin datos)' };
```

```
107     }
108     return { graph: { ...EMPTY_GRAPH, unit: 'u' }, name: 'Mapa nuevo' };
109   }
110
111   // ──────────────────────────── Persistencia local ────────────────────────────
112
113   const KEY_DESIGN = 'rutaq:design';
114   const KEY_GUATEMALA = 'rutaq:guatemala';
115   const KEY_PREFS = 'rutaq:prefs';
116
117   const storage = typeof localStorage === 'undefined' ? null : localStorage;
118
119   function readJson<T>(key: string): T | null {
120     try {
121       const raw = storage?.getItem(key);
122       return raw ? (JSON.parse(raw) as T) : null;
123     } catch {
124       return null;
125     }
126   }
```

```
127
128 function writeJson(key: string, value: unknown) {
129   try {
130     storage?.setItem(key, JSON.stringify(value));
131   } catch {
132     // Cuota llena o modo privado: el autoguardado es opcional.
133   }
134 }
135
136 // ──────────────────────────── Derivados ────────────────────────────
137
138 /** Ejecuta una función del núcleo sin dejar que un error rompa la UI. */
139 function safe<T>(fn: () => T, fallback: T): T {
140   try {
141     return fn();
142   } catch (e) {
143     console.warn('[RutaQ]', e instanceof Error ? e.message : e);
144     return fallback;
145   }
```

```
146   }
147
148   const NOT_VALIDATED: ValidationResult = { ok: false, issues: [] };
149
150   function applyAutoShapes(g: Graph): Graph {
151     const shapes = safe(() => autoEdgeShapes(g), {});
152     if (!g.edges.some((e) => e.shapeAuto && shapes[e.id] && shapes[e.id] !== e.shape)) return
      g;
153     return { ...g, edges: g.edges.map((e) => (e.shapeAuto && shapes[e.id] ? { ...e, shape:
      shapes[e.id]! } : e)) };
154   }
155
156   /** Recalcula validación, escala y resultados a partir de (graph, query). */
157   function derive(graph: Graph, query: Query | null) {
158     const ids = new Set(graph.nodes.map((n) => n.id));
159     if (query && (!ids.has(query.source) || (query.kind === 'path' && query.target &&
      !ids.has(query.target)))) {
160       query = null;
161     }
162     const validation = safe(() => validateGraph(graph), NOT_VALIDATED);
163     const scale = safe(() => detectScale(graph.edges.map((e) => e.weight)), { magnitude: 1,
      range: '—', pxPerUnit: 1 });
```

```
164    const solvable = validation.ok && graph.nodes.length > 0;
165
166    const floyd = solvable ? safe(() => floydWarshall(graph), null) : null;
167    const dj = solvable && query ? safe(() => dijkstra(graph, query.source), null) : null;
168    const tabular = dj ? safe(() => buildTabular(graph, dj), null) : null;
169    const shortest =
170      solvable && query?.kind === 'path' && query.target
171        ? safe(() => shortestPaths(graph, query.source, query.target!), null)
172        : null;
173    // Proyecto II: desglose de cada ruta mínima (tramos, acumulados, fórmula) y árbol de
   expansión mínima.
174    const breakdown = shortest ? safe(() => shortest.map((p) => routeBreakdown(graph, p)),
   null) : null;
175    const start = mstStart && ids.has(mstStart) ? mstStart : (graph.nodes[0]?.id ?? null);
176    const mst = validation.ok && start ? safe(() => ({ prim: prim(graph, start), kruskal:
   kruskal(graph) }), null) : null;
177
178    return {
179      breakdown,
180      mst,
181      mstStart: start,
182      query,
```

```
      validation,
      scale,
      floyd,
      dijkstra: dj,
      tabular,
      shortest,
      highlightedPath: null,
      allPaths: IDLE_ALL_PATHS,
      // Por defecto se muestra la solución completa; «Reproducir» reinicia desde la iteración 0.
      playback: { stepIndex: dj ? dj.steps.length - 1 : 0, playing: false, speedMs },
    };
}

const IDLE_ALL_PATHS: AllPathsState = { status: 'idle', found: 0, result: null, error: null };

let speedMs = 900;
/** Inicio de Prim elegido por el usuario (se valida contra el grafo en `derive`). */
let mstStart: NodeId | null = null;
```

```
const IDLE_TOUR: TourState = { status: 'idle', start: null, result: null, error: null };
let tourRun = 0;

// ──────────────────────────── Worker de S3 / layout ────────────────────────────

let worker: Worker | null = null;
let workerApi: Comlink.Remote<CoreWorkerApiP2> | null = null;

function getWorker(): Comlink.Remote<CoreWorkerApiP2> {
  if (!workerApi) {
    worker = new Worker(new URL('../core/worker.ts', import.meta.url), { type: 'module' });
    workerApi = Comlink.wrap<CoreWorkerApiP2>(worker);
  }
  return workerApi;
}

/** Cancelar = terminar el Worker (corta un DFS en curso de inmediato); el próximo uso crea otro. */
function killWorker() {
  worker?.terminate();
```

```
    worker = null;
    workerApi = null;
  }

  let allPathsRun = 0;
  let playTimer: ReturnType<typeof setInterval> | null = null;

  function stopTimer() {
    if (playTimer) clearInterval(playTimer);
    playTimer = null;
  }

  // ——————————————————————— Utilidades de edición ———————————————————————

  function nextNodeId(g: Graph): NodeId {
    let i = g.nodes.length + 1;
    const ids = new Set(g.nodes.map((n) => n.id));
    while (ids.has(`n${i}`)) i++;
    return `n${i}`;
  }
```

```
241
242  /** A, B, …, Z, AA, AB… — la primera etiqueta que no esté en uso. */
243  function nextLabel(g: Graph): string {
244    const used = new Set(g.nodes.map((n) => n.label));
245    for (let i = 0; ; i++) {
246      let s = '';
247      for (let k = i; k >= 0; k = Math.floor(k / 26) - 1) s = String.fromCharCode(65 + (k %
   26)) + s;
248      if (!used.has(s)) return s;
249    }
250  }
251
252  const samePair = (e: GEdge, a: NodeId, b: NodeId) =>
253    (e.source === a && e.target === b) || (e.source === b && e.target === a);
254
255  const validWeight = (w: number) => Number.isFinite(w) && w > 0;
256
257  function isGraph(v: unknown): v is Graph {
258    const g = v as Graph;
259    return (
```

```
    !!g &&
    Array.isArray(g.nodes) &&
    Array.isArray(g.edges) &&
    g.nodes.every((n) => typeof n.id === 'string' && Number.isFinite(n.x) &&
Number.isFinite(n.y)) &&
    g.edges.every((e) => typeof e.id === 'string' && typeof e.source === 'string' && typeof
e.weight === 'number')
  );
}

/** Completa campos opcionales de un grafo importado (tamaño, forma, unidad). */
function normalizeGraph(g: Graph): Graph {
  return {
    directed: false,
    unit: typeof g.unit === 'string' && g.unit ? g.unit : 'u',
    nodes: g.nodes.map((n) => ({
      ...n,
      label: typeof n.label === 'string' ? n.label : n.id,
      w: Number.isFinite(n.w) && n.w > 0 ? n.w : DEFAULT_NODE_SIZE.w,
      h: Number.isFinite(n.h) && n.h > 0 ? n.h : DEFAULT_NODE_SIZE.h,
    })),
```

```
    edges: g.edges.map((e) => ({ ...e, shape: e.shape ?? 'straight', shapeAuto: e.shapeAuto ?? true })),
  };
}

/** Quita tildes y pasa a minúsculas para comparar nombres. */
const plain = (s: string) =>
  s
    .normalize('NFD')
    .replace(/[\u0300-\u036f]/g, '')
    .trim()
    .toLowerCase();

/** Id de 3 letras a partir del nombre (sin tildes), único en el grafo: «Petén» → PET, luego PE2… */
function departmentId(g: Graph, label: string): NodeId {
  const base =
    plain(label)
      .replace(/[^a-z]/g, '')
      .toUpperCase()
      .slice(0, 3) || 'DEP';
```

```
298     const ids = new Set(g.nodes.map((n) => n.id));
299     if (!ids.has(base)) return base;
300     for (let i = 2; ; i++) {
301       const id = `${base.slice(0, 2)}${i}`;
302       if (!ids.has(id)) return id;
303     }
304   }
305
306   const validCoords = (lat?: number, lon?: number) =>
307     lat === undefined && lon === undefined
308       ? true
309       : Number.isFinite(lat) && Number.isFinite(lon) && Math.abs(lat!) <= 90 &&
      Math.abs(lon!) <= 180;
310
311   // ──────────────────────────── Tema ────────────────────────────
312
313   function applyTheme(theme: Theme) {
314     if (typeof document === 'undefined') return;
315     const dark =
316       theme === 'dark' ||
```

```
317     (theme === 'system' && typeof matchMedia !== 'undefined' && matchMedia('(prefers-color-
    scheme: dark)').matches);
318   document.documentElement.classList.toggle('dark', dark);
319 }
320
321 // ──────────────────────────── Store ────────────────────────────
322
323 const DEFAULT_TABULAR: TabularConfig = {
324   showAdjacency: true,
325   labelFormat: 'paren',
326   crossOut: 'column',
327   repeatPermanent: false,
328 };
329
330 interface Prefs {
331   theme: Theme;
332   tabularConfig: TabularConfig;
333   allPathsLimit: number;
334   keepProportion: boolean;
335   mode: AppMode;
```

```
}

const prefs = readJson<Partial<Prefs>>(KEY_PREFS) ?? {};

function initialGraph(mode: AppMode): {
  graph: Graph;
  name: string;
  departments?: Record<NodeId, { cabecera: string }>;
} {
  if (mode === 'guatemala') {
    const saved = readJson<GuatemalaDataset>(KEY_GUATEMALA);
    if (saved && Array.isArray(saved.nodes) && Array.isArray(saved.edges) && saved.meta) {
      try {
        return { ...guatemalaToGraph(saved), name: saved.meta.nombre };
      } catch {
        // Guardado dañado: se usa el dataset original.
      }
    }
    return guatemalaTemplate();
  }
```

```
356     if (mode === 'design') {
357       const saved = readJson<GraphFile>(KEY_DESIGN);
358       if (saved && isGraph(saved.graph)) return { graph: normalizeGraph(saved.graph), name:
    saved.name };
359       return templateGraph('ejemplo6');
360     }
361     return templateGraph('quetzaltenango');
362   }
363
364   const initialMode: AppMode = prefs.mode ?? 'quetzaltenango';
365   const initial = initialGraph(initialMode);
366   const initialG = applyAutoShapes(initial.graph);
367
368   /** Historial de deshacer/rehacer (con tipo explícito para evitar la referencia circular).
    */
369   function history(): TemporalState<{ graph: Graph; departments: Record<NodeId, { cabecera:
    string }> }> {
370     return useAppStore.temporal.getState();
371   }
372
373   export const useAppStore = create<AppState>()(
```

```
374    temporal(
375      (set, get) => {
376        /** Cambia el grafo y recalcula todo lo derivado en un solo `set` (= una entrada de historial). */
377        const commitGraph = (graph: Graph, extra: Partial<AppState> = {}) => {
378          stopTimer();
379          killWorkerIfRunning();
380          const g = applyAutoShapes(graph);
381          set({ graph: g, layoutSuggestion: null, ...derive(g, get().query), ...extra });
382        };
383
384        const killWorkerIfRunning = () => {
385          if (get().allPaths.status === 'running') {
386            allPathsRun++;
387            killWorker();
388          }
389        };
390
391        const setQuery = (query: Query | null) => {
392          stopTimer();
```

```
393           killWorkerIfRunning();
394           set(derive(get().graph, query));
395         };
396
397         /** Reemplaza el grafo sin dejar historial (cambio de modo, plantilla, importación).
    */
398         const replaceGraph = (graph: Graph, name: string, extra: Partial<AppState> = {}) => {
399           stopTimer();
400           killWorkerIfRunning();
401           const g = applyAutoShapes(graph);
402           set({
403             graph: g,
404             graphName: name,
405             selectedNodeIds: [],
406             selectedEdgeIds: [],
407             layoutSuggestion: null,
408             tour: IDLE_TOUR,
409             departments: {},
410             ...derive(g, null),
411             ...extra,
```

```
412          });
413          set({ outline: get().mode === 'guatemala' ? guatemalaOutline() : null });
414          history().clear();
415        };
416
417        /** Coloca un departamento: con coordenadas en su posición real; si no, en `at` o en el centro del mapa. */
418        const placeDepartment = (g: Graph, lat?: number, lon?: number, at?: { x: number; y: number }) => {
419          if (lat !== undefined && lon !== undefined && gtProjection) return gtProjection.project(lat, lon);
420          if (at) return at;
421          const xs = g.nodes.map((n) => n.x + n.w / 2);
422          const ys = g.nodes.map((n) => n.y + n.h / 2);
423          return xs.length
424            ? { x: (Math.min(...xs) + Math.max(...xs)) / 2, y: (Math.min(...ys) + Math.max(...ys)) / 2 }
425            : { x: 0, y: 0 };
426        };
427
428        return {
429          mode: initialMode,
```

```
      graph: initialG,
      graphName: initial.name,
      selectedNodeIds: [],
      selectedEdgeIds: [],
      layoutSuggestion: null,
      activeTab: 'grafica',
      tabularConfig: { ...DEFAULT_TABULAR, ...prefs.tabularConfig },
      allPathsLimit: prefs.allPathsLimit ?? 5000,
      theme: prefs.theme ?? 'system',
      keepProportion: prefs.keepProportion ?? false,
      departments: initial.departments ?? {},
      outline: initialMode === 'guatemala' ? guatemalaOutline() : null,
      coverMode: 'mst',
      tour: IDLE_TOUR,
      ...derive(initialG, null),

    // ── Modo y archivos ──
    setMode(mode) {
      if (mode === get().mode) return;
      const t = initialGraph(mode);
```

```
450             replaceGraph(t.graph, t.name, { mode, departments: t.departments ?? {} });
451           },
452           loadTemplate(template) {
453             if (template === 'guatemala') {
454               const t = guatemalaTemplate();
455               replaceGraph(t.graph, t.name, { mode: 'guatemala', departments: t.departments
    });
456               return;
457             }
458             const t = templateGraph(template);
459             replaceGraph(t.graph, t.name, { mode: template === 'quetzaltenango' ?
    'quetzaltenango' : 'design' });
460           },
461           exportGraphFile() {
462             const { graph, graphName } = get();
463             return { format: 'rutaq-graph', version: 1, name: graphName, graph };
464           },
465           importGraphFile(json) {
466             let data: unknown;
467             try {
468               data = JSON.parse(json);
```

```
469             } catch {
470               return { ok: false, message: 'El archivo no es un JSON válido.' };
471             }
472             const f = data as Partial<GraphFile> & Partial<Dataset>;
473             let graph: Graph | null = null;
474             let name = 'Mapa importado';
475             if (f.format === 'rutaq-graph' && isGraph(f.graph)) {
476               graph = normalizeGraph(f.graph);
477               name = typeof f.name === 'string' ? f.name : name;
478             } else if (f.meta && Array.isArray(f.nodes) && Array.isArray(f.edges)) {
479               // Proyecto II: un dataset con cabeceras es del módulo Guatemala (exportado con
    «Guardar» en ese modo).
480               if (f.nodes.every((n) => typeof (n as { cabecera?: unknown }).cabecera ===
    'string')) {
481                 const gt = guatemalaToGraph(f as GuatemalaDataset);
482                 replaceGraph(gt.graph, f.meta.nombre ?? name, { mode: 'guatemala',
    departments: gt.departments });
483                 return { ok: true };
484               }
485               graph = datasetToGraph(f as Dataset);
486               name = f.meta.nombre ?? name;
```

```
            }
            if (!graph) return { ok: false, message: 'El archivo no tiene el formato de un mapa de RutaQ.' };
            replaceGraph(graph, name, { mode: 'design' });
            return { ok: true };
          },

          // ── Edición ──
          addNode(at, label) {
            const g = get().graph;
            const id = nextNodeId(g);
            const node = {
              id,
              label: label?.trim() || nextLabel(g),
              x: at.x - DEFAULT_NODE_SIZE.w / 2,
              y: at.y - DEFAULT_NODE_SIZE.h / 2,
              ...DEFAULT_NODE_SIZE,
            };
            commitGraph({ ...g, nodes: [...g.nodes, node] });
            return id;
```

```
506         },
507         moveNodes(moves) {
508           const g = get().graph;
509           commitGraph({ ...g, nodes: g.nodes.map((n) => (moves[n.id] ? { ...n,
    ...moves[n.id] } : n)) });
510         },
511         resizeNode(id, box) {
512           const g = get().graph;
513           commitGraph({
514             ...g,
515             nodes: g.nodes.map((n) =>
516               n.id === id ? { ...n, ...box, w: Math.max(40, box.w), h: Math.max(28, box.h)
    } : n,
517             ),
518           });
519         },
520         renameNode(id, label) {
521           const g = get().graph;
522           const clean = label.trim();
523           if (!clean) return;
524           commitGraph({ ...g, nodes: g.nodes.map((n) => (n.id === id ? { ...n, label: clean
    } : n)) });
```

```
525         },
526         removeNodes(ids) {
527           const g = get().graph;
528           const del = new Set(ids);
529           commitGraph(
530             {
531               ...g,
532               nodes: g.nodes.filter((n) => !del.has(n.id)),
533               edges: g.edges.filter((e) => !del.has(e.source) && !del.has(e.target)),
534             },
535             { selectedNodeIds: [], selectedEdgeIds: [] },
536           );
537         },
538         addEdge(source, target, weight) {
539           const g = get().graph;
540           if (source === target) return { ok: false, reason: 'self-loop' };
541           if (!g.nodes.some((n) => n.id === source) || !g.nodes.some((n) => n.id ===
    target))
542             return { ok: false, reason: 'missing-node' };
543           if (!validWeight(weight)) return { ok: false, reason: 'invalid-weight' };
```

```
544             const dup = g.edges.find((e) => samePair(e, source, target));
545             if (dup) return { ok: false, reason: 'duplicate', existingEdgeId: dup.id };
546             const ids = new Set(g.edges.map((e) => e.id));
547             let id = `${source}-${target}`;
548             for (let i = 2; ids.has(id); i++) id = `${source}-${target}-${i}`;
549             commitGraph({
550               ...g,
551               edges: [...g.edges, { id, source, target, weight, shape: 'straight', shapeAuto:
    true }],
552             });
553             return { ok: true, edgeId: id };
554           },
555           setEdgeWeight(id, weight) {
556             const g = get().graph;
557             if (!validWeight(weight) || !g.edges.some((e) => e.id === id)) return false;
558             commitGraph({ ...g, edges: g.edges.map((e) => (e.id === id ? { ...e, weight } :
    e)) });
559             return true;
560           },
561           setEdgeShape(ids, shape) {
562             const g = get().graph;
```

```
563             const hit = (e: GEdge) => ids === 'all' || ids.includes(e.id);
564             commitGraph({ ...g, edges: g.edges.map((e) => (hit(e) ? { ...e, shape, shapeAuto: false } : e)) });
565           },
566           resetEdgeShapeAuto(ids) {
567             const g = get().graph;
568             const hit = (e: GEdge) => ids === 'all' || ids.includes(e.id);
569             commitGraph({
570               ...g,
571               edges: g.edges.map((e) => (hit(e) ? { ...e, shape: 'straight', shapeAuto: true } : e)),
572             });
573           },
574           removeEdges(ids) {
575             const g = get().graph;
576             const del = new Set(ids);
577             commitGraph({ ...g, edges: g.edges.filter((e) => !del.has(e.id)) }, { selectedEdgeIds: [] });
578           },
579           async applySuggestedLayout() {
580             const g = get().graph;
581             if (g.nodes.length === 0) return;
```

```
582            try {
583              const s = await getWorker().suggestLayout(g);
584              if (get().graph !== g) return; // el grafo cambió mientras se calculaba
585              commitGraph(
586                {
587                  ...g,
588                  nodes: g.nodes.map((n) => {
589                    const p = s.positions[n.id];
590                    return p ? { ...n, x: p.x - n.w / 2, y: p.y - n.h / 2 } : n;
591                  }),
592                },
593                { layoutSuggestion: s },
594              );
595            } catch (e) {
596              console.warn('[RutaQ] distribución sugerida:', e);
597            }
598          },
599          undo() {
600            history().undo();
```

```
601             set(derive(get().graph, get().query));
602           },
603           redo() {
604             history().redo();
605             set(derive(get().graph, get().query));
606           },
607           canUndo: () => history().pastStates.length > 0,
608           canRedo: () => history().futureStates.length > 0,
609
610           // ── Selección y consulta ──
611           select(nodeIds, edgeIds) {
612             set({ selectedNodeIds: nodeIds, selectedEdgeIds: edgeIds });
613           },
614           setTreeQuery(source) {
615             setQuery({ kind: 'tree', source });
616           },
617           setPathQuery(source, target) {
618             setQuery({ kind: 'path', source, target });
619           },
620           clearQuery() {
```

```
621             setQuery(null);
622           },
623
624           // ── Reproducción S1 ──
625           stepTo(index) {
626             const steps = get().dijkstra?.steps.length ?? 0;
627             if (steps === 0) return;
628             set({ playback: { ...get().playback, stepIndex: Math.max(0, Math.min(steps - 1,
    index)) } });
629           },
630           nextStep() {
631             get().stepTo(get().playback.stepIndex + 1);
632           },
633           prevStep() {
634             get().stepTo(get().playback.stepIndex - 1);
635           },
636           play() {
637             const last = (get().dijkstra?.steps.length ?? 0) - 1;
638             if (last < 0) return;
639             stopTimer();
```

```
      const start = get().playback.stepIndex >= last ? 0 : get().playback.stepIndex;
      set({ playback: { ...get().playback, stepIndex: start, playing: true } });
      playTimer = setInterval(() => {
        const { stepIndex } = get().playback;
        if (stepIndex >= last) get().pause();
        else get().stepTo(stepIndex + 1);
      }, get().playback.speedMs);
    },
    pause() {
      stopTimer();
      set({ playback: { ...get().playback, playing: false } });
    },
    toEnd() {
      get().pause();
      get().stepTo(Number.MAX_SAFE_INTEGER);
    },
    setSpeed(ms) {
      speedMs = Math.max(50, ms);
      const wasPlaying = get().playback.playing;
      set({ playback: { ...get().playback, speedMs } });
```

```
660           if (wasPlaying) get().play();
661         },
662
663         // — S3 —
664         runAllPaths() {
665           const { graph, query, allPathsLimit, validation } = get();
666           if (query?.kind !== 'path' || !query.target || !validation.ok) return;
667           killWorkerIfRunning();
668           const run = ++allPathsRun;
669           set({ allPaths: { status: 'running', found: 0, result: null, error: null },
highlightedPath: null });
670           const onProgress = Comlink.proxy((found: number) => {
671             if (run === allPathsRun) set({ allPaths: { ...get().allPaths, found } });
672           });
673           getWorker()
674             .allSimplePaths(graph, query.source, query.target, { limit: allPathsLimit,
timeoutMs: 8000 }, onProgress)
675             .then((result) => {
676               if (run === allPathsRun)
677                 set({ allPaths: { status: 'done', found: result.totalFound, result, error:
null } });
```

```
678             })
679             .catch((e: unknown) => {
680               if (run === allPathsRun)
681                 set({
682                   allPaths: { ...get().allPaths, status: 'error', error: e instanceof Error ? e.message : String(e) },
683                 });
684             });
685         },
686         cancelAllPaths() {
687           if (get().allPaths.status !== 'running') return;
688           allPathsRun++;
689           killWorker();
690           set({ allPaths: { ...get().allPaths, status: 'cancelled' } });
691         },
692         setAllPathsLimit(limit) {
693           if (Number.isInteger(limit) && limit > 0) set({ allPathsLimit: limit });
694         },
695         highlightPath(sel) {
696           set({ highlightedPath: sel });
```

```
697         },
698
699         // ── Proyecto II: Transporte Express ──
700         addDepartment({ label, cabecera, lat, lon, at }) {
701           const g = get().graph;
702           const name = label.trim();
703           if (!name) return { ok: false, reason: 'empty-name' };
704           if (g.nodes.some((n) => plain(n.label) === plain(name))) return { ok: false,
    reason: 'duplicate-name' };
705           if (!validCoords(lat, lon)) return { ok: false, reason: 'invalid-coords' };
706           const id = departmentId(g, name);
707           const c = placeDepartment(g, lat, lon, at);
708           // Sin coordenadas se guardan las del punto donde quedó, para que siga siendo un
    nodo del mapa.
709           const geo =
710             lat !== undefined && lon !== undefined
711               ? { lat, lon }
712               : gtProjection
713                 ? gtProjection.unproject(c.x, c.y)
714                 : {};
715           const node: GNode = {
```

```
716             id,
717             label: name,
718             x: Math.round(c.x - DATASET_NODE_SIZE.w / 2),
719             y: Math.round(c.y - DATASET_NODE_SIZE.h / 2),
720             ...DATASET_NODE_SIZE,
721             ...geo,
722           };
723           commitGraph(
724             { ...g, nodes: [...g.nodes, node] },
725             { departments: { ...get().departments, [id]: { cabecera: cabecera.trim() ||
    name } } },
726           );
727           return { ok: true, id };
728         },
729         updateDepartment(id, { label, cabecera, lat, lon }) {
730           const g = get().graph;
731           const node = g.nodes.find((n) => n.id === id);
732           if (!node) return { ok: false, reason: 'empty-name' };
733           const name = label === undefined ? node.label : label.trim();
734           if (!name) return { ok: false, reason: 'empty-name' };
```

```
735            if (g.nodes.some((n) => n.id !== id && plain(n.label) === plain(name)))
736              return { ok: false, reason: 'duplicate-name' };
737            const moving = lat !== undefined || lon !== undefined;
738            const nlat = lat ?? node.lat;
739            const nlon = lon ?? node.lon;
740            if (moving && !validCoords(nlat, nlon)) return { ok: false, reason: 'invalid-
coords' };
741            const c = moving && gtProjection ? gtProjection.project(nlat!, nlon!) : null;
742            commitGraph(
743              {
744                ...g,
745                nodes: g.nodes.map((n) =>
746                  n.id === id
747                    ? {
748                        ...n,
749                        label: name,
750                        ...(c
751                          ? { x: Math.round(c.x - n.w / 2), y: Math.round(c.y - n.h / 2),
lat: nlat, lon: nlon }
752                          : {}),
753                      }
```

```
                    : n,
                ),
              },
              {
                departments: {
                  ...get().departments,
                  [id]: { cabecera: cabecera?.trim() || get().departments[id]?.cabecera ||
name },
                },
              },
            );
            return { ok: true, id };
          },
          addRoad(source, target, km, road) {
            const g = get().graph;
            if (source === target) return { ok: false, reason: 'self-loop' };
            if (!g.nodes.some((n) => n.id === source) || !g.nodes.some((n) => n.id ===
target))
              return { ok: false, reason: 'missing-node' };
            if (!validWeight(km)) return { ok: false, reason: 'invalid-weight' };
            const dup = g.edges.find((e) => samePair(e, source, target));
```

```
773             if (dup) return { ok: false, reason: 'duplicate', existingEdgeId: dup.id };
774             // Convención del dataset: ids en orden alfabético («GUA-SAC»); bidireccional por
    definición.
775             const [a, b] = source < target ? [source, target] : [target, source];
776             const ids = new Set(g.edges.map((e) => e.id));
777             let id = `${a}-${b}`;
778             for (let i = 2; ids.has(id); i++) id = `${a}-${b}-${i}`;
779             const meta = { carretera: road?.trim() ?? '', fuente: 'Ingresada por el usuario'
    };
780             commitGraph({
781               ...g,
782               edges: [...g.edges, { id, source: a, target: b, weight: km, shape: 'straight',
    shapeAuto: true, meta }],
783             });
784             return { ok: true, edgeId: id };
785           },
786           updateRoad(id, { km, road }) {
787             const g = get().graph;
788             if (!g.edges.some((e) => e.id === id)) return false;
789             if (km !== undefined && !validWeight(km)) return false;
790             commitGraph({
791               ...g,
```

```
792               edges: g.edges.map((e) =>
793                 e.id === id
794                   ? {
795                       ...e,
796                       weight: km ?? e.weight,
797                       meta: road === undefined ? e.meta : { ...e.meta, carretera: road.trim() },
798                     }
799                   : e,
800               ),
801             });
802             return true;
803           },
804           restoreOriginal() {
805             try {
806               storage?.removeItem(KEY_GUATEMALA);
807             } catch {
808               // Sin acceso a localStorage: igual se recarga el original.
809             }
810             get().loadTemplate('guatemala');
```

```
811         },
812         exportGuatemalaDataset() {
813           const { graph, graphName, departments } = get();
814           const str = (v: unknown, d = '') => (typeof v === 'string' ? v : d);
815           return {
816             meta: {
817               nombre: graphName,
818               unidad: 'km',
819               fuentes: guatemalaOriginal?.meta.fuentes ?? ['Ingresado por el usuario'],
820               fecha: new Date().toISOString().slice(0, 10),
821             },
822             nodes: graph.nodes.map((n) => {
823               const geo =
824                 n.lat !== undefined && n.lon !== undefined
825                   ? { lat: n.lat, lon: n.lon }
826                   : gtProjection
827                     ? gtProjection.unproject(n.x + n.w / 2, n.y + n.h / 2)
828                     : { lat: 0, lon: 0 };
829               const orig = guatemalaOriginal?.nodes.find((o) => o.id === n.id);
830               return {
```

```
                id: n.id,
                label: n.label,
                cabecera: departments[n.id]?.cabecera ?? n.label,
                ...geo,
                fuente: orig && orig.lat === geo.lat && orig.lon === geo.lon ? orig.fuente : 'Ingresado por el usuario',
              };
            }),
            edges: graph.edges.map((e) => ({
              id: e.id,
              source: e.source,
              target: e.target,
              km: e.weight,
              carretera: str(e.meta?.carretera),
              fuente: str(e.meta?.fuente, 'Ingresada por el usuario'),
              verificado_google: e.meta?.verificado_google === true,
              km_google: typeof e.meta?.km_google === 'number' ? e.meta.km_google : null,
              nota: str(e.meta?.nota),
              geometry: str(e.meta?.geometry),
            })),
```

```
850             };
851           },
852           setCoverMode(mode) {
853             set({ coverMode: mode });
854           },
855           setMstStart(id) {
856             mstStart = id;
857             set(derive(get().graph, get().query));
858           },
859           runTour(start) {
860             const { graph } = get();
861             const run = ++tourRun;
862             set({ tour: { status: 'running', start, result: null, error: null } });
863             getWorker()
864               .deliveryTour(graph, start)
865               .then((result) => {
866                 if (run === tourRun) set({ tour: { status: 'done', start, result, error: null
    } });
867               })
868               .catch((e: unknown) => {
```

```
869               if (run === tourRun)
870                 set({
871                   tour: { status: 'error', start, result: null, error: e instanceof Error ? e.message : String(e) },
872                 });
873             });
874         },
875         cancelTour() {
876           if (get().tour.status !== 'running') return;
877           tourRun++;
878           killWorker();
879           set({ tour: { ...get().tour, status: 'cancelled' } });
880         },
881         projectLatLon(lat, lon) {
882           const mode = get().mode;
883           const proj = mode === 'guatemala' ? gtProjection : mode === 'quetzaltenango' ? qtzProjection : null;
884           return proj ? proj.project(lat, lon) : null;
885         },
886
887         // ── Preferencias ──
```

```
888           setActiveTab(tab) {
889             set({ activeTab: tab });
890           },
891           setTabularConfig(patch) {
892             set({ tabularConfig: { ...get().tabularConfig, ...patch } });
893           },
894           setTheme(theme) {
895             set({ theme });
896             applyTheme(theme);
897           },
898           setKeepProportion(on) {
899             set({ keepProportion: on });
900           },
901         };
902       },
903       {
904         // Solo el grafo entra al historial de deshacer/rehacer.
905         partialize: (s) => ({ graph: s.graph, departments: s.departments }),
906         equality: (a, b) => a.graph === b.graph && a.departments === b.departments,
907         limit: 100,
```

```
      },
    ),
  );

  // Autoguardado del modo diseño y de las preferencias.
  useAppStore.subscribe((s, prev) => {
    if (s.mode === 'design' && (s.graph !== prev.graph || s.graphName !== prev.graphName)) {
      writeJson(KEY_DESIGN, s.exportGraphFile());
    }
    // Proyecto II: el módulo Guatemala se autoguarda solo cuando el usuario lo modifica (no al abrirlo).
    if (
      s.mode === 'guatemala' &&
      prev.mode === 'guatemala' &&
      (s.graph !== prev.graph || s.departments !== prev.departments)
    ) {
      writeJson(KEY_GUATEMALA, s.exportGuatemalaDataset());
    }
    if (
      s.theme !== prev.theme ||
```

```
927       s.tabularConfig !== prev.tabularConfig ||
928       s.allPathsLimit !== prev.allPathsLimit ||
929       s.keepProportion !== prev.keepProportion ||
930       s.mode !== prev.mode
931     ) {
932       const p: Prefs = {
933         theme: s.theme,
934         tabularConfig: s.tabularConfig,
935         allPathsLimit: s.allPathsLimit,
936         keepProportion: s.keepProportion,
937         mode: s.mode,
938       };
939       writeJson(KEY_PREFS, p);
940     }
941   });
942
943   applyTheme(useAppStore.getState().theme);
944   if (typeof matchMedia !== 'undefined') {
945     matchMedia('(prefers-color-scheme: dark)').addEventListener('change', () =>
    applyTheme(useAppStore.getState().theme));
```

```
}
```

Interfaz del módulo Transporte Express

Componentes React del módulo Guatemala.

src/ui/components/guatemala/GuatemalaAboutModal.tsx

107 líneas.

```
/**
 * Diálogo «Acerca de» del Proyecto II (G5).
 * Muestra los datos institucionales de la Universidad Mariano Gálvez de Guatemala,
 * los nombres y carnés exactos de los 3 integrantes del grupo y la fecha de presentación.
 *
 * Datos estáticos según docs/PLAN_PROYECTO2.md.
 */
import Modal from '../Modal';


interface GuatemalaAboutModalProps {
  isOpen: boolean;
  onClose: () => void;
}


```

```
15  export default function GuatemalaAboutModal({ isOpen, onClose }: GuatemalaAboutModalProps)
    {
16    return (
17      <Modal isOpen={isOpen} onClose={onClose} title="Acerca de — Transporte Express (RutaQ)"
    testId="modal-about">
18        <div className="flex flex-col gap-4 text-xs leading-relaxed" data-testid="about-
    dialog-content">
19          {/* Encabezado Institucional */}
20          <div className="rounded-xl border border-blue-200 bg-blue-50/70 p-3.5 text-center
    dark:border-blue-900/50 dark:bg-blue-950/20">
21            <div className="text-base font-extrabold text-blue-900 dark:text-blue-100">
22              Universidad Mariano Gálvez de Guatemala
23            </div>
24            <div className="text-xs font-semibold text-blue-800 dark:text-blue-200 mt-0.5">
25              Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación
26            </div>
27            <div className="text-[11px] text-slate-600 dark:text-slate-400 mt-2">
28              Proyecto II: «Transporte Express» — Optimización de Rutas Departamentales
29            </div>
30          </div>
31
32          {/* Datos académicos */}
```

```
        <div className="grid grid-cols-2 gap-2 rounded-lg border border-slate-200 bg-slate-50 p-3 dark:border-slate-800 dark:bg-slate-900/60">
          <div>
            <span className="text-[11px] font-bold text-slate-500 dark:text-slate-400">Curso:</span>
            <div className="font-semibold text-slate-800 dark:text-slate-200">Investigación de Operaciones</div>
          </div>
          <div>
            <span className="text-[11px] font-bold text-slate-500 dark:text-slate-400">Catedrático:</span>
            <div className="font-semibold text-slate-800 dark:text-slate-200">Ing. Samuel Isaac Gómez Moreno</div>
          </div>
          <div>
            <span className="text-[11px] font-bold text-slate-500 dark:text-slate-400">Semestre y Carrera:</span>
            <div className="font-semibold text-slate-800 dark:text-slate-200">6.º semestre, Ingeniería en Sistemas</div>
          </div>
          <div>
            <span className="text-[11px] font-bold text-slate-500 dark:text-slate-400">Fecha de presentación:</span>
            <div className="font-semibold text-slate-800 dark:text-slate-200">Sábado 31 de octubre de 2026</div>
          </div>
```

```
        </div>


        {/* Integrantes del grupo y carnés */}
        <div className="rounded-xl border border-slate-200 bg-white p-3 dark:border-slate-800 dark:bg-slate-900">
          <div className="font-bold text-slate-800 dark:text-slate-200 mb-2">Integrantes del equipo de desarrollo:</div>
          <div className="overflow-x-auto">
            <table className="w-full text-left border-collapse" data-testid="table-team-members">
              <thead>
                <tr className="border-b border-slate-200 text-[11px] font-bold text-slate-500 uppercase dark:border-slate-700">
                  <th className="py-1.5 px-2">Nombre completo</th>
                  <th className="py-1.5 px-2 font-mono text-right">Carné</th>
                </tr>
              </thead>
              <tbody className="divide-y divide-slate-100 font-medium dark:divide-slate-800">
                <tr className="hover:bg-slate-50 dark:hover:bg-slate-800/50">
                  <td className="py-2 px-2 text-slate-800 dark:text-slate-200">Cristopher Alexis Castellanos Paz</td>
                  <td className="py-2 px-2 text-right font-mono font-bold text-blue-600 dark:text-blue-400">
```

```
                    2690-24-5972
                  </td>
                </tr>
                <tr className="hover:bg-slate-50 dark:hover:bg-slate-800/50">
                  <td className="py-2 px-2 text-slate-800 dark:text-slate-200">Leonel
Taracena Marroquín</td>
                  <td className="py-2 px-2 text-right font-mono font-bold text-blue-600
dark:text-blue-400">
                    2690-24-4072
                  </td>
                </tr>
                <tr className="hover:bg-slate-50 dark:hover:bg-slate-800/50">
                  <td className="py-2 px-2 text-slate-800 dark:text-slate-200">Pablo
Alexander Vásquez Rivera</td>
                  <td className="py-2 px-2 text-right font-mono font-bold text-blue-600
dark:text-blue-400">
                    2690-24-13123
                  </td>
                </tr>
              </tbody>
            </table>
          </div>
```

```
        </div>



        {/* Descripción del sistema */}

        <p className="text-[11px] text-slate-500 dark:text-slate-400 leading-relaxed">

          <strong>RutaQ — Transporte Express</strong> resuelve el problema de optimización de entrega de paquetes a

          través de la red vial de los 22 departamentos de Guatemala, integrando el algoritmo de Dijkstra para rutas más

          cortas, el árbol de expansión mínima con Prim y Kruskal, y recorridos heurísticos de reparto.

        </p>



        <div className="flex justify-end pt-1">

          <button

            type="button"

            data-testid="btn-close-about"

            onClick={onClose}

            className="min-h-[44px] rounded-lg bg-blue-600 px-5 py-2 font-bold text-white shadow-xs hover:bg-blue-700 focus-visible:ring-2 focus-visible:ring-blue-500"

          >

            Aceptar

          </button>
```

```
          </div>
        </div>
      </Modal>
    );
  }
```

src/ui/components/guatemala/GuatemalaAdminModal.tsx

906 líneas.

```
/**
 * Panel de administración de la red vial para Transporte Express (G2).
 * Incluye dos secciones:
 *   1. Departamentos: agregar, editar, eliminar, buscar; coordenadas opcionales.
 *   2. Carreteras: agregar, editar, eliminar, buscar; tabla ordenable (por origen, destino, carretera, km).
 * Operaciones globales:
 *   - Exportar JSON (dataset Guatemala)
 *   - Importar JSON
 *   - Restaurar datos originales con diálogo de confirmación.
 *
 * Lee y modifica: `graph`, `departments`, `addDepartment`, `updateDepartment`, `removeNodes`,
 * `addRoad`, `updateRoad`, `removeEdges`, `restoreOriginal`, `exportGuatemalaDataset`, `importGraphFile` desde `useApp`.
```

```
 */
import { useCallback, useMemo, useRef, useState } from 'react';
import type { NodeId } from '../../../core/types';
import { useApp } from '../../useApp';
import Modal from '../Modal';

interface GuatemalaAdminModalProps {
  isOpen: boolean;
  onClose: () => void;
}

type RoadSortField = 'source' | 'target' | 'road' | 'km';
type SortOrder = 'asc' | 'desc';

const getCarretera = (meta?: Record<string, unknown>): string =>
  typeof meta?.carretera === 'string' ? meta.carretera : '';

export default function GuatemalaAdminModal({ isOpen, onClose }: GuatemalaAdminModalProps)
{
  const graph = useApp((s) => s.graph);
```

```
32    const departments = useApp((s) => s.departments);
33    const addDepartment = useApp((s) => s.addDepartment);
34    const updateDepartment = useApp((s) => s.updateDepartment);
35    const removeNodes = useApp((s) => s.removeNodes);
36    const addRoad = useApp((s) => s.addRoad);
37    const updateRoad = useApp((s) => s.updateRoad);
38    const removeEdges = useApp((s) => s.removeEdges);
39    const restoreOriginal = useApp((s) => s.restoreOriginal);
40    const exportGuatemalaDataset = useApp((s) => s.exportGuatemalaDataset);
41    const importGraphFile = useApp((s) => s.importGraphFile);
42
43    const fileInputRef = useRef<HTMLInputElement>(null);
44
45    // Sección activa: 'departments' | 'roads'
46    const [activeSection, setActiveSection] = useState<'departments' | 'roads'>
  ('departments');
47
48    // Diálogo de confirmación para restaurar originales
49    const [showRestoreConfirm, setShowRestoreConfirm] = useState(false);
50
```

```
51    // Mensajes de error globales o de importación
52    const [globalMessage, setGlobalMessage] = useState<{ text: string; type: 'success' |
    'error' } | null>(null);
53
54    // ── ESTADO: DEPARTAMENTOS ──
55    const [deptSearch, setDeptSearch] = useState('');
56    const [newDeptLabel, setNewDeptLabel] = useState('');
57    const [newDeptCabecera, setNewDeptCabecera] = useState('');
58    const [newDeptLat, setNewDeptLat] = useState('');
59    const [newDeptLon, setNewDeptLon] = useState('');
60    const [deptFormError, setDeptFormError] = useState<string | null>(null);
61
62    const [editingDeptId, setEditingDeptId] = useState<NodeId | null>(null);
63    const [editDeptLabel, setEditDeptLabel] = useState('');
64    const [editDeptCabecera, setEditDeptCabecera] = useState('');
65    const [editDeptLat, setEditDeptLat] = useState('');
66    const [editDeptLon, setEditDeptLon] = useState('');
67
68    // ── ESTADO: CARRETERAS ──
69    const [roadSearch, setRoadSearch] = useState('');
```

```
70    const [newRoadSource, setNewRoadSource] = useState<NodeId>(graph.nodes[0]?.id ?? '');
71    const [newRoadTarget, setNewRoadTarget] = useState<NodeId>(graph.nodes[1]?.id ?? '');
72    const [newRoadKm, setNewRoadKm] = useState('');
73    const [newRoadName, setNewRoadName] = useState('');
74    const [roadFormError, setRoadFormError] = useState<{ message: string; existingEdgeId?:
    string } | null>(null);
75
76    const [editingRoadId, setEditingRoadId] = useState<string | null>(null);
77    const [editRoadKm, setEditRoadKm] = useState('');
78    const [editRoadName, setEditRoadName] = useState('');
79
80    // Ordenamiento de tabla de carreteras
81    const [sortField, setSortField] = useState<RoadSortField>('source');
82    const [sortOrder, setSortOrder] = useState<SortOrder>('asc');
83
84    const getNodeLabel = useCallback(
85      (id: NodeId): string => {
86        return graph.nodes.find((n) => n.id === id)?.label ?? id;
87      },
88      [graph.nodes],
```

```
  );

  // ── ACCIONES: DEPARTAMENTOS ──
  const handleAddDept = (e: React.FormEvent) => {
    e.preventDefault();
    setDeptFormError(null);
    const lat = newDeptLat.trim() ? parseFloat(newDeptLat) : undefined;
    const lon = newDeptLon.trim() ? parseFloat(newDeptLon) : undefined;

    const res = addDepartment({
      label: newDeptLabel,
      cabecera: newDeptCabecera,
      lat,
      lon,
    });

    if (res.ok) {
      setNewDeptLabel('');
      setNewDeptCabecera('');
      setNewDeptLat('');
```

```
109         setNewDeptLon('');
110         setGlobalMessage({ text: `Departamento «${newDeptLabel}» agregado con éxito.`, type:
    'success' });
111       } else {
112         if (res.reason === 'empty-name') {
113           setDeptFormError('El nombre del departamento no puede estar vacío.');
114         } else if (res.reason === 'duplicate-name') {
115           setDeptFormError('Ya existe un departamento con ese nombre.');
116         } else if (res.reason === 'invalid-coords') {
117           setDeptFormError('Coordenadas inválidas: latitud entre -90 y 90, longitud entre
    -180 y 180.');
118         }
119       }
120     };
121
122     const handleStartEditDept = (id: NodeId) => {
123       const node = graph.nodes.find((n) => n.id === id);
124       if (!node) return;
125       setEditingDeptId(id);
126       setEditDeptLabel(node.label);
127       setEditDeptCabecera(departments[id]?.cabecera ?? node.label);
```

```
      setEditDeptLat(node.lat !== undefined ? String(node.lat) : '');
      setEditDeptLon(node.lon !== undefined ? String(node.lon) : '');
    };

    const handleSaveEditDept = (id: NodeId) => {
      const lat = editDeptLat.trim() ? parseFloat(editDeptLat) : undefined;
      const lon = editDeptLon.trim() ? parseFloat(editDeptLon) : undefined;

      const res = updateDepartment(id, {
        label: editDeptLabel,
        cabecera: editDeptCabecera,
        lat,
        lon,
      });

      if (res.ok) {
        setEditingDeptId(null);
        setGlobalMessage({ text: 'Departamento actualizado correctamente.', type: 'success'
});
      } else {
```

```
147         alert(res.reason === 'duplicate-name' ? 'Ya existe un departamento con ese nombre.' :
    'Error al actualizar.');
148       }
149     };
150
151     const handleDeleteDept = (id: NodeId) => {
152       const name = getNodeLabel(id);
153       if (confirm(`¿Eliminar el departamento «${name}» y todas sus carreteras conectadas?`))
    {
154         removeNodes([id]);
155         setGlobalMessage({ text: `Departamento «${name}» eliminado.`, type: 'success' });
156       }
157     };
158
159     // ── ACCIONES: CARRETERAS ──
160     const handleAddRoad = (e: React.FormEvent) => {
161       e.preventDefault();
162       setRoadFormError(null);
163       const km = parseFloat(newRoadKm);
164
165       if (newRoadSource === newRoadTarget) {
```

```
166       setRoadFormError({ message: 'El origen y el destino no pueden ser el mismo
departamento.' });
167       return;
168     }
169     if (isNaN(km) || km <= 0) {
170       setRoadFormError({ message: 'El kilometraje debe ser un número positivo mayor a 0.'
});
171       return;
172     }
173
174     const res = addRoad(newRoadSource, newRoadTarget, km, newRoadName || undefined);
175     if (res.ok) {
176       setNewRoadKm('');
177       setNewRoadName('');
178       setGlobalMessage({ text: 'Carretera agregada exitosamente.', type: 'success' });
179     } else {
180       if (res.reason === 'duplicate') {
181         setRoadFormError({
182           message: 'Ya existe una carretera directa entre estos dos departamentos.',
183           existingEdgeId: res.existingEdgeId,
184         });
```

```
185         } else if (res.reason === 'self-loop') {
186           setRoadFormError({ message: 'No se permiten lazos hacia el mismo departamento.' });
187         } else if (res.reason === 'invalid-weight') {
188           setRoadFormError({ message: 'El kilometraje ingresado no es válido.' });
189         } else {
190           setRoadFormError({ message: 'Error al agregar la carretera.' });
191         }
192       }
193     };
194
195     const handleStartEditRoad = (id: string) => {
196       const edge = graph.edges.find((e) => e.id === id);
197       if (!edge) return;
198       setEditingRoadId(id);
199       setEditRoadKm(String(edge.weight));
200       setEditRoadName(getCarretera(edge.meta));
201     };
202
203     const handleSaveEditRoad = (id: string) => {
204       const km = parseFloat(editRoadKm);
```

```
205       if (isNaN(km) || km <= 0) {
206         alert('El kilometraje debe ser mayor a 0.');
207         return;
208       }
209       const ok = updateRoad(id, { km, road: editRoadName });
210       if (ok) {
211         setEditingRoadId(null);
212         setGlobalMessage({ text: 'Carretera actualizada correctamente.', type: 'success' });
213       }
214     };
215
216     const handleDeleteRoad = (id: string) => {
217       if (confirm('¿Eliminar esta carretera?')) {
218         removeEdges([id]);
219         setGlobalMessage({ text: 'Carretera eliminada.', type: 'success' });
220       }
221     };
222
223     // ── EXPORTAR / IMPORTAR / RESTAURAR ──
```

```
224   const handleExportJson = () => {
225     const data = exportGuatemalaDataset();
226     const blob = new Blob([JSON.stringify(data, null, 2)], { type: 'application/json' });
227     const url = URL.createObjectURL(blob);
228     const a = document.createElement('a');
229     a.href = url;
230     a.download = `guatemala_transporte_express_${new Date().toISOString().slice(0,
    10)}.json`;
231     a.click();
232     URL.revokeObjectURL(url);
233     setGlobalMessage({ text: 'Archivo JSON de Guatemala exportado exitosamente.', type:
    'success' });
234   };
235
236   const handleImportFile = (e: React.ChangeEvent<HTMLInputElement>) => {
237     const file = e.target.files?.[0];
238     if (!file) return;
239     const reader = new FileReader();
240     reader.onload = (evt) => {
241       const content = evt.target?.result as string;
242       if (content) {
```

```
          const res = importGraphFile(content);
          if (res.ok) {
            setGlobalMessage({ text: 'Dataset importado correctamente.', type: 'success' });
          } else {
            setGlobalMessage({ text: `Error al importar: ${res.message}`, type: 'error' });
          }
        }
      };
      reader.readAsText(file);
      if (fileInputRef.current) fileInputRef.current.value = '';
    };

    const handleConfirmRestore = () => {
      restoreOriginal();
      setShowRestoreConfirm(false);
      setGlobalMessage({ text: 'Se restauraron los datos originales de Guatemala.', type:
'success' });
    };

    // ── FILTRADO Y ORDENAMIENTO ──
```

```
  const filteredDepartments = useMemo(() => {
    const term = deptSearch.trim().toLowerCase();
    return graph.nodes.filter((n) => {
      const label = n.label.toLowerCase();
      const cab = (departments[n.id]?.cabecera ?? '').toLowerCase();
      return !term || label.includes(term) || cab.includes(term) ||
n.id.toLowerCase().includes(term);
    });
  }, [graph.nodes, departments, deptSearch]);

  const sortedRoads = useMemo(() => {
    const term = roadSearch.trim().toLowerCase();
    const list = graph.edges.filter((e) => {
      const s = getNodeLabel(e.source).toLowerCase();
      const t = getNodeLabel(e.target).toLowerCase();
      const r = getCarretera(e.meta).toLowerCase();
      return !term || s.includes(term) || t.includes(term) || r.includes(term);
    });

    return list.sort((a, b) => {
```

```
281         let cmp = 0;
282         if (sortField === 'source') {
283           cmp = getNodeLabel(a.source).localeCompare(getNodeLabel(b.source));
284         } else if (sortField === 'target') {
285           cmp = getNodeLabel(a.target).localeCompare(getNodeLabel(b.target));
286         } else if (sortField === 'road') {
287           cmp = getCarretera(a.meta).localeCompare(getCarretera(b.meta));
288         } else if (sortField === 'km') {
289           cmp = a.weight - b.weight;
290         }
291         return sortOrder === 'asc' ? cmp : -cmp;
292       });
293     }, [graph.edges, roadSearch, sortField, sortOrder, getNodeLabel]);
294
295     const toggleSort = (field: RoadSortField) => {
296       if (sortField === field) {
297         setSortOrder((prev) => (prev === 'asc' ? 'desc' : 'asc'));
298       } else {
299         setSortField(field);
300         setSortOrder('asc');
```

```
    }
  };

  return (
    <Modal
      isOpen={isOpen}
      onClose={onClose}
      title="Administrar Red Vial — Transporte Express"
      testId="modal-admin-network"
    >
      <div className="flex flex-col gap-4 text-xs max-h-[78vh] overflow-y-auto pr-1" data-testid="panel-admin-network">
        {/* Input oculto para importación de JSON */}
        <input
          type="file"
          ref={fileInputRef}
          onChange={handleImportFile}
          accept=".json,application/json"
          className="hidden"
          data-testid="input-admin-import-file"
```

```
320          />
321
322          {/* Barra superior de operaciones de archivo */}
323          <div className="flex flex-wrap items-center justify-between gap-2 border-b border-slate-200 pb-3 dark:border-slate-800">
324            <div className="flex items-center gap-1.5">
325              <button
326                type="button"
327                data-testid="btn-admin-export-json"
328                onClick={handleExportJson}
329                className="flex min-h-[40px] items-center gap-1.5 rounded-lg border border-slate-300 bg-white px-3 py-1.5 font-bold text-slate-700 shadow-2xs hover:bg-slate-50 dark:border-slate-700 dark:bg-slate-800 dark:text-slate-200"
330              >
331                <span>💾</span> Exportar JSON
332              </button>
333              <button
334                type="button"
335                data-testid="btn-admin-import-json"
336                onClick={() => fileInputRef.current?.click()}
337                className="flex min-h-[40px] items-center gap-1.5 rounded-lg border border-slate-300 bg-white px-3 py-1.5 font-bold text-slate-700 shadow-2xs hover:bg-slate-50 dark:border-slate-700 dark:bg-slate-800 dark:text-slate-200"
```

```
              >
                <span>📁</span> Importar JSON
              </button>
            </div>

            <button
              type="button"
              data-testid="btn-admin-restore-original"
              onClick={() => setShowRestoreConfirm(true)}
              className="min-h-[40px] rounded-lg border border-amber-300 bg-amber-50 px-3 py-1.5 font-bold text-amber-800 hover:bg-amber-100 dark:border-amber-800 dark:bg-amber-950/40 dark:text-amber-300"
            >
              🔄 Restaurar datos originales
            </button>
          </div>

          {/* Diálogo de confirmación para restaurar */}
          {showRestoreConfirm && (
            <div className="rounded-xl border border-amber-300 bg-amber-50 p-3.5 text-amber-950 dark:border-amber-900/60 dark:bg-amber-950/40 dark:text-amber-200 shadow-md">
```

```
            <p className="font-bold text-sm">¿Confirmar restauración de datos originales?
</p>
            <p className="mt-1 leading-relaxed">
              Esta acción restablecerá los 22 departamentos y carreteras oficiales de
Transporte Express. Se descartarán
              las modificaciones locales no guardadas.
            </p>
            <div className="mt-3 flex justify-end gap-2">
              <button
                type="button"
                data-testid="btn-cancel-restore"
                onClick={() => setShowRestoreConfirm(false)}
                className="min-h-[38px] rounded-lg border border-slate-300 bg-white px-3
py-1 font-semibold text-slate-700 hover:bg-slate-100 dark:border-slate-700 dark:bg-slate-
800 dark:text-slate-200"
              >
                Cancelar
              </button>
              <button
                type="button"
                data-testid="btn-confirm-restore"
                onClick={handleConfirmRestore}
```

```
                  className="min-h-[38px] rounded-lg bg-amber-600 px-3.5 py-1 font-bold text-
white shadow-xs hover:bg-amber-700"
                >
                  Sí, restaurar datos
                </button>
              </div>
            </div>
          )}

          {/* Mensaje global temporal */}
          {globalMessage && (
            <div
              className={`rounded-lg p-2.5 font-semibold text-xs ${
                globalMessage.type === 'success'
                  ? 'bg-emerald-50 text-emerald-800 dark:bg-emerald-950/50 dark:text-emerald-
300'
                  : 'bg-rose-50 text-rose-800 dark:bg-rose-950/50 dark:text-rose-300'
              }`}
            >
              {globalMessage.text}
            </div>
```

```
393          )}
394
395          {/* Selector de pestañas de administración */}
396          <div className="flex rounded-lg bg-slate-100 p-1 dark:bg-slate-800" role="tablist">
397            <button
398              type="button"
399              role="tab"
400              data-testid="tab-admin-departments"
401              aria-selected={activeSection === 'departments'}
402              onClick={() => setActiveSection('departments')}
403              className={`flex-1 min-h-[40px] rounded-md px-3 py-1.5 text-xs font-bold
transition-all ${
404                activeSection === 'departments'
405                  ? 'bg-white text-blue-700 shadow-xs dark:bg-slate-900 dark:text-blue-300'
406                  : 'text-slate-600 hover:text-slate-900 dark:text-slate-400 dark:hover:text-
white'
407              }`}
408            >
409              🏛️ Departamentos ({graph.nodes.length})
410            </button>
411            <button
```

```
412             type="button"
413             role="tab"
414             data-testid="tab-admin-roads"
415             aria-selected={activeSection === 'roads'}
416             onClick={() => setActiveSection('roads')}
417             className={`flex-1 min-h-[40px] rounded-md px-3 py-1.5 text-xs font-bold
transition-all ${
418               activeSection === 'roads'
419                 ? 'bg-white text-blue-700 shadow-xs dark:bg-slate-900 dark:text-blue-300'
420                 : 'text-slate-600 hover:text-slate-900 dark:text-slate-400 dark:hover:text-
white'
421             }`}
422           >
423             🛣️ Carreteras ({graph.edges.length})
424           </button>
425         </div>
426
427         {/* SECCIÓN 1: DEPARTAMENTOS */}
428         {activeSection === 'departments' && (
429           <div className="flex flex-col gap-4" data-testid="section-admin-departments">
430             {/* Formulario para agregar departamento */}
```

```
            <form
              onSubmit={handleAddDept}
              className="rounded-xl border border-slate-200 bg-slate-50/80 p-3 dark:border-slate-800 dark:bg-slate-900/60"
            >
              <h2 className="mb-2 font-bold text-slate-900 dark:text-slate-100">+ Agregar nuevo departamento</h2>
              <div className="grid grid-cols-1 gap-2.5 sm:grid-cols-2">
                <div>
                  <label className="block mb-1 font-semibold text-slate-700 dark:text-slate-300">
                    Nombre del departamento *
                  </label>
                  <input
                    type="text"
                    data-testid="input-dept-label"
                    value={newDeptLabel}
                    onChange={(e) => setNewDeptLabel(e.target.value)}
                    placeholder="Ej. Petén"
                    required
                    className="min-h-[40px] w-full rounded-lg border border-slate-300 bg-white px-2.5 py-1 text-xs shadow-2xs dark:border-slate-700 dark:bg-slate-800 dark:text-slate-100"
```

```
                    />
                  </div>
                  <div>
                    <label className="block mb-1 font-semibold text-slate-700 dark:text-
slate-300">
                      Cabecera departamental
                    </label>
                    <input
                      type="text"
                      data-testid="input-dept-cabecera"
                      value={newDeptCabecera}
                      onChange={(e) => setNewDeptCabecera(e.target.value)}
                      placeholder="Ej. Flores"
                      className="min-h-[40px] w-full rounded-lg border border-slate-300 bg-
white px-2.5 py-1 text-xs shadow-2xs dark:border-slate-700 dark:bg-slate-800 dark:text-
slate-100"
                    />
                  </div>
                  <div>
                    <label className="block mb-1 text-slate-600 dark:text-slate-400">Latitud
(opcional)</label>
                    <input
```

```
                    type="number"
                    step="any"
                    data-testid="input-dept-lat"
                    value={newDeptLat}
                    onChange={(e) => setNewDeptLat(e.target.value)}
                    placeholder="Ej. 16.9248"
                    className="min-h-[40px] w-full rounded-lg border border-slate-300 bg-white px-2.5 py-1 text-xs shadow-2xs dark:border-slate-700 dark:bg-slate-800 dark:text-slate-100"
                  />
                </div>
                <div>
                  <label className="block mb-1 text-slate-600 dark:text-slate-400">Longitud (opcional)</label>
                  <input
                    type="number"
                    step="any"
                    data-testid="input-dept-lon"
                    value={newDeptLon}
                    onChange={(e) => setNewDeptLon(e.target.value)}
                    placeholder="Ej. -89.8974"
```

```
                    className="min-h-[40px] w-full rounded-lg border border-slate-300 bg-white px-2.5 py-1 text-xs shadow-2xs dark:border-slate-700 dark:bg-slate-800 dark:text-slate-100"
                  />
                </div>
              </div>

              {deptFormError && (
                <div data-testid="dept-form-error" className="mt-2 text-rose-600 font-bold dark:text-rose-400">
                  {deptFormError}
                </div>
              )}

              <div className="mt-3 flex justify-end">
                <button
                  type="submit"
                  data-testid="btn-save-department"
                  onClick={handleAddDept}
                  className="min-h-[42px] rounded-lg bg-emerald-600 px-4 py-1.5 font-bold text-white shadow-xs hover:bg-emerald-700 focus-visible:ring-2 focus-visible:ring-emerald-500"
                >
```

```
                  Agregar departamento
                </button>
              </div>
            </form>

            {/* Listado de departamentos con buscador */}
            <div className="flex flex-col gap-2">
              <div className="flex items-center justify-between">
                <h3 className="font-bold text-slate-800 dark:text-slate-200">
                  Departamentos registrados ({filteredDepartments.length}):
                </h3>
                <input
                  type="text"
                  data-testid="input-dept-search"
                  value={deptSearch}
                  onChange={(e) => setDeptSearch(e.target.value)}
                  placeholder="Buscar departamento o cabecera…"
                  className="min-h-[38px] w-56 rounded-lg border border-slate-300 bg-white
px-2.5 py-1 text-xs shadow-2xs dark:border-slate-700 dark:bg-slate-800 dark:text-slate-100"
                />
```

```
522               </div>
523
524               <div className="overflow-x-auto rounded-xl border border-slate-200 shadow-xs
dark:border-slate-800 max-h-72">
525                 <table className="w-full text-left border-collapse" data-testid="table-
departments">
526                   <thead className="sticky top-0 z-10 border-b border-slate-200 bg-slate-
100 dark:border-slate-700 dark:bg-slate-800">
527                     <tr className="text-[11px] font-bold text-slate-700 dark:text-slate-300
uppercase">
528                       <th className="py-2 px-3 w-12">ID</th>
529                       <th className="py-2 px-3">Departamento</th>
530                       <th className="py-2 px-3">Cabecera</th>
531                       <th className="py-2 px-3">Ubicación (lat, lon)</th>
532                       <th className="py-2 px-3 text-right">Acciones</th>
533                     </tr>
534                   </thead>
535                   <tbody className="divide-y divide-slate-200 dark:divide-slate-800 text-
[11px]">
536                     {filteredDepartments.map((n) => {
537                       const isEditing = editingDeptId === n.id;
538                       const cabecera = departments[n.id]?.cabecera ?? n.label;
539                       return (
```

```
                        <tr
                          key={n.id}
                          className="hover:bg-slate-50 dark:hover:bg-slate-800/40"
                          data-testid={`dept-row-${n.id}`}
                        >
                          <td className="py-2 px-3 font-mono font-bold text-slate-500">
{n.id}</td>
                          <td className="py-2 px-3 font-semibold text-slate-800 dark:text-
slate-100">
                            {isEditing ? (
                              <input
                                type="text"
                                data-testid={`input-edit-dept-name-${n.id}`}
                                value={editDeptLabel}
                                onChange={(e) => setEditDeptLabel(e.target.value)}
                                className="w-full rounded border px-1.5 py-0.5 text-xs
font-semibold"
                              />
                            ) : (
                              n.label
                            )}
                          </td>
```

```
                          <td className="py-2 px-3 text-slate-600 dark:text-slate-300">
                            {isEditing ? (
                              <input
                                type="text"
                                data-testid={`input-edit-dept-cabecera-${n.id}`}
                                value={editDeptCabecera}
                                onChange={(e) => setEditDeptCabecera(e.target.value)}
                                className="w-full rounded border px-1.5 py-0.5 text-xs"
                              />
                            ) : (
                              cabecera
                            )}
                          </td>
                          <td className="py-2 px-3 font-mono text-slate-500">
                            {isEditing ? (
                              <div className="flex gap-1">
                                <input
                                  type="number"
                                  step="any"
```

```
                                value={editDeptLat}
                                onChange={(e) => setEditDeptLat(e.target.value)}
                                placeholder="Lat"
                                className="w-16 rounded border px-1 py-0.5 text-[10px]"
                              />
                              <input
                                type="number"
                                step="any"
                                value={editDeptLon}
                                onChange={(e) => setEditDeptLon(e.target.value)}
                                placeholder="Lon"
                                className="w-16 rounded border px-1 py-0.5 text-[10px]"
                              />
                            </div>
                          ) : n.lat !== undefined && n.lon !== undefined ? (
                            `${n.lat.toFixed(4)}, ${n.lon.toFixed(4)}`
                          ) : (
                            'Posición lienzo'
                          )}
                        </td>
```

```
                          <td className="py-2 px-3 text-right">
                            {isEditing ? (
                              <div className="flex justify-end gap-1">
                                <button
                                  type="button"
                                  data-testid={`btn-save-dept-${n.id}`}
                                  onClick={() => handleSaveEditDept(n.id)}
                                  className="min-h-[34px] rounded bg-emerald-600 px-2 py-
0.5 font-bold text-white hover:bg-emerald-700"
                                >
                                  Guardar
                                </button>
                                <button
                                  type="button"
                                  data-testid={`btn-cancel-edit-dept-${n.id}`}
                                  onClick={() => setEditingDeptId(null)}
                                  className="min-h-[34px] rounded border border-slate-300
bg-white px-2 py-0.5 font-semibold text-slate-700 hover:bg-slate-100"
                                >
                                  Cancelar
                                </button>
```

```
                              </div>
                            ) : (
                              <div className="flex justify-end gap-1">
                                <button
                                  type="button"
                                  data-testid={`btn-edit-dept-${n.id}`}
                                  onClick={() => handleStartEditDept(n.id)}
                                  className="min-h-[34px] rounded px-2 py-0.5 font-semibold
text-blue-600 hover:bg-blue-50 dark:text-blue-400 dark:hover:bg-blue-950/40"
                                >
                                  Editar
                                </button>
                                <button
                                  type="button"
                                  data-testid={`btn-delete-dept-${n.id}`}
                                  onClick={() => handleDeleteDept(n.id)}
                                  className="min-h-[34px] rounded px-2 py-0.5 font-semibold
text-rose-600 hover:bg-rose-50 dark:text-rose-400 dark:hover:bg-rose-950/40"
                                >
                                  Eliminar
                                </button>
```

```
                              </div>
                            )}
                          </td>
                        </tr>
                      );
                    })}
                  </tbody>
                </table>
              </div>
            </div>
          </div>
        )}

        {/* SECCIÓN 2: CARRETERAS */}
        {activeSection === 'roads' && (
          <div className="flex flex-col gap-4" data-testid="section-admin-roads">
            {/* Formulario para agregar carretera */}
            <form
              onSubmit={handleAddRoad}
```

```
655               className="rounded-xl border border-slate-200 bg-slate-50/80 p-3 dark:border-
slate-800 dark:bg-slate-900/60"
656             >
657               <h2 className="mb-2 font-bold text-slate-900 dark:text-slate-100">
658                 + Conectar departamentos (nueva carretera)
659               </h2>
660               <div className="grid grid-cols-1 gap-2.5 sm:grid-cols-2 lg:grid-cols-4">
661                 <div>
662                   <label className="block mb-1 font-semibold text-slate-700 dark:text-
slate-300">
663                     Departamento Origen *
664                   </label>
665                   <select
666                     data-testid="select-road-source"
667                     value={newRoadSource}
668                     onChange={(e) => setNewRoadSource(e.target.value)}
669                     required
670                     className="min-h-[40px] w-full rounded-lg border border-slate-300 bg-
white px-2.5 py-1 text-xs shadow-2xs dark:border-slate-700 dark:bg-slate-800 dark:text-
slate-100"
671                   >
672                     {graph.nodes.map((n) => (
```

```
673                       <option key={n.id} value={n.id}>
674                         {n.label} ({departments[n.id]?.cabecera || n.label})
675                       </option>
676                     ))}
677                   </select>
678                 </div>
679                 <div>
680                   <label className="block mb-1 font-semibold text-slate-700 dark:text-
slate-300">
681                     Departamento Destino *
682                   </label>
683                   <select
684                     data-testid="select-road-target"
685                     value={newRoadTarget}
686                     onChange={(e) => setNewRoadTarget(e.target.value)}
687                     required
688                     className="min-h-[40px] w-full rounded-lg border border-slate-300 bg-
white px-2.5 py-1 text-xs shadow-2xs dark:border-slate-700 dark:bg-slate-800 dark:text-
slate-100"
689                   >
690                     {graph.nodes.map((n) => (
691                       <option key={n.id} value={n.id}>
```

```
                        {n.label} ({departments[n.id]?.cabecera || n.label})
                      </option>
                    ))}
                  </select>
                </div>
                <div>
                  <label className="block mb-1 font-semibold text-slate-700 dark:text-
slate-300">
                    Kilometraje (km) *
                  </label>
                  <input
                    type="number"
                    step="any"
                    min="0.1"
                    data-testid="input-road-km"
                    value={newRoadKm}
                    onChange={(e) => setNewRoadKm(e.target.value)}
                    placeholder="Ej. 42.5"
                    required
```

```
                    className="min-h-[40px] w-full rounded-lg border border-slate-300 bg-white px-2.5 py-1 text-xs shadow-2xs dark:border-slate-700 dark:bg-slate-800 dark:text-slate-100"
                  />
                </div>
                <div>
                  <label className="block mb-1 text-slate-600 dark:text-slate-400">
                    Nombre carretera (ej. CA-1, CA-9)
                  </label>
                  <input
                    type="text"
                    data-testid="input-road-name"
                    value={newRoadName}
                    onChange={(e) => setNewRoadName(e.target.value)}
                    placeholder="Ej. CA-9 Norte"
                    className="min-h-[40px] w-full rounded-lg border border-slate-300 bg-white px-2.5 py-1 text-xs shadow-2xs dark:border-slate-700 dark:bg-slate-800 dark:text-slate-100"
                  />
                </div>
              </div>

```

```
728                {roadFormError && (
729                  <div
730                    data-testid="road-form-error"
731                    className="mt-2 text-rose-600 font-bold dark:text-rose-400 flex items-
center gap-2"
732                  >
733                    <span>{roadFormError.message}</span>
734                    {roadFormError.existingEdgeId && (
735                      <button
736                        type="button"
737                        data-testid="btn-edit-existing-road"
738                        onClick={() => handleStartEditRoad(roadFormError.existingEdgeId!)}
739                        className="underline text-blue-600 hover:text-blue-800 dark:text-
blue-400"
740                      >
741                        Editar la existente
742                      </button>
743                    )}
744                  </div>
745                )}
746
```

```
              <div className="mt-3 flex justify-end">
                <button
                  type="submit"
                  data-testid="btn-save-road"
                  onClick={handleAddRoad}
                  className="min-h-[42px] rounded-lg bg-blue-600 px-4 py-1.5 font-bold
text-white shadow-xs hover:bg-blue-700 focus-visible:ring-2 focus-visible:ring-blue-500"
                >
                  Agregar carretera
                </button>
              </div>
            </form>

            {/* Listado ordenable de carreteras con buscador */}
            <div className="flex flex-col gap-2">
              <div className="flex items-center justify-between">
                <h3 className="font-bold text-slate-800 dark:text-slate-200">
                  Carreteras registradas ({sortedRoads.length}):
                </h3>
                <input
```

```
                    type="text"
                    data-testid="input-road-search"
                    value={roadSearch}
                    onChange={(e) => setRoadSearch(e.target.value)}
                    placeholder="Buscar por departamento o carretera…"
                    className="min-h-[38px] w-64 rounded-lg border border-slate-300 bg-white
px-2.5 py-1 text-xs shadow-2xs dark:border-slate-700 dark:bg-slate-800 dark:text-slate-100"
                  />
                </div>

                <div className="overflow-x-auto rounded-xl border border-slate-200 shadow-xs
dark:border-slate-800 max-h-72">
                  <table className="w-full text-left border-collapse" data-testid="table-
roads">
                    <thead className="sticky top-0 z-10 border-b border-slate-200 bg-slate-
100 dark:border-slate-700 dark:bg-slate-800">
                      <tr className="text-[11px] font-bold text-slate-700 dark:text-slate-300
uppercase">
                        <th
                          onClick={() => toggleSort('source')}
                          data-testid="sort-road-source"
                          className="py-2 px-3 cursor-pointer hover:bg-slate-200
dark:hover:bg-slate-700 select-none"
                        >
```

```
784                         Origen {sortField === 'source' ? (sortOrder === 'asc' ? '▲' : '▼')
: ''}
785                       </th>
786                       <th
787                         onClick={() => toggleSort('target')}
788                         data-testid="sort-road-target"
789                         className="py-2 px-3 cursor-pointer hover:bg-slate-200
dark:hover:bg-slate-700 select-none"
790                       >
791                         Destino {sortField === 'target' ? (sortOrder === 'asc' ? '▲' : '▼')
: ''}
792                       </th>
793                       <th
794                         onClick={() => toggleSort('road')}
795                         data-testid="sort-road-name"
796                         className="py-2 px-3 cursor-pointer hover:bg-slate-200
dark:hover:bg-slate-700 select-none"
797                       >
798                         Carretera {sortField === 'road' ? (sortOrder === 'asc' ? '▲' : '▼')
: ''}
799                       </th>
800                       <th
801                         onClick={() => toggleSort('km')}
```

```
802                         data-testid="sort-road-km"
803                         className="py-2 px-3 text-right cursor-pointer hover:bg-slate-200
dark:hover:bg-slate-700 select-none"
804                       >
805                         Kilometraje {sortField === 'km' ? (sortOrder === 'asc' ? '▲' : '▼')
: ''}
806                       </th>
807                       <th className="py-2 px-3 text-right">Acciones</th>
808                     </tr>
809                   </thead>
810                   <tbody className="divide-y divide-slate-200 dark:divide-slate-800 text-
[11px]">
811                     {sortedRoads.map((e) => {
812                       const isEditing = editingRoadId === e.id;
813                       const srcName = getNodeLabel(e.source);
814                       const tgtName = getNodeLabel(e.target);
815                       const roadName = getCarretera(e.meta) || '—';
816
817                       return (
818                         <tr
819                           key={e.id}
```

```
                          className="hover:bg-slate-50 dark:hover:bg-slate-800/40"
                          data-testid={`road-row-${e.id}`}
                        >
                          <td className="py-2 px-3 font-semibold text-slate-800 dark:text-
slate-100">{srcName}</td>
                          <td className="py-2 px-3 font-semibold text-slate-800 dark:text-
slate-100">{tgtName}</td>
                          <td className="py-2 px-3 text-slate-600 dark:text-slate-300">
                            {isEditing ? (
                              <input
                                type="text"
                                data-testid={`input-edit-road-name-${e.id}`}
                                value={editRoadName}
                                onChange={(evt) => setEditRoadName(evt.target.value)}
                                className="w-full rounded border px-1.5 py-0.5 text-xs"
                              />
                            ) : (
                              roadName
                            )}
                          </td>
                          <td className="py-2 px-3 text-right font-mono font-bold text-
slate-900 dark:text-slate-100">
```

```
                            {isEditing ? (
                              <input
                                type="number"
                                step="any"
                                min="0.1"
                                data-testid={`input-edit-road-km-${e.id}`}
                                value={editRoadKm}
                                onChange={(evt) => setEditRoadKm(evt.target.value)}
                                className="w-20 rounded border px-1.5 py-0.5 text-xs text-
right font-mono font-bold"
                              />
                            ) : (
                              `${e.weight.toFixed(1)} km`
                            )}
                          </td>
                          <td className="py-2 px-3 text-right">
                            {isEditing ? (
                              <div className="flex justify-end gap-1">
                                <button
                                  type="button"
```

```
                                data-testid={`btn-save-road-${e.id}`}
                                onClick={() => handleSaveEditRoad(e.id)}
                                className="min-h-[34px] rounded bg-emerald-600 px-2 py-0.5 font-bold text-white hover:bg-emerald-700"
                              >
                                Guardar
                              </button>
                              <button
                                type="button"
                                data-testid={`btn-cancel-edit-road-${e.id}`}
                                onClick={() => setEditingRoadId(null)}
                                className="min-h-[34px] rounded border border-slate-300 bg-white px-2 py-0.5 font-semibold text-slate-700 hover:bg-slate-100"
                              >
                                Cancelar
                              </button>
                            </div>
                          ) : (
                            <div className="flex justify-end gap-1">
                              <button
                                type="button"
```

```
                                data-testid={`btn-edit-road-${e.id}`}
                                onClick={() => handleStartEditRoad(e.id)}
                                className="min-h-[34px] rounded px-2 py-0.5 font-semibold
text-blue-600 hover:bg-blue-50 dark:text-blue-400 dark:hover:bg-blue-950/40"
                              >
                                Editar
                              </button>
                              <button
                                type="button"
                                data-testid={`btn-delete-road-${e.id}`}
                                onClick={() => handleDeleteRoad(e.id)}
                                className="min-h-[34px] rounded px-2 py-0.5 font-semibold
text-rose-600 hover:bg-rose-50 dark:text-rose-400 dark:hover:bg-rose-950/40"
                              >
                                Eliminar
                              </button>
                            </div>
                          )}
                        </td>
                      </tr>
                    );
```

```
                    })}
                  </tbody>
                </table>
              </div>
            </div>
          </div>
        )}
      </div>
    </Modal>
  );
}
```

src/ui/components/guatemala/GuatemalaAssistantModal.tsx

256 líneas.

```
/**
 * Diálogo asistente de dos pasos para el cálculo de la mejor ruta (G3).
 * Pregunta:
 *   Paso 1: «¿Dónde se encuentra el paquete?» (origen)
 *   Paso 2: «¿A qué departamento debe llegar?» (destino)
 * Incluye filtro de búsqueda reactivo por nombre de departamento o cabecera departamental.
 *
```

```
 * Lee y modifica: `graph.nodes`, `departments`, `setPathQuery` desde `useApp`.
 */
import { useMemo, useState } from 'react';
import type { NodeId } from '../../../core/types';
import { useApp } from '../../useApp';
import Modal from '../Modal';


interface GuatemalaAssistantModalProps {
  isOpen: boolean;
  onClose: () => void;
  onCalculated?: () => void;
}


export default function GuatemalaAssistantModal({ isOpen, onClose, onCalculated }:
GuatemalaAssistantModalProps) {
  const graph = useApp((s) => s.graph);
  const departments = useApp((s) => s.departments);
  const setPathQuery = useApp((s) => s.setPathQuery);

  const [step, setStep] = useState<1 | 2>(1);
```

```
27    const [selectedSource, setSelectedSource] = useState<NodeId | null>(null);
28    const [sourceSearch, setSourceSearch] = useState('');
29    const [targetSearch, setTargetSearch] = useState('');
30
31    const resetState = () => {
32      setStep(1);
33      setSelectedSource(null);
34      setSourceSearch('');
35      setTargetSearch('');
36    };
37
38    const handleClose = () => {
39      resetState();
40      onClose();
41    };
42
43    // Departamentos para el paso 1 filtrados por búsqueda
44    const filteredSources = useMemo(() => {
45      const term = sourceSearch.trim().toLowerCase();
46      return graph.nodes.filter((n) => {
```

```
      const label = n.label.toLowerCase();
      const cabecera = (departments[n.id]?.cabecera ?? '').toLowerCase();
      return !term || label.includes(term) || cabecera.includes(term);
    });
  }, [graph.nodes, departments, sourceSearch]);

  // Departamentos para el paso 2 (excluyendo el origen) filtrados por búsqueda
  const filteredTargets = useMemo(() => {
    const term = targetSearch.trim().toLowerCase();
    return graph.nodes.filter((n) => {
      if (n.id === selectedSource) return false;
      const label = n.label.toLowerCase();
      const cabecera = (departments[n.id]?.cabecera ?? '').toLowerCase();
      return !term || label.includes(term) || cabecera.includes(term);
    });
  }, [graph.nodes, departments, selectedSource, targetSearch]);

  const handleSelectSource = (id: NodeId) => {
    setSelectedSource(id);
```

```
      setStep(2);
    };

    const handleSelectTarget = (id: NodeId) => {
      if (!selectedSource) return;
      setPathQuery(selectedSource, id);
      handleClose();
      if (onCalculated) onCalculated();
    };

    const sourceNode = graph.nodes.find((n) => n.id === selectedSource);
    const sourceCabecera = selectedSource ? departments[selectedSource]?.cabecera : '';

    return (
      <Modal
        isOpen={isOpen}
        onClose={handleClose}
        title="Asistente de Envíos — Transporte Express"
        testId="modal-assistant"
      >
```

```
      <div className="flex flex-col gap-4 text-xs" data-testid="assistant-wizard">
        {/* Barra de progreso de pasos */}
        <div className="flex items-center justify-between border-b border-slate-200 pb-3 dark:border-slate-800">
          <div className="flex items-center gap-2">
            <span
              className={`flex h-6 w-6 items-center justify-center rounded-full text-xs font-bold ${
                step === 1 ? 'bg-blue-600 text-white' : 'bg-emerald-600 text-white'
              }`}
            >
              {step === 1 ? '1' : '✓'}
            </span>
            <span
              className={`font-semibold ${step === 1 ? 'text-blue-600 dark:text-blue-400' : 'text-slate-600 dark:text-slate-400'}`}
            >
              ¿Dónde está el paquete?
            </span>
          </div>

```

```
            <span className="text-slate-300 dark:text-slate-700">→</span>


            <div className="flex items-center gap-2">
              <span
                className={`flex h-6 w-6 items-center justify-center rounded-full text-xs
font-bold ${
                  step === 2
                    ? 'bg-blue-600 text-white'
                    : 'bg-slate-200 text-slate-600 dark:bg-slate-800 dark:text-slate-400'
                }`}
              >
                2
              </span>
              <span
                className={`font-semibold ${step === 2 ? 'text-blue-600 dark:text-blue-400' :
'text-slate-400 dark:text-slate-600'}`}
              >
                ¿A dónde debe llegar?
              </span>
            </div>
          </div>
```

```
123
124          {/* PASO 1: ORIGEN */}
125          {step === 1 && (
126            <div className="flex flex-col gap-3" data-testid="assistant-step-1">
127              <div className="flex items-center justify-between">
128                <h2 className="text-sm font-bold text-slate-900 dark:text-slate-100">
129                  1. ¿Dónde se encuentra el paquete?
130                </h2>
131                <span className="text-slate-500">{filteredSources.length}
    departamentos</span>
132              </div>
133
134              {/* Buscador de origen */}
135              <input
136                type="text"
137                data-testid="input-assistant-source-search"
138                value={sourceSearch}
139                onChange={(e) => setSourceSearch(e.target.value)}
140                placeholder="Buscar departamento o cabecera…"
141                autoFocus
```

```
              className="min-h-[44px] w-full rounded-lg border border-slate-300 bg-white px-3 py-2 text-xs shadow-xs focus-visible:ring-2 focus-visible:ring-blue-500 focus-visible:outline-hidden dark:border-slate-700 dark:bg-slate-800 dark:text-slate-100"
            />


            {/* Listado interactivo de departamentos origen */}
            <div className="max-h-72 overflow-y-auto rounded-lg border border-slate-200 p-1 divide-y divide-slate-100 dark:border-slate-800 dark:divide-slate-800/60">
              {filteredSources.map((n) => {
                const cab = departments[n.id]?.cabecera ?? n.label;
                return (
                  <button
                    key={n.id}
                    type="button"
                    data-testid={`btn-select-source-${n.id}`}
                    onClick={() => handleSelectSource(n.id)}
                    className="flex min-h-[44px] w-full items-center justify-between px-3 py-2 text-left rounded-md hover:bg-blue-50 dark:hover:bg-blue-950/40 transition-colors"
                  >
                    <div>
                      <div className="font-bold text-slate-800 dark:text-slate-200">{n.label}</div>
                      <div className="text-[11px] text-slate-500 dark:text-slate-400">Cabecera: {cab}</div>
```

```
                    </div>
                    <span className="rounded bg-slate-100 px-2 py-0.5 text-[10px] font-mono
text-slate-600 dark:bg-slate-800 dark:text-slate-300">
                      {n.id}
                    </span>
                  </button>
                );
              })}
              {filteredSources.length === 0 && (
                <div className="p-4 text-center text-slate-500">No se encontraron
departamentos coincidentes.</div>
              )}
            </div>
          </div>
        )}

        {/* PASO 2: DESTINO */}
        {step === 2 && (
          <div className="flex flex-col gap-3" data-testid="assistant-step-2">
            {/* Origen fijado con opción de cambiar */}
```

```
178             <div className="flex items-center justify-between rounded-lg bg-emerald-50 p-
    2.5 dark:bg-emerald-950/30 border border-emerald-200 dark:border-emerald-900/50">
179               <div>
180                 <span className="text-[11px] font-medium text-emerald-800 dark:text-
    emerald-300">Origen fijado:</span>
181                 <div className="font-bold text-emerald-950 dark:text-emerald-100">
182                   {sourceNode?.label} ({sourceCabecera})
183                 </div>
184               </div>
185               <button
186                 type="button"
187                 data-testid="btn-change-source"
188                 onClick={() => setStep(1)}
189                 className="min-h-[36px] rounded-md px-2.5 text-[11px] font-semibold text-
    emerald-700 hover:bg-emerald-100 dark:text-emerald-300 dark:hover:bg-emerald-900/40"
190               >
191                 Cambiar origen
192               </button>
193             </div>
194
195             <div className="flex items-center justify-between">
196               <h2 className="text-sm font-bold text-slate-900 dark:text-slate-100">
```

```
                2. ¿A qué departamento debe llegar?
              </h2>
              <span className="text-slate-500">{filteredTargets.length} disponibles</span>
            </div>

            {/* Buscador de destino */}
            <input
              type="text"
              data-testid="input-assistant-target-search"
              value={targetSearch}
              onChange={(e) => setTargetSearch(e.target.value)}
              placeholder="Buscar departamento o cabecera destino…"
              autoFocus
              className="min-h-[44px] w-full rounded-lg border border-slate-300 bg-white
px-3 py-2 text-xs shadow-xs focus-visible:ring-2 focus-visible:ring-blue-500 focus-
visible:outline-hidden dark:border-slate-700 dark:bg-slate-800 dark:text-slate-100"
            />

            {/* Listado interactivo de departamentos destino */}
            <div className="max-h-72 overflow-y-auto rounded-lg border border-slate-200 p-1
divide-y divide-slate-100 dark:border-slate-800 dark:divide-slate-800/60">
```

```
215                {filteredTargets.map((n) => {
216                  const cab = departments[n.id]?.cabecera ?? n.label;
217                  return (
218                    <button
219                      key={n.id}
220                      type="button"
221                      data-testid={`btn-select-target-${n.id}`}
222                      onClick={() => handleSelectTarget(n.id)}
223                      className="flex min-h-[44px] w-full items-center justify-between px-3
py-2 text-left rounded-md hover:bg-purple-50 dark:hover:bg-purple-950/40 transition-colors"
224                    >
225                      <div>
226                        <div className="font-bold text-slate-800 dark:text-slate-200">
{n.label}</div>
227                        <div className="text-[11px] text-slate-500 dark:text-slate-
400">Cabecera: {cab}</div>
228                      </div>
229                      <span className="rounded bg-slate-100 px-2 py-0.5 text-[10px] font-mono
text-slate-600 dark:bg-slate-800 dark:text-slate-300">
230                        {n.id}
231                      </span>
232                    </button>
```

```
                );
              })}
              {filteredTargets.length === 0 && (
                <div className="p-4 text-center text-slate-500">No se encontraron departamentos coincidentes.</div>
              )}
            </div>
          </div>
        )}

        {/* Botones inferiores */}
        <div className="flex justify-end gap-2 pt-2 border-t border-slate-200 dark:border-slate-800">
          <button
            type="button"
            data-testid="btn-cancel-assistant"
            onClick={handleClose}
            className="min-h-[44px] rounded-lg border border-slate-300 bg-white px-4 py-2 font-semibold text-slate-700 hover:bg-slate-100 dark:border-slate-700 dark:bg-slate-800 dark:text-slate-200"
          >
            Cancelar
```

```
            </button>
          </div>
        </div>
      </Modal>
    );
  }
```

src/ui/components/guatemala/GuatemalaOutline.tsx

43 líneas.

```
/**
 * Contorno de la República de Guatemala para el módulo Transporte Express.
 * Muestra los polígonos del contorno del país proyectados en coordenadas del lienzo (Web Mercator),
 * como una capa SVG de fondo tenue que se desplaza y escala en sincronía con el viewport de React Flow.
 * Lee: `outline` y `mode` desde el store global (`useApp`).
 */
import { useViewport } from '@xyflow/react';
import { useApp } from '../../useApp';


export default function GuatemalaOutline() {
  const mode = useApp((s) => s.mode);
```

```
  const outline = useApp((s) => s.outline);
  const { x, y, zoom } = useViewport();

  if (mode !== 'guatemala' || !outline || outline.length === 0) {
    return null;
  }

  const pathData = outline
    .map((ring) => {
      if (!ring || ring.length === 0) return '';
      return 'M ' + ring.map((pt) => `${pt.x},${pt.y}`).join(' L ') + ' Z';
    })
    .filter(Boolean)
    .join(' ');

  if (!pathData) return null;

  return (
    <svg className="pointer-events-none absolute inset-0 h-full w-full overflow-visible z-
0" aria-hidden="true">
```

```
      <g transform={`translate(${x}, ${y}) scale(${zoom})`}>
        <path
          d={pathData}
          data-testid="guatemala-outline"
          className="fill-emerald-500/8 dark:fill-emerald-400/8 stroke-emerald-600/30 dark:stroke-emerald-400/25"
          style={{ vectorEffect: 'non-scaling-stroke' }}
          strokeWidth={1.5}
          strokeDasharray="4 3"
        />
      </g>
    </svg>
  );
}
```

src/ui/components/guatemala/GuatemalaPresentation.tsx

207 líneas.

```
/**
 * Modo Presentación a pantalla completa para el módulo Guatemala (G5).
 * Oculta los paneles secundarios (barra superior estándar, panel de pestañas),
 * maximiza el mapa a pantalla completa y muestra una tarjeta flotante de alta legibilidad
```

```
 * con el resultado activo (ruta óptima o red mínima) en tipografía grande (1280×720 y 1920×1080).
 * Permite cambiar entre los ejemplos rápidos y salir presionando la tecla Escape o el botón de salida.
 *
 * Lee: `mode`, `query`, `shortest`, `breakdown`, `mst`, `coverMode`, `setPathQuery`, `setCoverMode`, `clearQuery` desde `useApp`.
 */
import { useEffect } from 'react';
import type { NodeId } from '../../../core/types';
import { useApp } from '../../useApp';


interface GuatemalaPresentationProps {
  isOpen: boolean;
  onExit: () => void;
}


export default function GuatemalaPresentation({ isOpen, onExit }: GuatemalaPresentationProps) {
  const query = useApp((s) => s.query);
  const shortest = useApp((s) => s.shortest);
  const breakdown = useApp((s) => s.breakdown);
```

```
    const mst = useApp((s) => s.mst);
    const coverMode = useApp((s) => s.coverMode);
    const graph = useApp((s) => s.graph);
    const setPathQuery = useApp((s) => s.setPathQuery);
    const setCoverMode = useApp((s) => s.setCoverMode);
    const clearQuery = useApp((s) => s.clearQuery);

    // Escuchar tecla Escape para salir de presentación
    useEffect(() => {
      if (!isOpen) return;
      const handleKeyDown = (e: KeyboardEvent) => {
        if (e.key === 'Escape') {
          onExit();
        }
      };
      window.addEventListener('keydown', handleKeyDown);
      return () => window.removeEventListener('keydown', handleKeyDown);
    }, [isOpen, onExit]);

    if (!isOpen) return null;
```

```
43
44    const getNodeLabel = (id: NodeId): string => {
45      return graph.nodes.find((n) => n.id === id)?.label ?? id;
46    };
47
48    const currentBreakdown = breakdown?.[0];
49    const isPathActive = query?.kind === 'path' && query.target && shortest &&
      shortest.length > 0;
50    const isMstActive = coverMode === 'mst' || !isPathActive;
51
52    return (
53      <div
54        data-testid="guatemala-presentation-overlay"
55        className="fixed inset-0 z-50 pointer-events-none flex flex-col justify-between p-4
      sm:p-6 select-none"
56      >
57        {/* Barra superior flotante de la presentación */}
58        <header className="pointer-events-auto flex flex-wrap items-center justify-between
      gap-3 rounded-2xl border border-slate-700/80 bg-slate-900/90 px-5 py-3 shadow-2xl backdrop-
      blur-md text-white">
59          <div className="flex items-center gap-3">
60            <div className="flex h-10 w-10 items-center justify-center rounded-xl bg-blue-600
      font-black text-lg text-white shadow-md">
```

```
            TE
          </div>
          <div>
            <h1 className="text-lg font-black tracking-tight text-white sm:text-xl">Transporte Express — Guatemala</h1>
            <p className="text-xs text-slate-400">Modo Presentación · Red Vial Nacional de 22 Departamentos</p>
          </div>
        </div>

        {/* Ejemplos rápidos en presentación */}
        <div className="flex flex-wrap items-center gap-1.5" role="group" aria-label="Ejemplos rápidos">
          <button
            type="button"
            data-testid="btn-pres-que-iza"
            onClick={() => setPathQuery('QUE', 'IZA')}
            className={`min-h-[44px] rounded-lg px-3 py-1.5 text-xs font-bold transition-all ${
              query?.kind === 'path' && query.source === 'QUE' && query.target === 'IZA'
                ? 'bg-blue-600 text-white shadow-xs'
                : 'border border-slate-700 bg-slate-800 text-slate-200 hover:bg-slate-700'
```

```
              }`}
            >
              QUE → IZA
            </button>
            <button
              type="button"
              data-testid="btn-pres-gua-pet"
              onClick={() => setPathQuery('GUA', 'PET')}
              className={`min-h-[44px] rounded-lg px-3 py-1.5 text-xs font-bold transition-
all ${
                query?.kind === 'path' && query.source === 'GUA' && query.target === 'PET'
                  ? 'bg-blue-600 text-white shadow-xs'
                  : 'border border-slate-700 bg-slate-800 text-slate-200 hover:bg-slate-700'
              }`}
            >
              GUA → PET
            </button>
            <button
              type="button"
              data-testid="btn-pres-sma-jut"
```

```
              onClick={() => setPathQuery('SMA', 'JUT')}
              className={`min-h-[44px] rounded-lg px-3 py-1.5 text-xs font-bold transition-all ${
                query?.kind === 'path' && query.source === 'SMA' && query.target === 'JUT'
                  ? 'bg-blue-600 text-white shadow-xs'
                  : 'border border-slate-700 bg-slate-800 text-slate-200 hover:bg-slate-700'
              }`}
            >
              SMA → JUT
            </button>
            <button
              type="button"
              data-testid="btn-pres-mst"
              onClick={() => {
                setCoverMode('mst');
                clearQuery();
              }}
              className={`min-h-[44px] rounded-lg px-3 py-1.5 text-xs font-bold transition-all ${
                coverMode === 'mst' && !query
                  ? 'bg-emerald-600 text-white shadow-xs'
```

```
                : 'border border-slate-700 bg-slate-800 text-slate-200 hover:bg-slate-700'
            }`}
          >
            🌳 Red Mínima
          </button>
        </div>

        {/* Botón salir de presentación */}
        <button
          type="button"
          data-testid="btn-exit-presentation"
          onClick={onExit}
          className="min-h-[44px] rounded-xl bg-rose-600 px-4 py-2 text-xs font-bold text-white shadow-md hover:bg-rose-700 transition-colors focus-visible:ring-2 focus-visible:ring-rose-400"
        >
          ✕ Salir (Esc)
        </button>
      </header>

      {/* Tarjeta flotante inferior de resultados en gran tamaño */}
```

```
      <div className="pointer-events-auto w-full max-w-xl self-start overflow-y-auto rounded-2xl border border-slate-700/80 bg-slate-900/95 p-5 shadow-2xl backdrop-blur-md text-white max-h-[36vh] lg:fixed lg:right-6 lg:top-28 lg:max-h-[calc(100vh-8rem)] lg:w-[370px] lg:max-w-none">

        {isPathActive && currentBreakdown ? (

          <div className="space-y-3" data-testid="pres-route-card">

            <div className="flex items-center justify-between border-b border-slate-800 pb-2">

              <span className="text-xs font-bold text-blue-400 uppercase tracking-wider">

                Mejor ruta calculada (Dijkstra)

              </span>

              <span className="text-xs text-slate-400 font-mono">
{currentBreakdown.legs.length} tramos</span>

            </div>



            <div className="flex items-baseline justify-between gap-4">

              <div className="text-2xl sm:text-3xl font-black text-white">

                {getNodeLabel(currentBreakdown.nodes[0]!)} →{' '}

                {getNodeLabel(currentBreakdown.nodes[currentBreakdown.nodes.length - 1]!)}

              </div>

              <div

                data-testid="pres-total-distance"

                className="text-3xl sm:text-4xl font-black text-blue-400 font-mono"
```

```
              >
                {currentBreakdown.total.toFixed(1)} km
              </div>
            </div>

            <div className="text-sm font-mono text-slate-300 bg-slate-800/80 rounded-lg p-2.5 break-all">
              {currentBreakdown.formula}
            </div>

            {/* Secuencia resumida de departamentos */}
            <div className="flex flex-wrap items-center gap-1.5 text-xs font-bold text-slate-200 pt-1">
              {currentBreakdown.nodes.map((nodeId, i) => (
                <span key={`${nodeId}-${i}`} className="inline-flex items-center gap-1.5">
                  <span className="rounded-md bg-slate-800 px-2 py-1 border border-slate-700">
                    {getNodeLabel(nodeId)}
                  </span>
                  {i < currentBreakdown.nodes.length - 1 && <span className="text-slate-500">→</span>}
                </span>
```

```
172                ))}
173              </div>
174            </div>
175          ) : isMstActive && mst ? (
176            <div className="space-y-3" data-testid="pres-mst-card">
177              <div className="flex items-center justify-between border-b border-slate-800 pb-
2">
178                <span className="text-xs font-bold text-emerald-400 uppercase tracking-
wider">
179                  Red Mínima Departamental (Prim / Kruskal)
180                </span>
181                <span className="text-xs text-emerald-300 font-mono">
182                  {mst ? `${mst.prim.edges.length} carreteras` : '21 carreteras'}
183                </span>
184              </div>
185
186              <div className="flex items-baseline justify-between gap-4">
187                <div className="text-xl sm:text-2xl font-black text-white">Conexión completa
de los 22 departamentos</div>
188                <div data-testid="pres-mst-total" className="text-3xl sm:text-4xl font-black
text-emerald-400 font-mono">
189                  {mst.prim.total.toFixed(1)} km
```

```
              </div>
            </div>

            <p className="text-xs text-slate-300 leading-relaxed">
              ✓ Algoritmos de Prim y Kruskal verificados: ambos conectan todos los departamentos con un coste total
              idéntico de <strong>{mst.prim.total.toFixed(1)} km</strong> y {mst.prim.edges.length} carreteras sin
              formar ciclos.
            </p>
          </div>
        ) : (
          <div className="text-sm text-slate-300">
            Selecciona un ejemplo rápido arriba para proyectar los resultados.
          </div>
        )}
      </div>
    </div>
  );
}
```

`src/ui/components/guatemala/GuatemalaRouteBreakdown.tsx`

283 líneas.

```tsx
/**
 * Desglose de la mejor ruta para Transporte Express (G3).
 * Muestra:
 *   - Listado de departamentos en orden (A → B → … → Destino)
 *   - Total en grande (km)
 *   - Fórmula de distancias de los tramos (ej. 28.4 + 18.2 = 46.6 km)
 *   - Tabla de tramos (# | Desde | Hasta | Carretera | km del tramo | km acumulado)
 *   - Gestión de empates (todas las rutas mínimas empatadas seleccionables)
 *   - Mensaje claro si los departamentos no están conectados.
 *
 * Lee: `query`, `shortest`, `breakdown`, `highlightedPath`, `highlightPath`, `graph`, `departments` desde `useApp`.
 */
import type { NodeId } from '../../../core/types';
import { useApp } from '../../useApp';


interface GuatemalaRouteBreakdownProps {
  onOpenAssistant?: () => void;
}
```

```
19
20  export default function GuatemalaRouteBreakdown({ onOpenAssistant }:
    GuatemalaRouteBreakdownProps) {
21    const query = useApp((s) => s.query);
22    const shortest = useApp((s) => s.shortest);
23    const breakdown = useApp((s) => s.breakdown);
24    const highlightedPath = useApp((s) => s.highlightedPath);
25    const highlightPath = useApp((s) => s.highlightPath);
26    const graph = useApp((s) => s.graph);
27    const departments = useApp((s) => s.departments);
28    const clearQuery = useApp((s) => s.clearQuery);
29
30    const getNodeName = (id: NodeId): string => {
31      const d = departments[id];
32      const n = graph.nodes.find((node) => node.id === id);
33      if (d?.cabecera && n?.label && d.cabecera !== n.label) {
34        return `${n.label} (${d.cabecera})`;
35      }
36      return n?.label ?? id;
37    };
```

```
38
39    const getDeptShortName = (id: NodeId): string => {
40      return graph.nodes.find((n) => n.id === id)?.label ?? id;
41    };
42
43    // Sin consulta o consulta incompleta
44    if (!query || query.kind !== 'path' || !query.target) {
45      return (
46        <div
47          className="flex flex-col items-center justify-center p-6 text-center gap-3 text-xs"
48          data-testid="route-empty-state"
49        >
50          <div className="flex h-12 w-12 items-center justify-center rounded-full bg-blue-100
text-2xl text-blue-600 dark:bg-blue-950 dark:text-blue-300">
51            🚚
52          </div>
53          <div>
54            <h2 className="text-base font-bold text-slate-800 dark:text-slate-100">
55              Asistente de Rutas «Transporte Express»
56            </h2>
```

```
57            <p className="mt-1 text-slate-500 max-w-sm leading-relaxed dark:text-slate-400">
58              Seleccione el departamento de origen y destino para calcular la ruta más corta
    y ver su desglose en tramos.
59            </p>
60          </div>
61          {onOpenAssistant && (
62            <button
63              type="button"
64              data-testid="btn-open-assistant-breakdown"
65              onClick={onOpenAssistant}
66              className="min-h-[44px] rounded-lg bg-blue-600 px-4 py-2 font-bold text-white
    shadow-xs hover:bg-blue-700 transition-colors focus-visible:ring-2 focus-visible:ring-blue-
    500"
67            >
68              📦 Calcular mejor ruta
69            </button>
70          )}
71        </div>
72      );
73    }
74
75    // Grafo desconectado para este par
```

```
  if (shortest && shortest.length === 0) {
    return (
      <div className="p-4 text-xs" data-testid="no-route-alert">
        <div className="rounded-xl border border-rose-200 bg-rose-50/90 p-4 text-rose-900 shadow-xs dark:border-rose-900/60 dark:bg-rose-950/40 dark:text-rose-200">
          <div className="flex items-start gap-3">
            <span className="text-xl">⚠️</span>
            <div className="space-y-1">
              <h2 className="font-bold text-sm">Sin conexión por carretera</h2>
              <p className="leading-relaxed">
                No existe conexión vial entre <strong>{getNodeName(query.source)}</strong> y{' '}
                <strong>{getNodeName(query.target)}</strong>. La red vial se encuentra desconectada para este par de
                destinos.
              </p>
              <div className="pt-2">
                <button
                  type="button"
                  data-testid="btn-clear-disconnected-query"
                  onClick={clearQuery}
```

```
                  className="min-h-[38px] rounded-lg border border-rose-300 bg-white px-3 py-1 font-semibold text-rose-700 hover:bg-rose-50 dark:border-rose-800 dark:bg-slate-900 dark:text-rose-300"
                >
                  Limpiar consulta
                </button>
              </div>
            </div>
          </div>
        </div>
      );
    }

    // Ruta(s) encontrada(s)
    const activeIndex = highlightedPath?.from === 'shortest' ? highlightedPath.index : 0;
    const currentBreakdown = breakdown?.[activeIndex] ?? breakdown?.[0];

    if (!currentBreakdown) {
      return null;
    }
```

```
113
114   const isTied = (shortest?.length ?? 0) > 1;
115
116   return (
117     <div className="flex flex-col gap-4 p-4 text-xs" data-testid="route-breakdown-
container">
118       {/* Banner de empates si hay múltiples rutas mínimas con la misma distancia */}
119       {isTied && shortest && (
120         <div
121           className="rounded-xl border border-purple-200 bg-purple-50/80 p-3 shadow-xs
dark:border-purple-900/50 dark:bg-purple-950/30"
122           data-testid="ties-banner"
123         >
124           <div className="font-bold text-purple-900 dark:text-purple-200 mb-2">
125             ⭐ Se encontraron {shortest.length} rutas mínimas empatadas
({shortest[0]?.distance.toFixed(1)} km):
126           </div>
127           <div className="flex flex-wrap gap-1.5" role="tablist" aria-label="Rutas
empatadas">
128             {shortest.map((_, idx) => (
129               <button
130                 key={idx}
```

```
                type="button"
                role="tab"
                data-testid={`btn-tie-route-${idx}`}
                aria-selected={activeIndex === idx}
                onClick={() => highlightPath({ from: 'shortest', index: idx })}
                className={`min-h-[38px] rounded-lg px-3 py-1.5 text-xs font-bold
transition-all ${
                  activeIndex === idx
                    ? 'bg-purple-600 text-white shadow-xs'
                    : 'border border-purple-200 bg-white text-purple-700 hover:bg-purple-
100 dark:border-purple-800 dark:bg-slate-900 dark:text-purple-300'
                }`}
              >
                Ruta {idx + 1} {activeIndex === idx ? '(activa)' : ''}
              </button>
            ))}
          </div>
        </div>
      )}

      {/* Resumen principal: Total grande y Fórmula */}
```

```
      <div className="rounded-xl border border-blue-200 bg-blue-50/70 p-4 shadow-xs dark:border-blue-900/40 dark:bg-blue-950/20">
        <div className="flex flex-wrap items-end justify-between gap-3">
          <div>
            <div className="text-[11px] font-semibold text-blue-700 dark:text-blue-300 uppercase tracking-wider">
              Distancia total óptima
            </div>
            <div data-testid="route-total-distance" className="text-3xl font-black text-blue-800 dark:text-blue-200">
              {currentBreakdown.total.toFixed(1)} km
            </div>
          </div>

          <div className="text-right">
            <span className="text-[11px] font-semibold text-slate-500 dark:text-slate-400">
              Tramos: {currentBreakdown.legs.length} · Paradas: {currentBreakdown.nodes.length}
            </span>
          </div>
        </div>
```

```
168         {/* Fórmula matemática de la suma de tramos */}
169         <div className="mt-2.5 pt-2.5 border-t border-blue-200/60 dark:border-blue-900/40">
170           <div className="text-[10px] font-bold text-slate-500 dark:text-slate-400
    uppercase">Fórmula:</div>
171           <div
172             data-testid="route-formula"
173             className="font-mono text-xs font-bold text-slate-800 dark:text-slate-200
    break-all"
174           >
175             {currentBreakdown.formula}
176           </div>
177         </div>
178       </div>
179
180       {/* Listado de nodos en orden */}
181       <div className="rounded-xl border border-slate-200 bg-slate-50 p-3 dark:border-slate-
    800 dark:bg-slate-900/60">
182         <div className="mb-2 font-bold text-slate-700 dark:text-slate-300">Secuencia de
    departamentos en la ruta:</div>
183         <div data-testid="route-nodes-sequence" className="flex flex-wrap items-center gap-
    1.5 leading-relaxed">
184           {currentBreakdown.nodes.map((nodeId, idx) => {
185             const isFirst = idx === 0;
```

```
              const isLast = idx === currentBreakdown.nodes.length - 1;
              return (
                <span key={`${nodeId}-${idx}`} className="inline-flex items-center gap-1.5">
                  <span
                    data-testid={`route-node-pill-${nodeId}`}
                    className={`inline-flex items-center gap-1 rounded-md px-2 py-1 text-xs
font-bold shadow-2xs ${
                      isFirst
                        ? 'bg-emerald-600 text-white'
                        : isLast
                          ? 'bg-rose-600 text-white'
                          : 'bg-white text-slate-800 border border-slate-200 dark:bg-slate-
800 dark:text-slate-100 dark:border-slate-700'
                    }`}
                  >
                    <span className="font-mono text-[10px] opacity-80">{nodeId}</span>
                    <span>{getDeptShortName(nodeId)}</span>
                    {departments[nodeId]?.cabecera && (
                      <span className="text-[10px] opacity-80 font-normal">
({departments[nodeId].cabecera})</span>
                    )}
                  </span>
```

```
                {!isLast && <span className="font-bold text-slate-400">→</span>}
              </span>
            );
          })}
        </div>
      </div>

      {/* Tabla detallada de tramos */}
      <div className="overflow-x-auto rounded-xl border border-slate-200 shadow-xs
dark:border-slate-800">
        <table className="w-full min-w-[560px] text-left border-collapse" data-
testid="table-route-legs">
          <thead>
            <tr className="border-b border-slate-200 bg-slate-100 text-[11px] font-bold
text-slate-700 uppercase dark:border-slate-700 dark:bg-slate-800 dark:text-slate-300">
              <th className="py-2.5 px-3 w-10 text-center">#</th>
              <th className="py-2.5 px-3">Desde</th>
              <th className="py-2.5 px-3">Hasta</th>
              <th className="py-2.5 px-3">Carretera</th>
              <th className="py-2.5 px-3 text-right">km del tramo</th>
              <th className="py-2.5 px-3 text-right">km acumulado</th>
```

```
              </tr>
            </thead>
            <tbody className="divide-y divide-slate-200 font-mono text-[11px] dark:divide-slate-800">
              {currentBreakdown.legs.map((leg, idx) => (
                <tr
                  key={`${leg.edgeId}-${idx}`}
                  className="hover:bg-slate-50 dark:hover:bg-slate-800/40 transition-colors"
                  data-testid={`leg-row-${idx}`}
                >
                  <td className="py-2.5 px-3 text-center font-bold text-slate-500">{idx + 1}</td>
                  <td className="py-2.5 px-3 font-sans font-medium text-slate-800 dark:text-slate-200">
                    {getNodeName(leg.from)}
                  </td>
                  <td className="py-2.5 px-3 font-sans font-medium text-slate-800 dark:text-slate-200">
                    {getNodeName(leg.to)}
                  </td>
                  <td className="py-2.5 px-3 font-sans">
                    {leg.road ? (
```

```
                    <span className="rounded bg-blue-100 px-1.5 py-0.5 text-[10px] font-
bold text-blue-800 dark:bg-blue-950 dark:text-blue-300">
                      {leg.road}
                    </span>
                  ) : (
                    <span className="text-slate-400 italic">Red local</span>
                  )}
                </td>
                <td className="py-2.5 px-3 text-right font-semibold text-slate-800
dark:text-slate-200">
                  {leg.km.toFixed(1)} km
                </td>
                <td className="py-2.5 px-3 text-right font-bold text-blue-700 dark:text-
blue-300">
                  {leg.cumulative.toFixed(1)} km
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>

```

```
        {/* Botones de acción inferior */}
        <div className="flex items-center justify-between pt-1">
          {onOpenAssistant && (
            <button
              type="button"
              data-testid="btn-recalculate-route"
              onClick={onOpenAssistant}
              className="min-h-[44px] rounded-lg border border-slate-300 bg-white px-3 py-1.5
font-bold text-slate-700 hover:bg-slate-50 dark:border-slate-700 dark:bg-slate-800
dark:text-slate-200"
            >
              🔄 Nueva consulta de ruta
            </button>
          )}
          <button
            type="button"
            data-testid="btn-clear-route-query"
            onClick={clearQuery}
            className="min-h-[44px] rounded-lg px-3 py-1.5 text-xs text-slate-500 hover:text-
slate-800 dark:text-slate-400 dark:hover:text-slate-200"
          >
```

```
            ✕ Limpiar ruta
          </button>
        </div>
      </div>
    );
  }
```

src/ui/components/guatemala/GuatemalaToolbar.tsx

150 líneas.

```
/**
 * Barra de herramientas complementaria para el módulo Guatemala — Transporte Express (G1–G5).
 * Proporciona acceso directo a:
 *   - Botón destacado «Calcular mejor ruta» (asistente de dos pasos G3)
 *   - Ejemplos rápidos (QUE→IZA, GUA→PET, SMA→JUT, Red mínima)
 *   - Botón «Administrar red» (G2)
 *   - Botón «Modo presentación» (G5)
 *   - Botón «Acerca de» (G5)
 *
 * Lee y modifica: `query`, `setPathQuery`, `coverMode`, `setCoverMode`, `clearQuery` desde `useApp`.
 */
```

```
import { useApp } from '../../useApp';



interface GuatemalaToolbarProps {
  onOpenAssistant: () => void;
  onOpenAdmin: () => void;
  onOpenAbout: () => void;
  onStartPresentation: () => void;
}



export default function GuatemalaToolbar({
  onOpenAssistant,
  onOpenAdmin,
  onOpenAbout,
  onStartPresentation,
}: GuatemalaToolbarProps) {
  const query = useApp((s) => s.query);
  const setPathQuery = useApp((s) => s.setPathQuery);
  const coverMode = useApp((s) => s.coverMode);
  const setCoverMode = useApp((s) => s.setCoverMode);
  const clearQuery = useApp((s) => s.clearQuery);
```

```
32
33    return (
34      <div
35        data-testid="guatemala-toolbar"
36        className="shrink-0 flex items-center gap-2 border-b border-blue-100 bg-blue-50/70
px-2 py-1 shadow-2xs dark:border-slate-800 dark:bg-slate-900/80 sm:flex-wrap sm:justify-
between sm:px-3 sm:py-1.5"
37      >
38        {/* Botón principal del Asistente */}
39        <button
40          type="button"
41          data-testid="btn-calculate-route"
42          onClick={onOpenAssistant}
43          className="shrink-0 flex min-h-[38px] items-center gap-1.5 rounded-lg bg-blue-600
px-3 py-1.5 text-xs font-bold text-white shadow-xs hover:bg-blue-700 transition-all focus-
visible:ring-2 focus-visible:ring-blue-500"
44        >
45          <span>📦</span>
46          <span className="whitespace-nowrap">Calcular mejor ruta</span>
47        </button>
48
49        {/* Separador sutil en escritorio */}
```

```
      <span className="hidden sm:inline text-slate-300 dark:text-slate-700">|</span>


      {/* Fila con desplazamiento horizontal en móvil para ejemplos y acciones secundarias */}
      <div className="flex flex-1 items-center gap-1 overflow-x-auto no-scrollbar py-0.5 sm:flex-initial sm:overflow-visible sm:py-0">
        <span className="hidden lg:inline text-[11px] font-semibold text-slate-500 dark:text-slate-400 mr-0.5">
          Ejemplos:
        </span>
        <button
          type="button"
          data-testid="btn-example-que-iza"
          onClick={() => setPathQuery('QUE', 'IZA')}
          title="Quetzaltenango a Puerto Barrios (QUE → IZA)"
          className={`shrink-0 min-h-[36px] whitespace-nowrap rounded-md px-2.5 py-1 text-xs font-semibold transition-colors ${
            query?.kind === 'path' && query.source === 'QUE' && query.target === 'IZA'
              ? 'bg-blue-600 text-white shadow-2xs'
              : 'border border-slate-300 bg-white text-slate-700 hover:bg-slate-50 dark:border-slate-700 dark:bg-slate-800 dark:text-slate-200 dark:hover:bg-slate-700'
          }`}
        >
```

```
            QUE → IZA
          </button>
          <button
            type="button"
            data-testid="btn-example-gua-pet"
            onClick={() => setPathQuery('GUA', 'PET')}
            title="Ciudad de Guatemala a Flores (GUA → PET)"
            className={`shrink-0 min-h-[36px] whitespace-nowrap rounded-md px-2.5 py-1 text-
xs font-semibold transition-colors ${
              query?.kind === 'path' && query.source === 'GUA' && query.target === 'PET'
                ? 'bg-blue-600 text-white shadow-2xs'
                : 'border border-slate-300 bg-white text-slate-700 hover:bg-slate-50
dark:border-slate-700 dark:bg-slate-800 dark:text-slate-200 dark:hover:bg-slate-700'
            }`}
          >
            GUA → PET
          </button>
          <button
            type="button"
            data-testid="btn-example-sma-jut"
            onClick={() => setPathQuery('SMA', 'JUT')}
```

```
            title="San Marcos a Jutiapa (SMA → JUT)"
            className={`shrink-0 min-h-[36px] whitespace-nowrap rounded-md px-2.5 py-1 text-xs font-semibold transition-colors ${
              query?.kind === 'path' && query.source === 'SMA' && query.target === 'JUT'
                ? 'bg-blue-600 text-white shadow-2xs'
                : 'border border-slate-300 bg-white text-slate-700 hover:bg-slate-50 dark:border-slate-700 dark:bg-slate-800 dark:text-slate-200 dark:hover:bg-slate-700'
            }`}
          >
            SMA → JUT
          </button>
          <button
            type="button"
            data-testid="btn-example-mst"
            onClick={() => {
              setCoverMode('mst');
              clearQuery();
            }}
            title="Red mínima completa con Prim y Kruskal"
            className={`shrink-0 min-h-[36px] whitespace-nowrap rounded-md px-2.5 py-1 text-xs font-semibold transition-colors ${
              coverMode === 'mst' && !query
```

```
              ? 'bg-emerald-600 text-white shadow-2xs'
              : 'border border-slate-300 bg-white text-slate-700 hover:bg-slate-50 dark:border-slate-700 dark:bg-slate-800 dark:text-slate-200 dark:hover:bg-slate-700'
          }`}
        >
          🌳 Red Mínima
        </button>

        <span className="shrink-0 text-slate-300 dark:text-slate-700 sm:hidden">|</span>

        {/* Acciones de gestión, presentación e información */}
        <button
          type="button"
          data-testid="btn-admin-network"
          onClick={onOpenAdmin}
          className="shrink-0 flex min-h-[36px] items-center gap-1 rounded-lg border border-slate-300 bg-white px-2.5 py-1 text-xs font-semibold text-slate-700 shadow-2xs hover:bg-slate-50 dark:border-slate-700 dark:bg-slate-800 dark:text-slate-200 dark:hover:bg-slate-700"
          title="Administrar departamentos y carreteras"
        >
          <span>⚙️</span>
```

```
            <span className="hidden sm:inline">Administrar red</span>
          </button>

          <button
            type="button"
            data-testid="btn-presentation-mode"
            onClick={onStartPresentation}
            className="shrink-0 flex min-h-[36px] items-center gap-1 rounded-lg border border-purple-300 bg-purple-50 px-2.5 py-1 text-xs font-bold text-purple-700 shadow-2xs hover:bg-purple-100 dark:border-purple-800 dark:bg-purple-950/40 dark:text-purple-300"
            title="Modo pantalla completa para exposición"
          >
            <span>📽️</span>
            <span className="hidden sm:inline">Presentación</span>
          </button>

          <button
            type="button"
            data-testid="btn-about"
            onClick={onOpenAbout}
```

```
          className="shrink-0 flex min-h-[36px] min-w-[36px] items-center justify-center rounded-lg border border-slate-300 bg-white text-xs font-bold text-slate-700 shadow-2xs hover:bg-slate-50 dark:border-slate-700 dark:bg-slate-800 dark:text-slate-200"
          title="Acerca de los desarrolladores e institución"
        >
          ℹ️
        </button>
      </div>
    </div>
  );
}
```

`src/ui/components/tabs/MstTab.tsx`

407 líneas.

```
/**
 * Pestaña principal de cobertura y optimización de red (G4).
 * Permite seleccionar entre tres modos de conexión departamental:
 * (a) «Red mínima que conecta todos los departamentos (Prim / Kruskal)» — predeterminada
 * (b) «Rutas más cortas desde la bodega»
 * (c) «Recorrido de reparto (aproximación heurística)»
 *
 * Lee: `coverMode`, `setCoverMode`, `mst`, `dijkstra`, `setTreeQuery`, `tour`, `runTour`, `cancelTour`,
```

```
 * `graph`, `departments` desde `useApp`.
 */
import { useMemo, useState } from 'react';
import type { NodeId } from '../../../core/types';
import type { CoverMode } from '../../../store/types';
import { useApp } from '../../useApp';
import KruskalTab from './KruskalTab';
import PrimTab from './PrimTab';

export default function MstTab() {
  const coverMode = useApp((s) => s.coverMode);
  const setCoverMode = useApp((s) => s.setCoverMode);
  const mst = useApp((s) => s.mst);
  const graph = useApp((s) => s.graph);
  const departments = useApp((s) => s.departments);
  const dijkstra = useApp((s) => s.dijkstra);
  const query = useApp((s) => s.query);
  const setTreeQuery = useApp((s) => s.setTreeQuery);
  const tour = useApp((s) => s.tour);
  const runTour = useApp((s) => s.runTour);
```

```
  const cancelTour = useApp((s) => s.cancelTour);


  // Sub-pestaña activa dentro de la Red Mínima (AEM): 'prim' | 'kruskal' | 'aristas'
  const [mstAlgorithmTab, setMstAlgorithmTab] = useState<'prim' | 'kruskal' | 'aristas'>
('prim');


  // Estado para la bodega (F2) y para el inicio del recorrido de reparto
  const [selectedBodega, setSelectedBodega] = useState<NodeId>(
    query?.kind === 'tree' ? query.source : (graph.nodes[0]?.id ?? 'GUA'),
  );
  const [selectedTourStart, setSelectedTourStart] = useState<NodeId>(graph.nodes[0]?.id ??
'GUA');


  const getNodeName = (id: NodeId): string => {
    const d = departments[id];
    const n = graph.nodes.find((node) => node.id === id);
    if (d?.cabecera && n?.label && d.cabecera !== n.label) {
      return `${n.label} (${d.cabecera})`;
    }
    return n?.label ?? id;
  };
```

```
48
49    // Distancias desde la bodega ordenadas de menor a mayor
50    const bodegaDistances = useMemo(() => {
51      if (!dijkstra || query?.kind !== 'tree') return [];
52      return graph.nodes
53        .filter((n) => n.id !== dijkstra.source)
54        .map((n) => {
55          const dist = dijkstra.labels[n.id]?.dist ?? Infinity;
56          return {
57            id: n.id,
58            label: n.label,
59            cabecera: departments[n.id]?.cabecera ?? n.label,
60            dist,
61          };
62        })
63        .sort((a, b) => a.dist - b.dist);
64    }, [dijkstra, query, graph.nodes, departments]);
65
66    const primTotal = mst?.prim.total ?? 0;
```

```
  const kruskalTotal = mst?.kruskal.total ?? 0;
  const totalsMatch = Math.abs(primTotal - kruskalTotal) < 1e-4;


  return (
    <div className="flex flex-col gap-4 p-4 text-xs" data-testid="mst-tab-container">
      {/* Selector superior de modo de cobertura G4 */}
      <div className="rounded-xl border border-slate-200 bg-slate-50 p-3 dark:border-slate-800 dark:bg-slate-900/60">
        <label htmlFor="cover-mode-select" className="block mb-1.5 font-bold text-slate-800 dark:text-slate-200">
          Tipo de cobertura de red (G4):
        </label>
        <select
          id="cover-mode-select"
          data-testid="select-cover-mode"
          value={coverMode}
          onChange={(e) => setCoverMode(e.target.value as CoverMode)}
          aria-label="Tipo de cobertura de red"
          className="min-h-[44px] w-full rounded-lg border border-slate-300 bg-white px-3 py-2 text-xs font-semibold text-slate-800 shadow-xs focus-visible:ring-2 focus-visible:ring-blue-500 focus-visible:outline-hidden dark:border-slate-700 dark:bg-slate-800 dark:text-slate-100"
        >
```

```
            <option value="mst">
              (a) Red mínima que conecta todos los departamentos (Prim / Kruskal) — Predeterminada
            </option>
            <option value="spt">(b) Rutas más cortas desde la bodega</option>
            <option value="tour">(c) Recorrido de reparto (aproximación heurística)</option>
          </select>
        </div>

        {/* CASO A: RED MÍNIMA (AEM) CON PRIM Y KRUSKAL */}
        {coverMode === 'mst' && (
          <div className="flex flex-col gap-3">
            {/* Tarjeta de verificación cruzada y total */}
            <div className="flex flex-wrap items-center justify-between gap-3 rounded-xl border border-emerald-200 bg-emerald-50/80 p-3.5 shadow-xs dark:border-emerald-900/60 dark:bg-emerald-950/30">
              <div>
                <div className="text-[11px] font-semibold text-emerald-800 dark:text-emerald-300">
                  Árbol de Expansión Mínima (AEM)
                </div>
                <div
```

```
                  data-testid="mst-total-distance"
                  className="text-2xl font-black text-emerald-900 dark:text-emerald-100"
                >
                  {primTotal.toFixed(1)} km
                </div>
              </div>

              <div
                data-testid="mst-same-total-notice"
                className="rounded-lg border border-emerald-300 bg-white/80 px-3 py-1.5 text-
xs font-semibold text-emerald-900 dark:border-emerald-700 dark:bg-slate-900/80 dark:text-
emerald-200"
              >
                {totalsMatch ? (
                  <span>
                    ✓ Verificación cruzada: Prim ({primTotal.toFixed(1)} km) y Kruskal
({kruskalTotal.toFixed(1)} km)
                    coinciden exactamente.
                  </span>
                ) : (
                  <span className="text-amber-700 dark:text-amber-300">
                    ⚠️ Atención: Los totales de Prim y Kruskal difieren.
```

```
                </span>
              )}
            </div>
          </div>

          {/* Selector de sub-pestañas: Prim, Kruskal, Lista de aristas */}
          <div className="flex rounded-lg bg-slate-100 p-1 dark:bg-slate-800"
role="tablist">
            <button
              type="button"
              role="tab"
              data-testid="tab-mst-prim"
              aria-selected={mstAlgorithmTab === 'prim'}
              onClick={() => setMstAlgorithmTab('prim')}
              className={`flex-1 min-h-[38px] rounded-md px-3 py-1.5 text-xs font-bold
transition-colors ${
                mstAlgorithmTab === 'prim'
                  ? 'bg-white text-emerald-700 shadow-xs dark:bg-slate-900 dark:text-
emerald-400'
                  : 'text-slate-600 hover:text-slate-900 dark:text-slate-400
dark:hover:text-white'
              }`}
```

```
              >
                Prim (Paso a paso)
              </button>
              <button
                type="button"
                role="tab"
                data-testid="tab-mst-kruskal"
                aria-selected={mstAlgorithmTab === 'kruskal'}
                onClick={() => setMstAlgorithmTab('kruskal')}
                className={`flex-1 min-h-[38px] rounded-md px-3 py-1.5 text-xs font-bold
transition-colors ${
                  mstAlgorithmTab === 'kruskal'
                    ? 'bg-white text-emerald-700 shadow-xs dark:bg-slate-900 dark:text-
emerald-400'
                    : 'text-slate-600 hover:text-slate-900 dark:text-slate-400
dark:hover:text-white'
                }`}
              >
                Kruskal (Aristas ordenadas)
              </button>
              <button
                type="button"
```

```
159                role="tab"
160                data-testid="tab-mst-edges"
161                aria-selected={mstAlgorithmTab === 'aristas'}
162                onClick={() => setMstAlgorithmTab('aristas')}
163                className={`flex-1 min-h-[38px] rounded-md px-3 py-1.5 text-xs font-bold
transition-colors ${
164                  mstAlgorithmTab === 'aristas'
165                    ? 'bg-white text-emerald-700 shadow-xs dark:bg-slate-900 dark:text-
emerald-400'
166                    : 'text-slate-600 hover:text-slate-900 dark:text-slate-400
dark:hover:text-white'
167                }`}
168              >
169                Carreteras del árbol ({mst?.prim.edges.length ?? 0})
170              </button>
171            </div>
172
173            {/* Contenido de la sub-pestaña activa */}
174            {mstAlgorithmTab === 'prim' && <PrimTab />}
175            {mstAlgorithmTab === 'kruskal' && <KruskalTab />}
176            {mstAlgorithmTab === 'aristas' && (
```

```
            <div className="overflow-x-auto rounded-xl border border-slate-200 shadow-xs dark:border-slate-800">
              <table className="w-full text-left border-collapse" data-testid="table-mst-edges">
                <thead>
                  <tr className="border-b border-slate-200 bg-slate-100 text-[11px] font-bold text-slate-700 uppercase dark:border-slate-700 dark:bg-slate-800 dark:text-slate-300">
                    <th className="py-2 px-3 w-10">#</th>
                    <th className="py-2 px-3">Conexión departamental</th>
                    <th className="py-2 px-3">Carretera</th>
                    <th className="py-2 px-3 text-right">km</th>
                  </tr>
                </thead>
                <tbody className="divide-y divide-slate-200 font-mono text-[11px] dark:divide-slate-800">
                  {mst?.prim.edges.map((edgeId, i) => {
                    const edge = graph.edges.find((e) => e.id === edgeId);
                    if (!edge) return null;
                    const src = graph.nodes.find((n) => n.id === edge.source);
                    const tgt = graph.nodes.find((n) => n.id === edge.target);
                    return (
                      <tr key={edgeId} className="hover:bg-slate-50 dark:hover:bg-slate-800/50">
```

```
195                         <td className="py-2 px-3 text-slate-500">{i + 1}</td>
196                         <td className="py-2 px-3 font-sans font-medium text-slate-800
dark:text-slate-200">
197                           {src?.label ?? edge.source} ⇔ {tgt?.label ?? edge.target}
198                         </td>
199                         <td className="py-2 px-3 font-sans text-slate-600 dark:text-slate-
400">
200                           {(typeof edge.meta?.carretera === 'string' ? edge.meta.carretera
: '') || '—'}
201                         </td>
202                         <td className="py-2 px-3 text-right font-bold text-slate-900
dark:text-slate-100">
203                           {edge.weight.toFixed(1)} km
204                         </td>
205                       </tr>
206                     );
207                   })}
208                 </tbody>
209               </table>
210             </div>
211           )}
212         </div>
```

```
213         )}
214
215         {/* CASO B: RUTAS MÁS CORTAS DESDE LA BODEGA (SPT) */}
216         {coverMode === 'spt' && (
217           <div className="flex flex-col gap-3" data-testid="spt-section">
218             <div className="flex flex-wrap items-center justify-between gap-2 rounded-xl
    border border-blue-200 bg-blue-50/70 p-3 dark:border-blue-900/40 dark:bg-blue-950/20">
219               <div className="flex items-center gap-2">
220                 <label htmlFor="select-spt-bodega" className="font-bold text-blue-950
    dark:text-blue-200">
221                   Bodega central (Origen):
222                 </label>
223                 <select
224                   id="select-spt-bodega"
225                   data-testid="select-spt-bodega"
226                   value={selectedBodega}
227                   onChange={(e) => {
228                     const id = e.target.value;
229                     setSelectedBodega(id);
230                     setTreeQuery(id);
231                   }}
```

```
                 className="min-h-[44px] rounded-lg border border-blue-300 bg-white px-3 py-1.5 text-xs font-semibold text-slate-800 shadow-xs focus-visible:ring-2 focus-visible:ring-blue-500 focus-visible:outline-hidden dark:border-slate-700 dark:bg-slate-800 dark:text-slate-200"
               >
                 {graph.nodes.map((n) => (
                   <option key={n.id} value={n.id}>
                     {n.label} ({departments[n.id]?.cabecera || n.label})
                   </option>
                 ))}
               </select>
             </div>
             <button
               type="button"
               data-testid="btn-apply-spt"
               onClick={() => setTreeQuery(selectedBodega)}
               className="min-h-[44px] rounded-lg bg-blue-600 px-3.5 py-1.5 text-xs font-bold text-white shadow-xs hover:bg-blue-700 focus-visible:ring-2 focus-visible:ring-blue-500"
             >
               Calcular árbol de rutas
             </button>
           </div>
```

```
250
251           {/* Tabla de distancias a los otros 21 departamentos */}
252           <div className="overflow-x-auto rounded-xl border border-slate-200 shadow-xs
dark:border-slate-800">
253             <table className="w-full text-left border-collapse" data-testid="table-spt-
distances">
254               <thead>
255                 <tr className="border-b border-slate-200 bg-slate-100 text-[11px] font-bold
text-slate-700 uppercase dark:border-slate-700 dark:bg-slate-800 dark:text-slate-300">
256                   <th className="py-2 px-3 w-10">#</th>
257                   <th className="py-2 px-3">Departamento destino</th>
258                   <th className="py-2 px-3">Cabecera</th>
259                   <th className="py-2 px-3 text-right">Distancia más corta</th>
260                 </tr>
261               </thead>
262               <tbody className="divide-y divide-slate-200 font-mono text-[11px]
dark:divide-slate-800">
263                 {bodegaDistances.map((item, idx) => (
264                   <tr key={item.id} className="hover:bg-slate-50 dark:hover:bg-slate-
800/50">
265                     <td className="py-2 px-3 text-slate-500">{idx + 1}</td>
266                     <td className="py-2 px-3 font-sans font-medium text-slate-800
dark:text-slate-200">{item.label}</td>
```

```
                    <td className="py-2 px-3 font-sans text-slate-600 dark:text-slate-400">
{item.cabecera}</td>

                    <td className="py-2 px-3 text-right font-bold text-blue-700 dark:text-
blue-300">

                      {item.dist === Infinity ? 'Inalcanzable' : `${item.dist.toFixed(1)}
km`}

                    </td>

                  </tr>

                ))}

              </tbody>

            </table>

          </div>

        </div>

      )}



      {/* CASO C: RECORRIDO DE REPARTO (APROXIMACIÓN HEURÍSTICA) */}

      {coverMode === 'tour' && (

        <div className="flex flex-col gap-3" data-testid="tour-section">

          {/* Aviso informativo de heurística */}

          <div className="rounded-xl border border-amber-200 bg-amber-50/80 p-3 text-xs
text-amber-950 dark:border-amber-900/40 dark:bg-amber-950/30 dark:text-amber-200">

            <div className="flex items-start gap-2">
```

```
              <span className="text-base">💡</span>
              <div>
                <p className="font-bold">Aproximación heurística (Problema del Repartidor / TSP)</p>
                <p className="mt-0.5 leading-relaxed text-slate-600 dark:text-slate-300">
                  Visita todos los departamentos alcanzables y regresa al punto de partida. Utiliza el método del
                  <strong> vecino más cercano</strong> seguido de refinamiento local <strong>2-opt</strong> sobre las
                  distancias mínimas de Dijkstra. No garantiza el óptimo global absoluto pero ofrece una ruta eficiente
                  en tiempo polinomial.
                </p>
              </div>
            </div>
          </div>

          {/* Selector de inicio del recorrido */}
          <div className="flex flex-wrap items-center justify-between gap-2 rounded-xl border border-slate-200 bg-slate-50 p-3 dark:border-slate-800 dark:bg-slate-900/60">
            <div className="flex items-center gap-2">
              <label htmlFor="select-tour-start" className="font-bold text-slate-800 dark:text-slate-200">
                Punto de partida:
```

```
              </label>
              <select
                id="select-tour-start"
                data-testid="select-tour-start"
                value={selectedTourStart}
                onChange={(e) => setSelectedTourStart(e.target.value)}
                className="min-h-[44px] rounded-lg border border-slate-300 bg-white px-3 py-1.5 text-xs font-semibold text-slate-800 shadow-xs focus-visible:ring-2 focus-visible:ring-blue-500 focus-visible:outline-hidden dark:border-slate-700 dark:bg-slate-800 dark:text-slate-200"
              >
                {graph.nodes.map((n) => (
                  <option key={n.id} value={n.id}>
                    {n.label} ({departments[n.id]?.cabecera || n.label})
                  </option>
                ))}
              </select>
            </div>

            <div className="flex items-center gap-2">
              {tour.status === 'running' ? (
```

```
                <button
                  type="button"
                  data-testid="btn-cancel-tour"
                  onClick={cancelTour}
                  className="min-h-[44px] rounded-lg bg-rose-600 px-4 py-1.5 text-xs font-bold text-white shadow-xs hover:bg-rose-700"
                >
                  Cancelar cálculo
                </button>
              ) : (
                <button
                  type="button"
                  data-testid="btn-run-tour"
                  onClick={() => runTour(selectedTourStart)}
                  className="min-h-[44px] rounded-lg bg-purple-600 px-4 py-1.5 text-xs font-bold text-white shadow-xs hover:bg-purple-700 focus-visible:ring-2 focus-visible:ring-purple-500"
                >
                  🚚 Calcular recorrido de reparto
                </button>
              )}
            </div>
```

```
340            </div>
341
342            {/* Estado del cálculo */}
343            {tour.status === 'running' && (
344              <div
345                className="flex items-center justify-center gap-2 py-6 text-sm text-slate-
500"
346                data-testid="tour-running"
347              >
348                <span className="inline-block h-4 w-4 animate-spin rounded-full border-2
border-purple-600 border-t-transparent" />
349                <span>Optimizando recorrido con vecino más cercano y 2-opt…</span>
350              </div>
351            )}
352
353            {tour.status === 'error' && (
354              <div
355                className="rounded-lg bg-rose-50 p-3 text-xs text-rose-700 dark:bg-rose-
950/40 dark:text-rose-300"
356                data-testid="tour-error"
357              >
```

```
              Error al calcular el recorrido: {tour.error}
            </div>
          )}

          {tour.status === 'done' && tour.result && (
            <div
              className="flex flex-col gap-3 rounded-xl border border-purple-200 bg-purple-50/60 p-3.5 dark:border-purple-900/50 dark:bg-purple-950/20"
              data-testid="tour-result-container"
            >
              <div className="flex items-center justify-between">
                <span className="font-bold text-purple-900 dark:text-purple-200">
                  Recorrido completo calculado ({tour.result.order.length} paradas):
                </span>
                <span data-testid="tour-total-km" className="text-xl font-black text-purple-700 dark:text-purple-300">
                  {tour.result.total.toFixed(1)} km
                </span>
              </div>

              {/* Lista numerada de paradas del recorrido */}
```

```
              <div className="overflow-x-auto rounded-lg border border-purple-200 bg-white
p-2.5 dark:border-purple-900/60 dark:bg-slate-900">

                <ol className="divide-y divide-slate-100 text-xs dark:divide-slate-800"
data-testid="tour-stops-list">

                  {tour.result.order.map((nodeId, idx) => (

                    <li key={`${nodeId}-${idx}`} className="flex items-center justify-
between py-1 px-1">

                      <span className="flex items-center gap-2">

                        <span className="flex h-5 w-5 items-center justify-center rounded-
full bg-purple-100 text-[10px] font-bold text-purple-800 dark:bg-purple-950 dark:text-
purple-300">

                          {idx + 1}

                        </span>

                        <span className="font-medium text-slate-800 dark:text-slate-200">
{getNodeName(nodeId)}</span>

                      </span>

                      {idx === 0 && (

                        <span className="rounded bg-emerald-100 px-1.5 py-0.5 text-[10px]
font-bold text-emerald-800 dark:bg-emerald-950 dark:text-emerald-300">

                          Salida

                        </span>

                      )}

                      {tour.result && idx === tour.result.order.length - 1 && (

                        <span className="rounded bg-rose-100 px-1.5 py-0.5 text-[10px]
font-bold text-rose-800 dark:bg-rose-950 dark:text-rose-300">
```

```
                          Retorno
                        </span>
                      )}
                    </li>
                  ))}
                </ol>
              </div>
            </div>
          )}
        </div>
      )}
    </div>
  );
}
```

src/ui/components/tabs/PrimTab.tsx

253 líneas.

```
/**
 * Pestaña de solución tabulada paso a paso del Algoritmo de Prim (AEM).
 * Muestra el proceso de corte iterativo: la arista mínima que cruza el corte hacia un nodo no visitado,
 * el nuevo nodo incorporado, las columnas tachadas y el total acumulado en kilómetros.
```

```
 *
 * Lee: `mst.prim`, `graph`, `departments`, `mstStart`, `setMstStart` desde `useApp`.
 */
import { useEffect, useState } from 'react';
import type { NodeId } from '../../../core/types';
import { useApp } from '../../useApp';

export default function PrimTab() {
  const mst = useApp((s) => s.mst);
  const graph = useApp((s) => s.graph);
  const departments = useApp((s) => s.departments);
  const mstStart = useApp((s) => s.mstStart);
  const setMstStart = useApp((s) => s.setMstStart);

  const prim = mst?.prim;
  const steps = prim?.steps ?? [];
  const totalSteps = steps.length;
  const lastStepIndex = Math.max(0, totalSteps - 1);

```

```
  const [stepOverride, setStepOverride] = useState<number | null>(null);
  const [isPlaying, setIsPlaying] = useState<boolean>(false);
  const [speedMs] = useState<number>(800);

  const currentStepIndex = stepOverride !== null ? Math.min(stepOverride, lastStepIndex) :
lastStepIndex;

  // Reproducción paso a paso
  useEffect(() => {
    if (!isPlaying) return;
    const timer = setInterval(() => {
      setStepOverride((prev) => {
        const current = prev !== null ? prev : lastStepIndex;
        if (current >= lastStepIndex) {
          setIsPlaying(false);
          return current;
        }
        return current + 1;
      });
    }, speedMs);
```

```
43      return () => clearInterval(timer);
44    }, [isPlaying, lastStepIndex, speedMs]);
45
46    if (!prim || steps.length === 0) {
47      return (
48        <div className="flex h-48 items-center justify-center p-6 text-center text-sm text-
slate-500 dark:text-slate-400">
49          <p>No hay cálculo de árbol de expansión mínima disponible.</p>
50        </div>
51      );
52    }
53
54    const getNodeName = (id: NodeId): string => {
55      const d = departments[id];
56      const n = graph.nodes.find((node) => node.id === id);
57      if (d?.cabecera && n?.label && d.cabecera !== n.label) {
58        return `${n.label} (${d.cabecera})`;
59      }
60      return n?.label ?? id;
61    };
```

```
62
63    const getEdgeInfo = (edgeId: string | null) => {
64      if (!edgeId) return null;
65      const edge = graph.edges.find((e) => e.id === edgeId);
66      if (!edge) return { id: edgeId, source: '', target: '', weight: 0, road: '' };
67      const srcName = graph.nodes.find((n) => n.id === edge.source)?.label ?? edge.source;
68      const tgtName = graph.nodes.find((n) => n.id === edge.target)?.label ?? edge.target;
69      const road = edge.meta?.carretera ? ` (${edge.meta.carretera})` : '';
70      return {
71        id: edge.id,
72        source: srcName,
73        target: tgtName,
74        weight: edge.weight,
75        road,
76      };
77    };
78
79    const currentStep = steps[currentStepIndex] ?? steps[0]!;
80
81    return (
```

```
    <div className="flex flex-col gap-3 p-3 text-xs" data-testid="prim-tab-container">
      {/* Controles superiores: selector de inicio y reproducción */}
      <div className="flex flex-wrap items-center justify-between gap-2 rounded-xl border border-slate-200 bg-slate-50 p-2.5 dark:border-slate-800 dark:bg-slate-900/60">
        {/* Selector de nodo inicial de Prim */}
        <div className="flex items-center gap-1.5">
          <label htmlFor="prim-start-select" className="font-semibold text-slate-700 dark:text-slate-300">
            Nodo inicial:
          </label>
          <select
            id="prim-start-select"
            data-testid="select-prim-start"
            value={mstStart ?? graph.nodes[0]?.id ?? ''}
            onChange={(e) => setMstStart(e.target.value)}
            className="min-h-[40px] rounded-lg border border-slate-300 bg-white px-2.5 py-1 text-xs font-medium text-slate-800 shadow-xs focus-visible:ring-2 focus-visible:ring-blue-500 focus-visible:outline-hidden dark:border-slate-700 dark:bg-slate-800 dark:text-slate-200"
          >
            {graph.nodes.map((n) => (
              <option key={n.id} value={n.id}>
```

```
                {n.label} {departments[n.id]?.cabecera ? `(${departments[n.id]?.cabecera})` : ''}
              </option>
            ))}
          </select>
        </div>

        {/* Controles de reproducción */}
        <div className="flex items-center gap-1">
          <button
            type="button"
            data-testid="btn-prim-prev"
            onClick={() => setStepOverride(Math.max(0, currentStepIndex - 1))}
            disabled={currentStepIndex <= 0}
            aria-label="Paso anterior de Prim"
            className="flex min-h-[38px] min-w-[38px] items-center justify-center rounded-lg border border-slate-300 bg-white font-bold text-slate-700 shadow-xs hover:bg-slate-100 disabled:opacity-40 dark:border-slate-700 dark:bg-slate-800 dark:text-slate-200"
            title="Paso anterior"
          >
            ◀
          </button>
```

```
            <button
              type="button"
              data-testid="btn-prim-play"
              onClick={() => {
                if (isPlaying) {
                  setIsPlaying(false);
                } else {
                  if (currentStepIndex >= totalSteps - 1) setStepOverride(0);
                  setIsPlaying(true);
                }
              }}
              aria-label={isPlaying ? 'Pausa' : 'Reproducir'}
              className="flex min-h-[38px] min-w-[38px] items-center justify-center rounded-
lg bg-emerald-600 font-bold text-white shadow-xs hover:bg-emerald-700 focus-visible:ring-2
focus-visible:ring-emerald-500"
              title={isPlaying ? 'Pausar' : 'Reproducir'}
            >
              {isPlaying ? '⏸' : '▶'}
            </button>
            <button
              type="button"
```

```
              data-testid="btn-prim-next"
              onClick={() => setStepOverride(Math.min(totalSteps - 1, currentStepIndex + 1))}
              disabled={currentStepIndex >= totalSteps - 1}
              aria-label="Paso siguiente de Prim"
              className="flex min-h-[38px] min-w-[38px] items-center justify-center rounded-lg border border-slate-300 bg-white font-bold text-slate-700 shadow-xs hover:bg-slate-100 disabled:opacity-40 dark:border-slate-700 dark:bg-slate-800 dark:text-slate-200"
              title="Paso siguiente"
            >
              ▶
            </button>
            <button
              type="button"
              data-testid="btn-prim-end"
              onClick={() => {
                setIsPlaying(false);
                setStepOverride(totalSteps - 1);
              }}
              aria-label="Ir al final de Prim"
              className="flex min-h-[38px] min-w-[38px] items-center justify-center rounded-lg border border-slate-300 bg-white font-bold text-slate-700 shadow-xs hover:bg-slate-100 dark:border-slate-700 dark:bg-slate-800 dark:text-slate-200"
```

```
155             title="Solución final"
156           >
157             ⏭
158           </button>
159           <span className="ml-2 font-mono text-xs font-semibold text-slate-600 dark:text-
slate-400">
160             Paso {currentStepIndex} de {totalSteps - 1}
161           </span>
162         </div>
163       </div>
164
165       {/* Resumen del paso activo */}
166       <div className="rounded-xl border border-emerald-200 bg-emerald-50/70 p-2.5 text-xs
text-emerald-950 dark:border-emerald-900/50 dark:bg-emerald-950/30 dark:text-emerald-200">
167         <div className="flex flex-wrap items-center justify-between gap-2">
168           <div>
169             <span className="font-bold">Iteración {currentStep.iter}: </span>
170             {currentStep.chosenEdge ? (
171               <span>
172                 Arista elegida:{' '}
173                 <strong className="underline">
```

```
                  {getEdgeInfo(currentStep.chosenEdge)?.source} ↔
{getEdgeInfo(currentStep.chosenEdge)?.target}

                </strong>{' '}

                ({getEdgeInfo(currentStep.chosenEdge)?.weight.toFixed(1)}
km{getEdgeInfo(currentStep.chosenEdge)?.road})

              </span>

            ) : (

              <span>

                Nodo inicial: <strong>{getNodeName(currentStep.inTree[0] ?? '')}</strong>

              </span>

            )}

          </div>

          <div className="font-bold">

            Total hasta aquí: <span className="font-mono text-sm">
{currentStep.totalSoFar.toFixed(1)} km</span>

          </div>

        </div>

      </div>



      {/* Tabla de pasos completa con fila actual resaltada */}

      <div className="overflow-x-auto rounded-xl border border-slate-200 shadow-xs
dark:border-slate-800">
```

```
192         <table className="w-full min-w-[620px] text-left border-collapse" data-
testid="table-prim-steps">
193           <thead>
194             <tr className="border-b border-slate-200 bg-slate-100 text-[11px] font-bold
text-slate-700 uppercase tracking-wider dark:border-slate-700 dark:bg-slate-800 dark:text-
slate-300">
195               <th className="py-2 px-2.5 w-10 text-center">#</th>
196               <th className="py-2 px-2.5">Arista mínima (Corte)</th>
197               <th className="py-2 px-2.5">Nodo incorporado</th>
198               <th className="py-2 px-2.5">Columnas tachadas</th>
199               <th className="py-2 px-2.5 text-right">km tramo</th>
200               <th className="py-2 px-2.5 text-right">Total acumulado</th>
201             </tr>
202           </thead>
203           <tbody className="divide-y divide-slate-200 font-mono text-[11px] dark:divide-
slate-800">
204             {steps.map((st, idx) => {
205               const isSelected = idx === currentStepIndex;
206               const edgeInfo = getEdgeInfo(st.chosenEdge);
207               const newlyAddedNode = st.inTree[st.inTree.length - 1];
208
209               return (
```

```
                <tr
                  key={st.iter}
                  onClick={() => setStepOverride(idx)}
                  className={`cursor-pointer transition-colors ${
                    isSelected
                      ? 'bg-emerald-100 font-semibold text-emerald-950 dark:bg-emerald-950/60 dark:text-emerald-100'
                      : idx % 2 === 0
                        ? 'bg-white hover:bg-slate-50 dark:bg-slate-900 dark:hover:bg-slate-800/60'
                        : 'bg-slate-50/60 hover:bg-slate-100 dark:bg-slate-800/30 dark:hover:bg-slate-800/60'
                  }`}
                  data-testid={`prim-row-${st.iter}`}
                >
                  <td className="py-2 px-2.5 text-center font-bold text-slate-600 dark:text-slate-400">{st.iter}</td>
                  <td className="py-2 px-2.5 font-sans">
                    {edgeInfo ? (
                      <span className="inline-flex items-center gap-1 rounded-md bg-emerald-500/15 px-1.5 py-0.5 font-semibold text-emerald-800 dark:text-emerald-300">
                        {edgeInfo.source} ↔ {edgeInfo.target}
                        {edgeInfo.road && <span className="text-[10px] text-slate-500">{edgeInfo.road}</span>}
```

```
                      </span>
                    ) : (
                      <span className="italic text-slate-400">Inicio (árbol vacío)</span>
                    )}
                  </td>
                  <td className="py-2 px-2.5 font-sans font-medium">
                    {newlyAddedNode ? getNodeName(newlyAddedNode) : '—'}
                  </td>
                  <td className="py-2 px-2.5 text-slate-600 dark:text-slate-400">
                    <span className="line-through">{st.crossedOut.join(', ')}</span>
                  </td>
                  <td className="py-2 px-2.5 text-right font-medium">
                    {edgeInfo ? `${edgeInfo.weight.toFixed(1)} km` : '0.0 km'}
                  </td>
                  <td className="py-2 px-2.5 text-right font-bold text-slate-900 dark:text-
slate-100">
                    {st.totalSoFar.toFixed(1)} km
                  </td>
                </tr>
              );
```

```
            })}
          </tbody>
        </table>
      </div>
    </div>
  );
}
```

src/ui/components/tabs/KruskalTab.tsx

139 líneas.

```
/**
 * Pestaña de solución tabulada del Algoritmo de Kruskal (AEM).
 * Muestra las aristas ordenadas de menor a mayor peso y la decisión de aceptación o
rechazo
 * (por formación de ciclo con Union-Find), junto con el total acumulado en kilómetros.
 *
 * Lee: `mst.kruskal`, `graph`, `departments` desde `useApp`.
 */
import { useApp } from '../../useApp';


export default function KruskalTab() {
  const mst = useApp((s) => s.mst);
```

```
  const graph = useApp((s) => s.graph);
  const departments = useApp((s) => s.departments);

  const kruskal = mst?.kruskal;
  const steps = kruskal?.steps ?? [];

  if (!kruskal || steps.length === 0) {
    return (
      <div className="flex h-48 items-center justify-center p-6 text-center text-sm text-slate-500 dark:text-slate-400">
        <p>No hay cálculo de Kruskal disponible.</p>
      </div>
    );
  }

  // Descartamos el paso inicial (iter 0) para mostrar directamente la evaluación de cada arista
  const evalSteps = steps.filter((s) => s.iter > 0);

  const getEdgeDetails = (edgeId: string) => {
    const edge = graph.edges.find((e) => e.id === edgeId);
```

```
    if (!edge) return { id: edgeId, source: '', target: '', weight: 0, road: '' };
    const src = graph.nodes.find((n) => n.id === edge.source);
    const tgt = graph.nodes.find((n) => n.id === edge.target);
    const srcName = src
      ? `${src.label}${departments[src.id]?.cabecera ? `
(${departments[src.id]?.cabecera})` : ''}`
      : edge.source;
    const tgtName = tgt
      ? `${tgt.label}${departments[tgt.id]?.cabecera ? `
(${departments[tgt.id]?.cabecera})` : ''}`
      : edge.target;
    return {
      id: edge.id,
      source: srcName,
      target: tgtName,
      weight: edge.weight,
      road: (typeof edge.meta?.carretera === 'string' ? edge.meta.carretera : '') || '—',
    };
  };

  const totalAccepted = kruskal.edges.length;
```

```
  const totalRejected = evalSteps.length - totalAccepted;


  return (
    <div className="flex flex-col gap-3 p-3 text-xs" data-testid="kruskal-tab-container">
      {/* Resumen estadístico de Kruskal */}
      <div className="grid grid-cols-2 gap-2 sm:grid-cols-4">
        <div className="rounded-xl border border-slate-200 bg-slate-50 p-2.5 dark:border-slate-800 dark:bg-slate-900/60">
          <div className="text-[11px] font-medium text-slate-500 dark:text-slate-400">Aristas evaluadas</div>
          <div className="text-lg font-bold text-slate-800 dark:text-slate-200">{evalSteps.length}</div>
        </div>
        <div className="rounded-xl border border-emerald-200 bg-emerald-50/70 p-2.5 dark:border-emerald-900/40 dark:bg-emerald-950/30">
          <div className="text-[11px] font-medium text-emerald-700 dark:text-emerald-300">Aceptadas (árbol)</div>
          <div className="text-lg font-bold text-emerald-800 dark:text-emerald-200">{totalAccepted}</div>
        </div>
        <div className="rounded-xl border border-rose-200 bg-rose-50/70 p-2.5 dark:border-rose-900/40 dark:bg-rose-950/30">
          <div className="text-[11px] font-medium text-rose-700 dark:text-rose-300">Rechazadas (ciclo)</div>
```

```
          <div className="text-lg font-bold text-rose-800 dark:text-rose-200">{totalRejected}</div>
        </div>
        <div className="rounded-xl border border-blue-200 bg-blue-50/70 p-2.5 dark:border-blue-900/40 dark:bg-blue-950/30">
          <div className="text-[11px] font-medium text-blue-700 dark:text-blue-300">Total del árbol</div>
          <div className="text-lg font-extrabold text-blue-800 dark:text-blue-200">{kruskal.total.toFixed(1)} km</div>
        </div>
      </div>

      {/* Tabla ordenada de aristas examinadas */}
      <div className="overflow-x-auto rounded-xl border border-slate-200 shadow-xs dark:border-slate-800">
        <table className="w-full min-w-[620px] text-left border-collapse" data-testid="table-kruskal-steps">
          <thead>
            <tr className="border-b border-slate-200 bg-slate-100 text-[11px] font-bold text-slate-700 uppercase tracking-wider dark:border-slate-700 dark:bg-slate-800 dark:text-slate-300">
              <th className="py-2 px-2.5 w-10 text-center">#</th>
              <th className="py-2 px-2.5">Arista (Origen ↔ Destino)</th>
              <th className="py-2 px-2.5">Carretera</th>
              <th className="py-2 px-2.5 text-right">km</th>
```

```
              <th className="py-2 px-2.5 text-center">Decisión Union-Find</th>
              <th className="py-2 px-2.5 text-right">Total acumulado</th>
            </tr>
          </thead>
          <tbody className="divide-y divide-slate-200 font-mono text-[11px] dark:divide-slate-800">
            {evalSteps.map((st) => {
              const edgeId = st.chosenEdge ?? st.rejectedEdge ?? '';
              const details = getEdgeDetails(edgeId);
              const isAccepted = Boolean(st.chosenEdge);

              return (
                <tr
                  key={st.iter}
                  className={`transition-colors ${
                    isAccepted
                      ? 'bg-emerald-50/50 hover:bg-emerald-100/70 dark:bg-emerald-950/20 dark:hover:bg-emerald-950/40'
                      : 'bg-rose-50/30 hover:bg-rose-100/40 dark:bg-rose-950/15 dark:hover:bg-rose-950/30 opacity-75'
                  }`}
                  data-testid={`kruskal-row-${st.iter}`}
```

```
                >
                  <td className="py-2 px-2.5 text-center font-bold text-slate-600
dark:text-slate-400">{st.iter}</td>
                  <td className="py-2 px-2.5 font-sans font-medium">
                    <span
                      className={
                        isAccepted
                          ? 'text-slate-900 dark:text-slate-100'
                          : 'line-through text-slate-500 dark:text-slate-400'
                      }
                    >
                      {details.source} ↔ {details.target}
                    </span>
                  </td>
                  <td className="py-2 px-2.5 font-sans text-slate-600 dark:text-slate-400">
{details.road}</td>
                  <td className="py-2 px-2.5 text-right font-medium">
{details.weight.toFixed(1)} km</td>
                  <td className="py-2 px-2.5 text-center font-sans">
                    {isAccepted ? (
                      <span className="inline-flex items-center gap-1 rounded-full bg-
emerald-100 px-2 py-0.5 text-[10px] font-bold text-emerald-800 dark:bg-emerald-950/80
dark:text-emerald-300">
```

```
                        ✓ Aceptada
                      </span>
                    ) : (
                      <span className="inline-flex items-center gap-1 rounded-full bg-rose-
100 px-2 py-0.5 text-[10px] font-bold text-rose-800 dark:bg-rose-950/80 dark:text-rose-
300">
                        ✗ Ciclo (rechazada)
                      </span>
                    )}
                  </td>
                  <td className="py-2 px-2.5 text-right font-bold text-slate-900 dark:text-
slate-100">
                    {st.totalSoFar.toFixed(1)} km
                  </td>
                </tr>
              );
            })}
          </tbody>
        </table>
      </div>
    </div>
```

```
    );
  }
```